

Μαρκάκης Κωνσταντίνος, 3 ^ο έτος	1084541
Τσοπάνογλου Χρυσή, 3 ^ο έτος	1084596
Μελιτσόπουλος Φώτιος, 3 ^ο έτος	1084600

Προκειμένου τα αρχεία να τρέξουν σωστά, παρακαλούμε να ενημερώσετε όλα τα absolute paths εντός των αρχείων sourceStatements.sql και bulkInserts.sql (ο κώδικας που συμπεριλαμβάνεται στα παραρτήματα κώδικα στο τέλος της παρούσας αναφοράς, καθώς και στο συμπιεσμένο αρχείο markakis_108541_melitsopoulos_1084600_tsoranoglou_1084596_B.zip δομείται με βάση τα absolute paths που εισηγάγαμε εμείς για να λειτουργεί σωστά το πρόγραμμα). Δυστυχώς, ενώ προσπαθήσαμε πάρα πολύ, δεν μπορέσαμε να βρούμε τρόπο ώστε τόσο ο κώδικας της Βάσης Δεδομένων travel agency, όσο και το GUI, να μπορεί να λειτουργήσει με relative paths.

Η παρούσα αναφορά δομείται ως εξής:

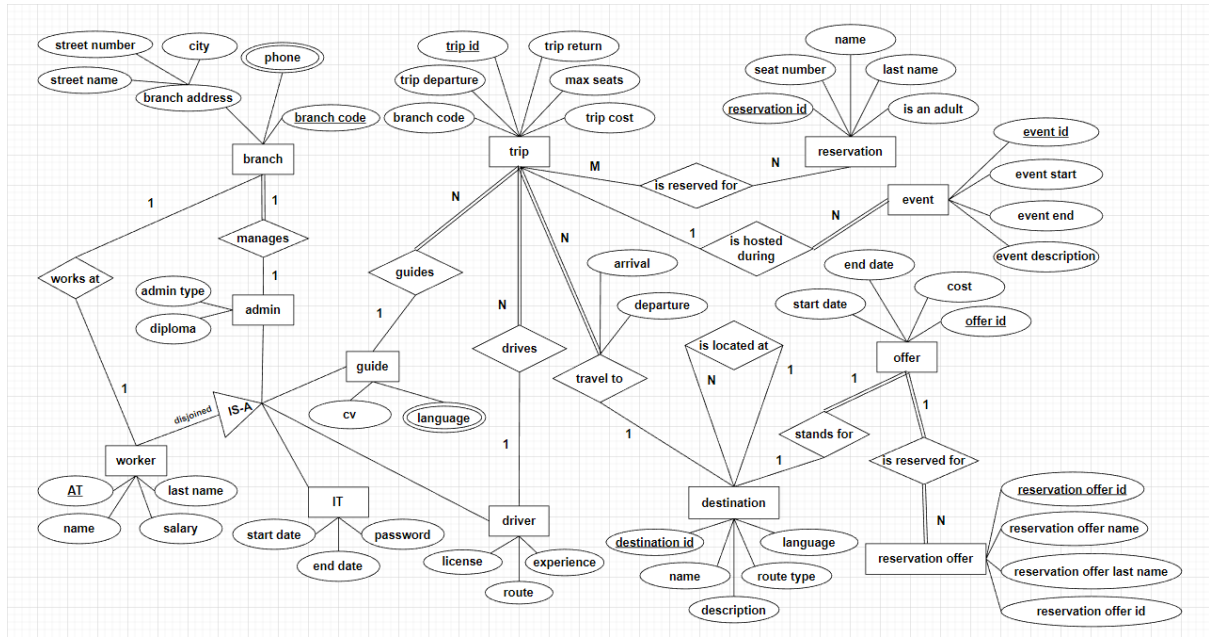
Μέρος Α : Σχεδιασμός ΒΔ (mySQL)	σελ. 2
• Κεφάλαιο 1 ^ο ER – Σχεδίαση Βάσης	σελ. 2
• Κεφάλαιο 2 ^ο Stored Procedures	σελ. 7
• Κεφάλαιο 3 ^ο Triggers	σελ. 12
Μέρος Β : GUI (JAVA)	σελ. 14
• Κεφάλαιο 4 ^ο GUI	σελ. 14
• Κεφάλαιο 5 ^ο Προερατικά	ΔΕΝ ΕΙΝΑΙ ΥΛΟΠΟΙΗΜΕΝΟ
Παράρτημα Κώδικα ΒΔ (MySQL)	σελ. 27
Παράρτημα Κώδικα GUI (Java)	σελ. 66

(!) Ο λόγος για τον οποίο τα παραρτήματα κώδικα βρίσκονται στο τέλος της αναφοράς και ο κώδικας δεν βρίσκεται διαιρεμένος και διάσπαρτος στα σημεία που αναφέρονταν στην εκφώνηση είναι κυρίως λόγω μεγέθους τόσο της υλοποίησης σε mySQL, όσο και της υλοποίησης σε Java. Ειδικά το τμήμα της Java, λόγω πολλαπλών προβλημάτων κατέληξε να είναι πολύ μεγαλύτερο σε μέγεθος, απ' ότι υπολογίζαμε σαν ομάδα.

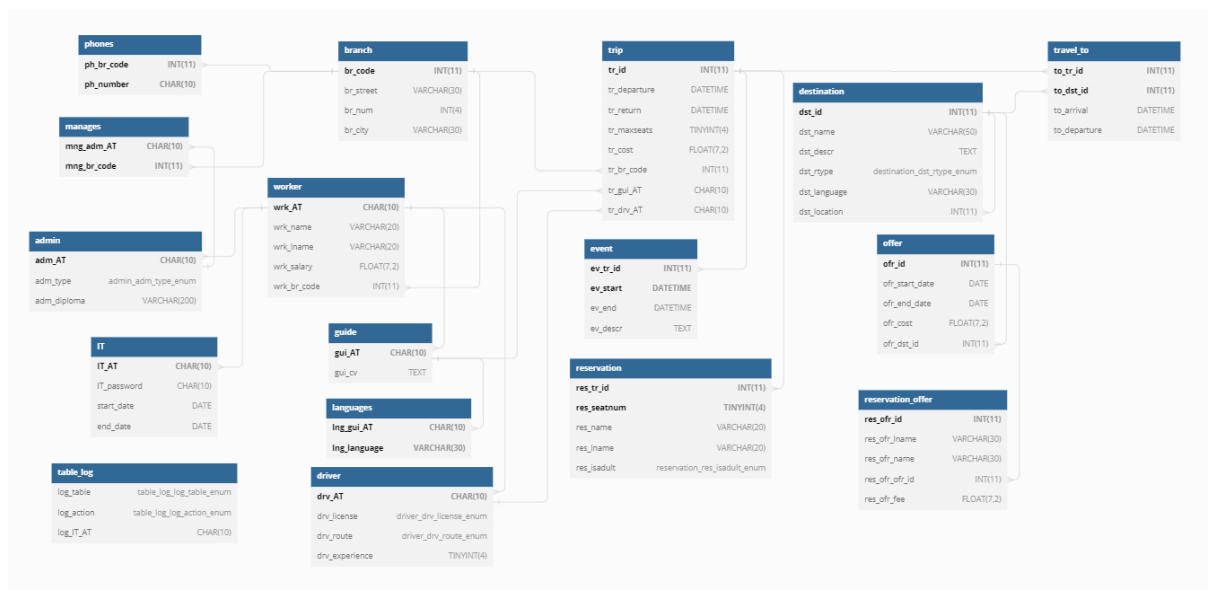
Μέρος Α : Σχεδιασμός ΒΔ (mySQL)

Κεφάλαιο 1^ο | Ε-R – Σχεδίαση Βάσης :

Τελικό Ε-R διάγραμμα ΒΔ :



Τελικό Σχεσιακό Μοντέλο ΒΔ :



(!) Όλοι οι πίνακες έχουν δημιουργηθεί ώστε να περιέχουν τα πεδία, όπως ορίζονται από την εκφώνηση. Σε αυτό το σημείο θα αναλυθούν μόνον οι πίνακες στους οποίους εμείς προσθέσαμε είτε επιπρόσθετα πεδία, είτε ειδικές ρυθμίσεις στα ήδη υπάρχοντα – ζητούμενα, από την εκφώνηση.

branch :

Πίνακας που περιέχει τα πεδία br_code, br_street, br_num & br_city. Το πεδίο br_code ορίζεται ως NOT NULL, καθώς το πεδίο αυτό αποτελεί primary key. Επιπλέον ορίζεται και ως AUTO_INCREMENT, λόγω ευκολίας στην εισαγωγή καινούριων εγγραφών στον πίνακα.

phones :

Πίνακας που περιέχει τα πεδία ph_br_code & ph_number. Και τα δύο πεδία ορίζονται ως NOT NULL καθώς ο συνδυασμός τους αποτελεί primary key. Στην περίπτωση διαγραφής ή ενημέρωσης του πεδίου br_code του πίνακα **branch**, η αλλαγή αυτή θα μεταφερθεί και σε όλες τις σχετικές εγγραφές του πίνακα **phones**, που αντιστοιχεί αυτή η τιμή εντός του πεδίου ph_br_code.

worker :

Πίνακας που περιέχει τα πεδία wrk_AT, wrk_name, wrk_lname, wrk_salary & wrk_br_code. Το πεδίο wrk_AT ορίζεται ως NOT NULL, εφόσον αποτελεί primary key. Τα πεδία wrk_name & wrk_lname ορίζονται ως DEFAULT 'unknown', επειδή σαν ομάδα, επιλέγουμε σχεδιαστικά να μην εμφανίζεται η τιμή NULL εντός του ονοματεπώνυμου, εντός των εγγραφών. Στην περίπτωση ενημέρωσης του πεδίου br_code του πίνακα **branch**, η ενημέρωση αυτή θα μεταφερθεί και σε όλες τις σχετικές εγγραφές του πίνακα **worker**, που αντιστοιχεί αυτή η τιμή εντός του πεδίου wrk_br_code. Σε περίπτωση διαγραφής οι αντίστοιχες εγγραφές θα λάβουν την τιμή NULL στο πεδίο wrk_br_code.

driver :

Πίνακας που περιέχει τα πεδία drv_AT, drv_license, drv_route & drv_experience. Το πεδίο drv_AT ορίζεται ως NOT NULL, εφόσον αποτελεί primary key. Τα πεδία drv_license & drv_route ορίζονται επίσης ως NOT NULL, ούτως ώστε να επιλέγεται πάντα μια από τις τιμές εντός του αντίστοιχου ENUM. Το πεδίο drv_experience ορίζεται ως DEFAULT 0, για να διευκολύνει την εισαγωγή εγγραφών που αντιστοιχούν σε άπειρους οδηγούς. Στην περίπτωση διαγραφής ή ενημέρωσης του πεδίου wrk_AT του πίνακα **worker**, η αλλαγή αυτή θα μεταφερθεί και στην σχετική εγγραφή του πίνακα **driver**, που αντιστοιχεί αυτή η τιμή εντός του πεδίου drv_AT.

admin :

Πίνακας που περιέχει τα πεδία adm_AT, adm_type & adm_diploma. Το πεδίο adm_AT ορίζεται ως NOT NULL, εφόσον αποτελεί primary key. Το πεδίο adm_type ορίζεται ως NOT NULL, ούτως ώστε να επιλέγεται πάντα μια από τις τιμές εντός του αντίστοιχου ENUM. Στην περίπτωση διαγραφής ή ενημέρωσης του πεδίου wrk_AT του πίνακα **worker**, η αλλαγή αυτή θα μεταφερθεί και στην σχετική εγγραφή του πίνακα **admin**, που αντιστοιχεί αυτή η τιμή εντός του πεδίου adm_AT.

manages :

Πίνακας που περιέχει τα πεδία mng_adm_AT & mng_br_code. Και τα δύο πεδία ορίζονται ως NOT NULL καθώς ο συνδυασμός τους αποτελεί primary key. Στην περίπτωση διαγραφής ή ενημέρωσης του πεδίου adm_AT του πίνακα **admin** ή του πεδίου br_code του πίνακα **branch**, η αλλαγή αυτή θα μεταφερθεί και στην σχετική εγγραφή του πίνακα **manages**, που αντιστοιχεί αυτή η τιμή εντός του πεδίου mng_adm_AT ή mng_br_code, αντίστοιχα.

guide :

Πίνακας που περιέχει τα πεδία gui_AT & gui_cn. Το πεδίο gui_AT ορίζεται ως NOT NULL, εφόσον αποτελεί primary key. Το πεδίο gui_cn ορίζεται ως NOT NULL, ώστε να μην υπάρχουν ημιτελείς εγγραφές στον πίνακα **guide**. Στην περίπτωση διαγραφής ή ενημέρωσης του πεδίου wrk_AT του πίνακα **worker**, η αλλαγή αυτή θα μεταφερθεί και στην σχετική εγγραφή του πίνακα **guide**, που αντιστοιχεί αυτή η τιμή εντός του πεδίου gui_AT.

languages :

Πίνακας που περιέχει τα πεδία lng_gui_AT & lng_language. Και τα δύο πεδία ορίζονται ως NOT NULL καθώς ο συνδυασμός τους αποτελεί primary key. Στην περίπτωση διαγραφής ή ενημέρωσης του πεδίου gui_AT του πίνακα **guide**, η αλλαγή αυτή θα μεταφερθεί και σε όλες τις σχετικές εγγραφές του πίνακα **languages**, που αντιστοιχεί αυτή η τιμή εντός του πεδίου lng_gui_AT.

IT :

Πίνακας που περιέχει τα πεδία IT_AT, IT_password, start_date & end_date. Το πεδίο IT_AT ορίζεται ως NOT NULL, εφόσον αποτελεί primary key. Το πεδίο IT_password ορίζεται ως NOT NULL, καθώς το περιεχόμενο αυτού, εφόσον χρησιμοποιείται σαν βάση της υλοποίησης του GUI, είναι αδύνατο να είναι NULL. Το πεδίο start_date ορίζεται ως NOT NULL, επειδή περιγράφει σε μορφή ημερομηνίας την αρχή της εργασίας του IT στο travel agency. Το πεδίο end_date ορίζεται ως DEFAULT NULL, εφόσον τυπικά, δεν γίνεται κάποιος να προσληφθεί και να απολυθεί στην ίδια ημέρα. Στην περίπτωση διαγραφής ή ενημέρωσης του πεδίου wrk_AT του πίνακα **worker**, η αλλαγή αυτή θα μεταφερθεί και στην σχετική εγγραφή του πίνακα **IT**, που αντιστοιχεί αυτή η τιμή εντός του πεδίου IT_AT.

trip :

Πίνακας που περιέχει τα πεδία tr_id, tr_departure, tr_return, tr_maxseats, tr_cost, tr_br_code, tr_gui_AT & tr_drv_AT. Το πεδίο tr_id ορίζεται ως NOT NULL, εφόσον αποτελεί primary key. Επιπλέον ορίζεται και ως AUTO_INCREMENT, λόγω ευκολίας στην εισαγωγή καινούριων εγγραφών στον πίνακα. Τα πεδία tr_departure, tr_maxseats και tr_br_code ορίζονται ως NOT NULL, από δική μας σχεδιαστική επιλογή, καθώς θεωρούμε πως για να εισαχθεί ένα ταξίδι στον πίνακα **trip**, πρέπει τουλάχιστον να είναι γνωστή η ημερομηνία αποχώρησης (tr_departure), το μέγιστο πλήθος ατόμων που μπορούν να συμμετάσχουν (tr_maxseats), καθώς και το ταξιδιωτικό γραφείο το οποίο είναι υπεύθυνο (tr_br_code). Στην περίπτωση διαγραφής ή ενημέρωσης του πεδίου br_code του πίνακα **branch** ή του πεδίου gui_AT του πίνακα **guide** ή του πεδίου drv_AT του πίνακα **driver**, η αλλαγή αυτή θα μεταφερθεί και σε όλες τις σχετικές εγγραφές του πίνακα **trip**, που αντιστοιχεί αυτή η τιμή εντός του πεδίου tr_br_code ή tr_gui_AT ή tr_drv_AT, αντίστοιχα.

event :

Πίνακας που περιέχει τα πεδία `ev_tr_id`, `ev_start`, `ev_end` & `ev_descr`. Τα πεδία `ev_tr_id` & `ev_start` ορίζονται ως NOT NULL, καθώς ο συνδυασμός τους αποτελεί primary key. Τα πεδία `ev_end` & `ev_descr` ορίζονται ως NOT NULL, καθώς θεωρούμε πως ένα event θα είναι καλώς ορισμένο, συνεπώς δεν πρόκειται να εισαχθούν ποτέ στον πίνακα event ημιτελείς εγγραφές. Στην περίπτωση διαγραφής ή ενημέρωσης του πεδίου `tr_id` του πίνακα **trip**, η αλλαγή αυτή θα μεταφερθεί και στην σχετική εγγραφή του πίνακα **event**, που αντιστοιχεί αυτή η τιμή εντός του πεδίου `ev_tr_id`.

destination :

Πίνακας που περιέχει τα πεδία `dst_id`, `dst_name`, `dst_descr` & `dst_rtype`, `dst_language` & `dst_location`. Το πεδίο `dst_id` ορίζεται ως NOT NULL, εφόσον αποτελεί primary key. Επιπλέον ορίζεται και ως AUTO_INCREMENT, λόγω ευκολίας στην εισαγωγή καινούριων εγγραφών στον πίνακα. Τα πεδία `dst_rtype` & `dst_language` επίσης ορίζονται ως NOT NULL, από δική μας σχεδιαστική επιλογή, καθώς θεωρούμε πως για να εισαχθεί ένας προορισμός στον πίνακα **destination**, πρέπει να είναι γνωστός ο τύπος της διαδρομής για τον προορισμό (`dst_rtype`), καθώς και η γλώσσα που μιλούν εκεί (`dst_language`). Στην περίπτωση διαγραφής ή ενημέρωσης του πεδίου `dst_id` του πίνακα **destination**, η αλλαγή αυτή θα μεταφερθεί και στην σχετική εγγραφή του πίνακα **destination** (αναδρομική αναφορά), που αντιστοιχεί αυτή η τιμή εντός του πεδίου `dst_id`.

travel_to :

Πίνακας που περιέχει τα πεδία `to_tr_id`, `to_dst_id`, `to_arrival` & `to_departure`. Τα πεδία `to_tr_id` & `to_dst_id` ορίζονται ως NOT NULL, καθώς ο συνδυασμός τους αποτελεί primary key. Στην περίπτωση διαγραφής ή ενημέρωσης του πεδίου `tr_id` του πίνακα **trip** ή του πεδίου `dst_id` του πίνακα **destination**, η αλλαγή αυτή θα μεταφερθεί και σε όλες τις σχετικές εγγραφές του πίνακα **travel_to**, που αντιστοιχεί αυτή η τιμή εντός του πεδίου `to_tr_id` ή `to_dst_id`, αντίστοιχα.

reservation :

Πίνακας που περιέχει τα πεδία `res_tr_id`, `res_seatnum`, `res_name`, `res_lname`, `res_isadult`. Τα πεδία `res_tr_id` & `res_seatnum` ορίζονται ως NOT NULL, καθώς ο συνδυασμός τους αποτελεί primary key. Τα πεδία `res_name`, `res_lname` & `res_isadult` επίσης ορίζονται ως NOT NULL, από δική μας σχεδιαστική επιλογή, καθώς θεωρούμε πως για να εισαχθεί μια κράτηση στον πίνακα **reservation**, πρέπει να είναι γνωστό το ονοματεπώνυμο του ατόμου που πραγματοποιεί την κράτηση (`res_name`, `res_lname`), καθώς και το αν είναι ενήλικας ή όχι (`res_isadult`). Στην περίπτωση διαγραφής ή ενημέρωσης του πεδίου `tr_id` του πίνακα **trip**, η αλλαγή αυτή θα μεταφερθεί και σε όλες τις σχετικές εγγραφές του πίνακα **reservation**, που αντιστοιχεί αυτή η τιμή εντός του πεδίου `res_tr_id`.

current_active_user :

Πίνακας που περιέχει το πεδίο `user_IT_AT`. Δεν περιέχει κλειδί καθώς λειτουργεί ως buffer διασύνδεσης του GUI με την ΒΔ, αποθηκεύοντας κάθε φορά τον αριθμό ταυτότητας του εκάστοτε IT worker που έχει συνδεθεί μέσω GUI.

offer :

Πίνακας που περιέχει τα πεδία `ofr_id`, `ofr_start_date`, `ofr_end_date`, `ofr_cost` & `ofr_dst_id`. Το πεδίο `ofr_id` ορίζεται ως NOT NULL, εφόσον αποτελεί primary key. Επιπλέον ορίζεται και ως AUTO_INCREMENT, λόγω ευκολίας στην εισαγωγή καινούριων εγγραφών στον πίνακα. Τα πεδία `ofr_start_date`, `ofr_end_date`, `ofr_cost` & `ofr_dst_id` επίσης ορίζονται ως NOT NULL, από δική μας σχεδιαστική επιλογή, καθώς θεωρούμε πως για να εισαχθεί μια προσφορά στον πίνακα **offer**, πρέπει να είναι γνωστές οι ημερομηνίες έναρξης & λήξης της προσφοράς (`ofr_start_date`, `ofr_end_date`), το κόστος της προσφοράς (`ofr_cost`), καθώς και ο προορισμός για τον οποίο ισχύει η προσφορά (`ofr_dst_id`). Στην περίπτωση διαγραφής ή ενημέρωσης του πεδίου `dst_id` του πίνακα **destination**, η αλλαγή αυτή θα μεταφερθεί και σε όλες τις σχετικές εγγραφές του πίνακα **offer**, που αντιστοιχεί αυτή η τιμή εντός του πεδίου `ofr_dst_id`.

reservation_offer :

Πίνακας που περιέχει τα πεδία `res_ofr_id`, `res_ofr_lname`, `res_ofr_name`, `res_ofr_ofr_id` & `res_ofr_fee`. Το πεδίο `res_ofr_id` ορίζεται ως NOT NULL, εφόσον αποτελεί primary key. Επιπλέον ορίζεται και ως AUTO_INCREMENT, λόγω ευκολίας στην εισαγωγή καινούριων εγγραφών στον πίνακα. Τα πεδία `res_ofr_lname`, `res_ofr_name` ορίζονται ως DEFAULT 'unknown', επειδή σαν ομάδα, επιλέγουμε σχεδιαστικά να μην εμφανίζεται η τιμή NULL εντός του ονοματεπώνυμου, εντός των εγγραφών. Ακόμα, τα πεδία `res_ofr_lname`, `res_ofr_name`, `res_ofr_ofr_id` & `res_ofr_fee` επίσης ορίζονται ως NOT NULL, από δική μας σχεδιαστική επιλογή, καθώς θεωρούμε πως για να εισαχθεί μια κράτηση προσφοράς στον πίνακα **reservation_offer**, πρέπει να είναι γνωστό το ονοματεπώνυμο του ατόμου που πραγματοποιεί την κράτηση (`res_ofr_lname`, `res_ofr_name`), η προσφορά στην οποία κάνουν κράτηση (`res_ofr_ofr_id`), καθώς και το κόστος της προσφοράς (`res_ofr_fee`).

table_log :

Πίνακας που περιέχει τα πεδία `log_table`, `log_action` & `log_IT_AT`. Τα πεδία `log_table`, `log_action` & `log_IT_AT` ορίζονται ως NOT NULL, από δική μας σχεδιαστική επιλογή, καθώς θεωρούμε πως για να εισαχθεί μια εγγραφή στον πίνακα **log**, θα πρέπει να είναι γνωστός ο πίνακας πάνω στον οποίο έγινε η πράξη (`log_table`), η πράξη η ίδια (`log_action`), καθώς και το άτομο που την πραγματοποίησε (`log_IT_AT`).

(!) Σε αυτό το σημείο αξίζει να σημειωθεί πως ο πίνακας **table_log** δεν περιέχει primary key. Αυτό συμβαίνει διότι, εφόσον δεν ζητείται κάποιας μορφής χρήση του πίνακα **table_log**, πέρα από την εμφάνιση όλων των περιεχομένων του, καθώς και δεν υπάρχει κάποια αναφορά σε αντίστοιχο πεδίο εντός της εκφώνησης του project, σαν σχεδιαστική επιλογή αποφασίσαμε να μην το συμπεριλάβουμε. Κατανοούμε πως αυτό δεν αντιπροσωπεύει πλήρως "πραγματικές" συνθήκες, ωστόσο, ενδεχομένως να το είχαμε υλοποιήσει, εφόσον υπήρχε ανάγκη από πλευρά υπόλοιπης υλοποίησης, ή ζητούμενων της εκφώνησης.

(!) Ακόμα, το πεδίο `log_IT_AT` δεν κάνει reference τον πίνακα **IT**, τουλάχιστον ρητά, από πλευρά `mysql`, ώστε να υπάρχει μια επαλήθευση των τιμών εντός του πεδίου `log_IT_AT`, σε σχέση με τις τιμές `IT_AT` εντός του πίνακα **IT**. Αυτό συμβαίνει διότι, ο τρόπος διαχείρισης και διασύνδεσης μεταξύ της βάσης δεδομένων `travel_agency` από την πλευρά της `mysql` και του `GUI` από την πλευρά της `JAVA`, γίνεται με τέτοιο τρόπο, ώστε να μην χρειάζεται αυτή η ρητή δήλωση (βλέπε πίνακα **current_active_user**).

Κεφάλαιο 2° | Stored Procedures :

`assign_driver () {} :`

Stored procedure το οποίο δέχεται ως ορίσματα τα στοιχεία ενός καινούριου driver, τον εισάγει στον πίνακα worker, driver, καθώς και στο branch με τους ελάχιστους drivers.

Ειδικότερα, με το πρώτο SELECT query επιλέγουμε το ελάχιστο πλήθος των drivers ανά branch, ενώ με το δεύτερο επιλέγουμε τον πρώτο κωδικό branch που έχει το ελάχιστο αυτό πλήθος. Στη συνέχεια, εισάγουμε την καινούρια εγγραφή εντός των πινάκων **worker** & **driver**, βάσει τον ορισμάτων που λαμβάνει το procedure.

```
mysql> select count(*) as 'number_of_drivers'
-> from branch
-> inner join worker
-> on br_code = wrk_br_code
-> inner join driver
-> on wrk_AT = drv_AT
-> group by br_code;
+-----+
| number_of_drivers |
+-----+
|                1 |
|                1 |
|                2 |
|                1 |
|                1 |
|                2 |
+-----+
6 rows in set (0.00 sec)
```

```
mysql> call assign_driver('0101010102', 'Mixelis', 'Karagiannis', 36000, 'B', 'ABROAD', 5);
Query OK, 1 row affected (0.00 sec)

mysql> select count(*) as 'number_of_drivers'
-> from branch
-> inner join worker
-> on br_code = wrk_br_code
-> inner join driver
-> on wrk_AT = drv_AT
-> group by br_code;
+-----+
| number_of_drivers |
+-----+
|                2 |
|                1 |
|                2 |
|                1 |
|                1 |
|                2 |
+-----+
6 rows in set (0.00 sec)
```

```
mysql> call assign_driver('0101010101', 'Vaggelis', 'Kostis', 35000, 'A', 'LOCAL', 4);
Query OK, 1 row affected (0.00 sec)

mysql> select count(*) as 'number_of_drivers'
-> from branch
-> inner join worker
-> on br_code = wrk_br_code
-> inner join driver
-> on wrk_AT = drv_AT
-> group by br_code;
+-----+
| number_of_drivers |
+-----+
|                2 |
|                2 |
|                2 |
|                1 |
|                1 |
|                2 |
+-----+
6 rows in set (0.00 sec)
```

check_branch_trips () {} :

Stored procedure το οποίο δέχεται ως όρισμα τον κωδικό ενός branch, καθώς και δύο ημερομηνίες. Επιστρέφει το κόστος, το πλήθος των διαθέσιμων και μη θέσεων, καθώς και το ονοματεπώνυμο των driver και guide, για κάθε ταξίδι που διοργανώνεται από το συγκεκριμένο branch, του οποίου η ημερομηνία αναχώρησης βρίσκεται ανάμεσα στις δύο ημερομηνίες.

```
mysql> select *
-> from trip
-> order by tr_br_code asc;
+-----+-----+-----+-----+-----+-----+-----+-----+
| tr_id | tr_departure | tr_return | tr_maxseats | tr_cost | tr_br_code | tr_gui_AT | tr_drv_AT |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 4 | 2023-04-21 12:15:00 | 2023-04-28 10:00:00 | 94 | 2345.00 | 3 | NULL | 0000000016 |
| 7 | 2023-08-27 08:15:00 | 2023-08-31 10:45:00 | 119 | 6578.00 | 5 | 0000000030 | NULL |
| 1 | 2023-02-17 11:30:00 | NULL | 96 | 3400.00 | 6 | 0000000022 | 0000000000 |
| 3 | 2023-04-05 10:00:00 | 2023-04-18 11:45:00 | 78 | NULL | 8 | 0000000022 | 0000000019 |
| 8 | 2023-03-06 12:30:00 | 2023-03-12 11:55:00 | 120 | 500.00 | 8 | 0000000024 | 0000000028 |
| 6 | 2023-07-13 14:45:00 | 2023-07-19 10:00:00 | 87 | 3835.00 | 9 | 0000000020 | 0000000028 |
| 5 | 2023-05-16 18:00:00 | 2023-05-26 10:15:00 | 35 | 9678.00 | 11 | 0000000027 | 0000000010 |
| 2 | 2023-03-15 12:00:00 | 2023-03-21 10:00:00 | 123 | 8745.00 | 12 | 0000000015 | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+
8 rows in set (0.00 sec)
```

```
mysql> call check_branch_trips(6, '2022-01-01 00:00:00', '2025-01-01 00:00:00');
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| tr_cost | tr_maxseats | reservations | empty seats | Driver lname | Driver name | Guide lname | Guide name | tr_departure | tr_return |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 3400.00 | 96 | 2 | 94 | Prodromou | Elpida | Lampropoulos | Marios | 2023-02-17 11:30:00 | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)

mysql> call check_branch_trips(1, '2022-01-01 00:00:00', '2025-01-01 00:00:00');
Empty set (0.00 sec)
```

```
mysql> call check_branch_trips(6, '2022-01-01 00:00:00', '2022-01-02 00:00:00');
Empty set (0.00 sec)
```


delete_admin () {} :

Stored procedure το οποίο δέχεται σαν όρισμα ένα ονοματεπώνυμο, και αν αυτό αντιστοιχεί σε κάποιον admin, τότε τον διαγράφει, εκτός και αν ο admin κάνει manage ένα branch, στην οποία περίπτωση ο χρήστης ενημερώνεται με κατάλληλο μήνυμα.

Ειδικότερα, με το πρώτο SELECT query επιλέγουμε τον αριθμό ταυτότητας του admin, του οποίου το ονοματεπώνυμο ταυτίζεται με αυτό που δέχεται το procedure σαν όρισμα. Αν ο αριθμός ταυτότητας είναι NULL τότε δεν υπάρχει admin με τέτοιο όνομα, συνεπώς και το procedure τερματίζει, διαφορετικά, με το δεύτερο SELECT query ελέγχουμε αν ο αριθμός ταυτότητας του admin αυτού, βρίσκεται εντός του πίνακα **manages**. Αν ναι, τότε εμφανίζεται κατάλληλο μήνυμα ενημερώνοντας τον χρήστη, διαφορετικά ο admin διαγράφεται από τον πίνακα **admin**.

```
mysql> select *
-> from admin
-> inner join worker
-> on adm_AT = wrk_AT
-> left join manages
-> on mng_adm_AT = adm_AT
-> where wrk_lname = "Anastopoulou";
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| adm_AT | adm_type | adm_diploma | wrk_AT | wrk_name | wrk_lname | wrk_salary | wrk_br_code | mng_adm_AT | mng_br_code |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 0000000013 | ADMINISTRATIVE | Marketing | 0000000013 | Lydia | Anastopoulou | 72000.00 | 6 | 0000000013 | 12 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> call delete_admin("Lydia", "Anastopoulou");
+-----+-----+
| Message |
+-----+-----+
| This admin manages a branch, so he cannot be deleted. |
+-----+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

```
mysql> select *
-> from admin
-> inner join worker
-> on adm_AT = wrk_AT
-> left join manages
-> on mng_adm_AT = adm_AT
-> where wrk_lname = "Oikonomou";
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| adm_AT | adm_type | adm_diploma | wrk_AT | wrk_name | wrk_lname | wrk_salary | wrk_br_code | mng_adm_AT | mng_br_code |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 0000000002 | ACCOUNTING | Economics | 0000000002 | Lazaros | Oikonomou | NULL | NULL | NULL | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> call delete_admin("Lazaros", "Oikonomou");
+-----+-----+
| Message |
+-----+-----+
| Deleted successfully! |
+-----+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

```
mysql> select *
-> from admin
-> inner join worker
-> on adm_AT = wrk_AT
-> left join manages
-> on mng_adm_AT = adm_AT
-> where wrk_lname = "Kexagia";
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| adm_AT | adm_type | adm_diploma | wrk_AT | wrk_name | wrk_lname | wrk_salary | wrk_br_code | mng_adm_AT | mng_br_code |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 0000000021 | ACCOUNTING | NULL | 0000000021 | Niki | Kexagia | NULL | 12 | NULL | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> call delete_admin("Niki", "Kexagia");
+-----+-----+
| Message |
+-----+-----+
| Deleted successfully! |
+-----+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql> select *
-> from admin
-> inner join worker
-> on adm_AT = wrk_AT
-> left join manages
-> on mng_adm_AT = adm_AT
-> where wrk_lname = "Kexagia";
Empty set (0.00 sec)
```

find_reservation_offer_by_fee () {} :

Stored procedure το οποίο δέχεται σαν όρισμα δύο χρηματικές τιμές, και εμφανίζει το ονοματεπώνυμο όλων όσων έχουν πραγματοποιήσει κράτηση προσφοράς, και το κόστος αυτής βρίσκεται εντός του φάσματος που ορίζουν οι δύο χρηματικές τιμές.

```
mysql> CALL find_reservation_offer_by_fee(50,70);
```

res_ofr_lname	res_ofr_name
Buck	Shirley
Gibbs	Brooklynn
Berger	Sienna
Bryan	Lane
Cisneros	Gianna
Franklin	Ellie
Drake	Zaid
Cabrera	Kenyon
Duke	Valentina
Park	Tony
Silva	Shawn
Patel	Christina
Willis	Noah
Brennan	Isis
Moore	Nathen
Mccann	Darius
Ballard	Kailyn
Bauer	Tyler
Faulkner	Belinda
Key	Andreas
Forbes	Nola
Hooper	Julian
Wang	Cordell
Bright	Karina
Dixon	Dayana
Russell	Alvaro
Mcguire	Mira
Weiss	Julian
Huff	Maurice
Parrish	Bennett
Blanchard	Celia
Montes	Alanna
Foley	Lisa
Wilkinson	Jasper
Anderson	Adriel
Harris	Roman
Sullivan	Brooklyn
Thompson	Valentina
Hess	Daniela
Bowen	Eva
Pugh	Jaylyn
Moran	Danielle
Moore	Royce
Haney	Donte
Ritter	Jaylen
Horne	Isabell
Molina	Magdalena
Long	Angelique
Martin	Lyric
Pacheco	Russell
Velasquez	Brielle
Hurst	Cameron
Eaton	Liana
Johnston	Rogelio

find_reservation_by_last_name () {} :

Stored procedure το οποίο δέχεται σαν όρισμα ένα επώνυμο πελάτη και εμφανίζει το ονοματεπώνυμο του πελάτη, το αναγνωριστικό της κράτησης προσφοράς, καθώς και το πλήθος των κρατήσεων που έχει πραγματοποιήσει ο πελάτης.

```
C:\Windows\system32\cmd.exe - mysql -u root -p
mysql> CALL find_reservation_by_last_name('Tate');
+-----+-----+-----+-----+
| res_ofr_lname | res_ofr_name | res_ofr_ofr_id | count(*) |
+-----+-----+-----+-----+
| Tate          | Milagros     | 1               | 2         |
| Tate          | Nola         | 1               | 2         |
| Tate          | Karlee       | 1               | 2         |
| Tate          | Romeo       | 1               | 2         |
| Tate          | Eddie       | 1               | 2         |
| Tate          | Myah        | 1               | 2         |
| Tate          | Marques     | 1               | 2         |
| Tate          | Trinity     | 1               | 2         |
| Tate          | Cannon      | 1               | 2         |
| Tate          | Jorge       | 1               | 2         |
| Tate          | Beatrice    | 1               | 2         |
| Tate          | Asher       | 1               | 2         |
| Tate          | Kieran      | 1               | 2         |
| Tate          | Caitlyn     | 1               | 2         |
| Tate          | Milagros    | 2               | 2         |
| Tate          | Nola        | 2               | 2         |
| Tate          | Karlee      | 2               | 2         |
| Tate          | Romeo       | 2               | 2         |
| Tate          | Eddie       | 2               | 2         |
| Tate          | Myah        | 2               | 2         |
| Tate          | Marques     | 2               | 2         |
| Tate          | Trinity     | 2               | 2         |
| Tate          | Cannon      | 2               | 2         |
| Tate          | Jorge       | 2               | 2         |
| Tate          | Beatrice    | 2               | 2         |
| Tate          | Asher       | 2               | 2         |
| Tate          | Kieran      | 2               | 2         |
| Tate          | Caitlyn     | 2               | 2         |
| Tate          | Milagros    | 3               | 2         |
| Tate          | Nola        | 3               | 2         |
| Tate          | Karlee      | 3               | 2         |
| Tate          | Romeo       | 3               | 2         |
| Tate          | Eddie       | 3               | 2         |
| Tate          | Myah        | 3               | 2         |
| Tate          | Marques     | 3               | 2         |
| Tate          | Trinity     | 3               | 2         |
| Tate          | Cannon      | 3               | 2         |
| Tate          | Jorge       | 3               | 2         |
| Tate          | Beatrice    | 3               | 2         |
| Tate          | Asher       | 3               | 2         |
| Tate          | Kieran      | 3               | 2         |
| Tate          | Caitlyn     | 3               | 2         |
+-----+-----+-----+-----+
42 rows in set (0.00 sec)

Query OK, 0 rows affected (0.10 sec)

mysql> _
```

Αιτιολόγηση επιλογής index :

Τα παραπάνω stored procedures υλοποιήθηκαν δοκιμαστικά με 3 διαφορετικούς τρόπους (χωρίς index, B-tree index, hashing index), ώστε να συγκριθεί η απόκρισή τους σε queries πάνω στην ΒΔ. Εικότερα, επιλέγεται η υλοποίηση με B-tree index, λόγω ταχύτητας, όπως φαίνεται πειραματικά παρακάτω:

Για το stored procedure call find_reservation_offer_by_fee (50, 70) :

Είδος Index	Χρόνος Εκτέλεσης Stored Procedure (ms)		
	Χωρίς Index	B-Tree Index	Hashing Index
Εκτέλεση # 01	38,256	77,941	78,312
Εκτέλεση # 02	38,388	36,983	22,928
Εκτέλεση # 03	25,960	14,183	14,740
Εκτέλεση # 04	25,720	14,036	14,437
Εκτέλεση # 05	24,127	14,701	36,600
Εκτέλεση # 06	62,429	14,383	13,989
Εκτέλεση # 07	54,136	15,686	14,088
Εκτέλεση # 08	37,710	14,798	35,956
Εκτέλεση # 09	28,920	14,912	14,372
Εκτέλεση # 10	43,734	14,676	15,351
Μέσος Όρος	37,998 ≈ 38,0	23,230 ≈ 23,2	26,077 ≈ 26,1

*Βελτίωση B-tree : $(MO_X - MO_B) / MO_X \times 100 \% = (38,0 - 23,2) / 38,0 \times 100 \% = 38,9 \%$.

*Βελτίωση Hashing : $(MO_X - MO_H) / MO_X \times 100 \% = (38,0 - 26,1) / 38,0 \times 100 \% = 31,3 \%$.

Για το stored procedure call find_reservation_by_last_name ("Tate") :

Είδος Index	Χρόνος Εκτέλεσης Stored Procedure (ms)		
	Χωρίς Index	B-Tree Index	Hashing Index
Εκτέλεση # 01	60,936	10,103	55,306
Εκτέλεση # 02	60,447	01,635	80,339
Εκτέλεση # 03	73,465	01,579	92,083
Εκτέλεση # 04	73,656	01,540	77,013
Εκτέλεση # 05	60,846	01,787	63,594
Εκτέλεση # 06	67,682	01,553	67,698
Εκτέλεση # 07	78,908	01,884	75,954
Εκτέλεση # 08	54,160	01,638	59,308
Εκτέλεση # 09	60,221	01,525	73,344
Εκτέλεση # 10	53,239	01,752	82,530
Μέσος Όρος	64,357 ≈ 64,4	02,500 = 02,5	72,717 ≈ 72,7

*Βελτίωση B-tree : $(MO_X - MO_B) / MO_X \times 100 \% = (64,4 - 02,5) / 64,4 \times 100 \% = 96,1 \%$.

*Βελτίωση Hashing : $(MO_X - MO_H) / MO_X \times 100 \% = (64,4 - 72,7) / 64,4 \times 100 \% = - 12,9 \%$.

* MO_X = Μέσος Όρος Χωρίς Index, MO_B = Μέσος Όρος με B-tree Index, MO_H = Μέσος Όρος με Hashing Index.

Κεφάλαιο 3° | Triggers :

[table]_[action]_log_trigger :

Triggers τα οποία ενεργοποιούνται, αφού το αντίστοιχο action που περιέχεται στην ονομασία έχει πραγματοποιηθεί στον αντίστοιχο πίνακα table. Εισάγουν στον πίνακα **log** καινούρια εγγραφή που περιέχει τον πίνακα table στον οποίο πραγματοποιήθηκε το action [**trip, reservation, event, travel_to, destination**], το ίδιο το action [**insert, update, delete**], καθώς και τον αριθμό ταυτότητας του IT worker, ο οποίος πραγματοποίησε το action. Ο αριθμός ταυτότητας του IT worker που επεξεργάζεται κάθε φορά την βάση μέσω του GUI αποθηκεύεται, κατά το log in του χρήστη, στον πίνακα **active_user**, του οποίου το περιεχόμενο διαγράφεται κάθε φορά που ο χρήστης κάνει log out. Η είσοδος των τιμών εντός του πίνακα **active_user** γίνεται μέσω του κώδικα σε JAVA, εντός του GUI.

```
mysql> select *
-> from table_log;
Empty set (0.00 sec)

mysql> insert into trip values(NULL, '2024-04-15 12:00:00', '2024-06-21 10:00:00', 123, 8745, 00000000012, '00000000015', NULL);
Query OK, 1 row affected (0.00 sec)

mysql> select *
-> from table_log;
+-----+-----+-----+
| log_table | log_action | log_IT_AT |
+-----+-----+-----+
| Trip      | Insert     | 0000000039 |
+-----+-----+-----+
1 row in set (0.00 sec)
```

```
mysql> delete from event
-> where ev_descr = "E3 2023";
Query OK, 1 row affected (0.00 sec)

mysql> select *
-> from table_log;
+-----+-----+-----+
| log_table | log_action | log_IT_AT |
+-----+-----+-----+
| Trip      | Insert     | 0000000039 |
| Event     | Delete     | 0000000039 |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> update reservation
-> set res_name = "Vaggelis"
-> where res_tr_id = 5;
Query OK, 3 rows affected (0.00 sec)
Rows matched: 3 Changed: 3 Warnings: 0

mysql> select *
-> from table_log;
+-----+-----+-----+
| log_table | log_action | log_IT_AT |
+-----+-----+-----+
| Trip      | Insert     | 0000000039 |
| Event     | Delete     | 0000000039 |
| Reservation | Update     | 0000000039 |
| Reservation | Update     | 0000000039 |
| Reservation | Update     | 0000000039 |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

check_reservation_of_trip :

Trigger το οποίο ενεργοποιείται πριν το update του πίνακα **trip**, και αποτρέπει την αλλαγή του κόστους, της ημερομηνίας αναχώρησης και επιστροφής ενός ταξιδιού, εφόσον σε αυτό έχει ήδη καταχωρηθεί τουλάχιστον μια κράτηση.

```
mysql> select tr_id as 'trip id', tr_cost as 'trip cost', tr_departure as 'trip departure', tr_return as 'trip return', count(res_tr_id) as 'reservations'
-> from trip
-> left join reservation
-> on tr_id = res_tr_id
-> group by tr_id
-> limit 7,2;
+-----+-----+-----+-----+-----+
| trip id | trip cost | trip departure | trip return | reservations |
+-----+-----+-----+-----+-----+
|      8 |    500.00 | 2023-03-06 12:30:00 | 2023-03-12 11:55:00 |          3 |
|      9 |   3400.00 | 2024-02-03 11:30:00 | NULL |          0 |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> update trip
-> set tr_departure = "2023-02-07 12:30:00"
-> where tr_id = 8;
ERROR 1644 (45000): ERROR
```

```
mysql> select tr_id as 'trip id', tr_cost as 'trip cost', tr_departure as 'trip departure', tr_return as 'trip return', count(res_tr_id) as 'reservations'
-> from trip
-> left join reservation
-> on tr_id = res_tr_id
-> group by tr_id
-> limit 7,2;
+-----+-----+-----+-----+-----+
| trip id | trip cost | trip departure | trip return | reservations |
+-----+-----+-----+-----+-----+
|      8 |    500.00 | 2023-03-06 12:30:00 | 2023-03-12 11:55:00 |          3 |
|      9 |   3400.00 | 2024-02-03 11:30:00 | NULL |          0 |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> update trip
-> set tr_return = "2023-05-18 11:30:00"
-> where tr_id = 8;
ERROR 1644 (45000): ERROR
```

```
mysql> select tr_id as 'trip id', tr_cost as 'trip cost', tr_departure as 'trip departure', tr_return as 'trip return', count(res_tr_id) as 'reservations'
-> from trip
-> left join reservation
-> on tr_id = res_tr_id
-> group by tr_id
-> limit 7,2;
+-----+-----+-----+-----+-----+
| trip id | trip cost | trip departure | trip return | reservations |
+-----+-----+-----+-----+-----+
|      8 |    500.00 | 2023-03-06 12:30:00 | 2023-03-12 11:55:00 |          3 |
|      9 |   3400.00 | 2024-02-03 11:30:00 | NULL |          0 |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> update trip
-> set tr_cost = 800
-> where tr_id = 8;
ERROR 1644 (45000): ERROR
```

```
mysql> select tr_id as 'trip id', tr_cost as 'trip cost', tr_departure as 'trip departure', tr_return as 'trip return', count(res_tr_id) as 'reservations'
-> from trip
-> left join reservation
-> on tr_id = res_tr_id
-> group by tr_id
-> limit 7,2;
+-----+-----+-----+-----+-----+
| trip id | trip cost | trip departure | trip return | reservations |
+-----+-----+-----+-----+-----+
|      8 |    500.00 | 2023-03-06 12:30:00 | 2023-03-12 11:55:00 |          3 |
|      9 |   3400.00 | 2024-02-03 11:30:00 | NULL |          0 |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> update trip
-> set tr_cost = 7000
-> where tr_id = 9;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select tr_id as 'trip id', tr_cost as 'trip cost', tr_departure as 'trip departure', tr_return as 'trip return', count(res_tr_id) as 'reservations'
-> from trip
-> left join reservation
-> on tr_id = res_tr_id
-> group by tr_id
-> limit 7,2;
+-----+-----+-----+-----+-----+
| trip id | trip cost | trip departure | trip return | reservations |
+-----+-----+-----+-----+-----+
|      8 |    500.00 | 2023-03-06 12:30:00 | 2023-03-12 11:55:00 |          3 |
|      9 |   7000.00 | 2024-02-03 11:30:00 | NULL |          0 |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

check_worker_salary_change :

Trigger το οποίο ενεργοποιείται πριν το update του πίνακα **worker**, και αποτρέπει την μείωση του μισθού ενός υπαλλήλου.

```
mysql> select *
-> from worker
-> limit 1;
+-----+-----+-----+-----+-----+
| wrk_AT | wrk_name | wrk_lname | wrk_salary | wrk_br_code |
+-----+-----+-----+-----+-----+
| 0000000001 | Nikolaos | Dimitriadis | 65000.00 | 1 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> update worker
-> set wrk_salary = 63000
-> where wrk_AT = "0000000001";
ERROR 1644 (45000): ERROR
```

```
mysql> select *
-> from worker
-> limit 1;
+-----+-----+-----+-----+-----+
| wrk_AT | wrk_name | wrk_lname | wrk_salary | wrk_br_code |
+-----+-----+-----+-----+-----+
| 0000000001 | Nikolaos | Dimitriadis | 65000.00 | 1 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> update worker
-> set wrk_salary = 65000
-> where wrk_AT = "0000000001";
Query OK, 0 rows affected (0.00 sec)
Rows matched: 1 Changed: 0 Warnings: 0

mysql> select *
-> from worker
-> limit 1;
+-----+-----+-----+-----+-----+
| wrk_AT | wrk_name | wrk_lname | wrk_salary | wrk_br_code |
+-----+-----+-----+-----+-----+
| 0000000001 | Nikolaos | Dimitriadis | 65000.00 | 1 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

```
mysql> select *
-> from worker
-> limit 1;
+-----+-----+-----+-----+-----+
| wrk_AT | wrk_name | wrk_lname | wrk_salary | wrk_br_code |
+-----+-----+-----+-----+-----+
| 0000000001 | Nikolaos | Dimitriadis | 65000.00 | 1 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> update worker
-> set wrk_salary = 68000
-> where wrk_AT = "0000000001";
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> select *
-> from worker
-> limit 1;
+-----+-----+-----+-----+-----+
| wrk_AT | wrk_name | wrk_lname | wrk_salary | wrk_br_code |
+-----+-----+-----+-----+-----+
| 0000000001 | Nikolaos | Dimitriadis | 68000.00 | 1 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Μέρος Β : GUI

Κεφάλαιο 4° | Επεξήγηση GUI :

Εισαγωγική αναφορά :

Η ανάπτυξη του GUI έγινε με τη χρήση της εφαρμογής NetBeans και φυσικά δόθηκε μεγαλύτερη βάση στην λειτουργικότητα και υλοποίηση όλων των απαιτούμενων ζητούμενων παρά στην αισθητική. Γι' αυτό το λόγο, τόσο η ίδια η γραφική διεπαφή, όσο και ο κώδικας υλοποίηση των λειτουργιών της, δεν είναι βέλτιστοι. Σε αυτό βέβαια έπαιξε αρκετά μεγάλο ρόλο και το ίδιο το NetBeans, που σε αρκετά σημεία δημιουργούσε τον δικό του κώδικα, τον οποίο δεν επέτρεπε να τροποποιήσουμε, οπότε αναγκαζόμασταν να γράψουμε κώδικα με διαφορετική λογική από αυτή που θέλαμε ή προτιμούσαμε.

Στα πλαίσια της σχεδίασης, οι ζητούμενες δυνατότητες υλοποιούνται από ένα πλήθος διαφορετικών παραθύρων, στα οποία ο χρήστης μπορεί να περιηγείται ελεύθερα, ανάλογα με το τι θέλει να κάνει. Αυτή η αναφορά θα επικεντρωθεί στην ανάλυση αυτών των παραθύρων και τις λειτουργίες τους οπότε θα έχει και αντίστοιχη τμηματική μορφή ανά παράθυρο.

Επιπλέον, τα αρχεία κώδικα που αφορούν την υλοποίηση του GUI συμπεριλαμβάνονται τόσο στο παράρτημα κώδικα που βρίσκεται στο τέλος της αναφοράς, όσο και στο συμπιεσμένο αρχείο markakis_1084541_melitsopoulos_1084600_tsoranoglou_1084596_B.zip όπως δημιουργήθηκαν από το NetBeans. Το ίδιο το πρόγραμμα δημιουργούσε μόνο του χωρία κώδικα τα οποία δεν ήταν εφικτό να αφαιρέσουμε εμείς εντός αυτού. Ακόμα, δεν θέλαμε να διακινδυνεύσουμε το να μη λειτουργεί το GUI όπως έχει δημιουργηθεί σαν φιλοσοφία και υλοποίηση, συνεπώς προτιμήσαμε να συμπεριλάβουμε τους φακέλους των αρχείων αυτούσιους.

LoginWindow :

Πρόκειται για το παράθυρο που ανοίγει αρχικά όταν γίνεται εκτέλεση του προγράμματος. Διαθέτει πεδία username και password και ζητά από το χρήστη να εισάγει τα δεδομένα αυτά. Αφού έχουν επιτυχώς εισαχθεί όλα τα απαραίτητα δεδομένα, επιχειρείται ταυτοποίηση του χρήστη.

Ένα πρώτο query επιστρέφει πίσω όλα τα απαραίτητα στοιχεία χρήστη για τους χρήστες με το όνομα που δόθηκε. Αν έχει επιστραφεί οτιδήποτε, τότε γίνεται έλεγχος αν κάποιος από τους χρήστες που επιστράφηκαν έχει το σωστό κωδικό. Αν ναι, τότε ο χρήστης έχει ταυτοποιηθεί επιτυχώς και γίνεται το login. Φυσικά αν υπάρχει οποιοδήποτε πρόβλημα, ο χρήστης ενημερώνεται καταλλήλως.

Ένα νέο query ανανεώνει τον πίνακα current_active_user της βάσης, όπου αποθηκεύεται ο αριθμός ταυτότητας του χρήστη που έκανε login για να μπορεί αργότερα να χρησιμοποιηθεί για εισαγωγή στον πίνακα log_table.

Τέλος, ανοίγει ένα νέο παράθυρο **MainMenu** και τερματίζεται το τωρινό **LoginWindow**.

MainMenu :

Είναι το πρώτο και κύριο παράθυρο, στο οποίο ο χρήστης έχει πρόσβαση όσο χρησιμοποιεί τη γραφική διεπαφή. Μέσω αυτού του παραθύρου επιλέγει μια από όλες τις λειτουργίες που του προσφέρει η διεπαφή πατώντας το αντίστοιχο κουμπί και μεταφέρεται σε καινούργιο παράθυρο, που υλοποιεί την αντίστοιχη λειτουργία. Επίσης υπάρχει κουμπί logout που τον επιστρέφει στο **LoginWindow** και τον αφαιρεί παράλληλα από τον πίνακα current_active_user.

EditTableMenu :

Είναι το βασικό μενού που υλοποιεί τις δυνατότητες εισαγωγής, τροποποίησης και διαγραφής στη βάση. Προσφέρει τη δυνατότητα παρατήρησης του πίνακα ο οποίος έχει επιλεγεί από το χρήστη ώστε να μπορεί να έχει καλύτερη ιδέα των αλλαγών που επρόκειτο να εκτελέσει.

Αρχικά ο χρήστης πρέπει να επιλέξει μια ενέργεια και έναν πίνακα. Η επιλογή του πίνακα από τη βάση δυναμικά αλλάζει το περιεχόμενο που απεικονίζεται στον πίνακα της διεπαφής. Αφού ο χρήστης επιλέξει τα απαιτούμενα, πατά το κουμπί της επιβεβαίωσης και έτσι ανοίγει το κατάλληλο υπό-παράθυρο για την εκτέλεση της επιλεγμένης ενέργειας. Τα υπό-παράθυρα που μπορούν να ανοίξουν κατ' αυτό το τρόπο είναι τα **InsertPage**, **UpdatePage**, **DeletePage**.

Φυσικά από εδώ υπάρχει και κουμπί επιστροφής στο αρχικό μενού.

InsertPage :

Υπό-παράθυρο που ανοίγει από το **EditTableMenu** και υλοποιεί την ενέργεια της εισαγωγής σε πίνακα της βάσης. Στον χρήστη εμφανίζονται κενά πεδία με τα απαραίτητα δεδομένα του πίνακα που έχει επιλέξει. Αφού ο χρήστης γεμίσει τα πεδία όπως επιθυμεί πατά το κουμπί της εισαγωγής. Αν υπάρξει πρόβλημα κατά της εισαγωγή ο χρήστης ενημερώνεται, αλλιώς η εισαγωγή γίνεται και η αλλαγή μπορεί να φανεί και στον πίνακα του κύριου παραθύρου **EditTableMenu**, το οποίο είναι ανοιχτό παράλληλα.

Updatepage :

Υπό-παράθυρο που ανοίγει από το **EditTableMenu** και υλοποιεί την ενέργεια της ενημέρωσης σε πίνακα της βάσης. Ο χρήστης μπορεί να επιλέξει από μια λίστα που περιέχει τις τωρινές εγγραφές του πίνακα που έχει επιλέξει. Τα δεδομένα της επιλεγμένης εγγραφής εμφανίζονται σε έναν πίνακα με μια μόνο γραμμή και ο χρήστης μπορεί ελεύθερα να τα τροποποιήσει. Αφού έχει κάνει τις απαραίτητες αλλαγές, μπορεί να πατήσει το κουμπί ενημέρωσης. Αν η ενημέρωση δεν γίνει, ο χρήστης ενημερώνεται, αλλιώς μπορεί να δει την αλλαγή που έκανε κατευθείαν στον πίνακα του κύριου παραθύρου **EditTableMenu**, το οποίο είναι ανοιχτό παράλληλα.

DeletePage :

Υπό-παράθυρο που ανοίγει από το **EditTableMenu** και υλοποιεί την ενέργεια της διαγραφής από πίνακα της βάσης. Ο χρήστης μπορεί να επιλέξει από μια λίστα που περιέχει τις τωρινές εγγραφές του πίνακα που έχει επιλέξει. Έπειτα πατώντας το κουμπί της διαγραφής, η επιλεγμένη εγγραφή διαγράφεται από την βάση και η αλλαγή φαίνεται επίσης στον πίνακα του κύριου παραθύρου **EditTableMenu**, το οποίο είναι ανοιχτό παράλληλα.

LogPage :

Παράθυρο μέσω του οποίου ο χρήστης μπορεί να δει τα περιεχόμενα του πίνακα `table_log`. Περιέχει ένα πίνακα, στον οποίο εισάγονται τα δεδομένα που επιστρέφονται από το `query`.

AddITPage :

Το παράθυρο που επιτρέπει στον χρήστη να εισάγει νέο υπεύθυνο πληροφορικής. Παρέχονται πεδία στα οποία ο χρήστης μπορεί να εισάγει τα απαραίτητα δεδομένα για μια νέα εγγραφή του πίνακα IT. Όταν ο χρήστης εισάγει τα δεδομένα πατάει το κουμπί εισαγωγής. Αν υπάρξει πρόβλημα ο χρήστης ενημερώνεται, αλλιώς η εισαγωγή γίνεται κανονικά.

CustomerReservationPage :

Το παράθυρο που υλοποιεί την αναζήτηση κρατήσεων προσφορών ταξιδιού με βάση του επώνυμο του πελάτη. Ο χρήστης πρέπει να δώσει στο πεδίο ονόματος, το επώνυμο που θέλει και αφού πατήσει το κουμπί αναζήτησης, στον πίνακα εμφανίζονται τα δεδομένα της αναζήτησης. Για τη λειτουργία του συγκεκριμένου παραθύρου γίνεται χρήση της αντίστοιχης `stored procedure` που υλοποιήθηκε στο κομμάτι της SQL.

BranchTripInfoPage :

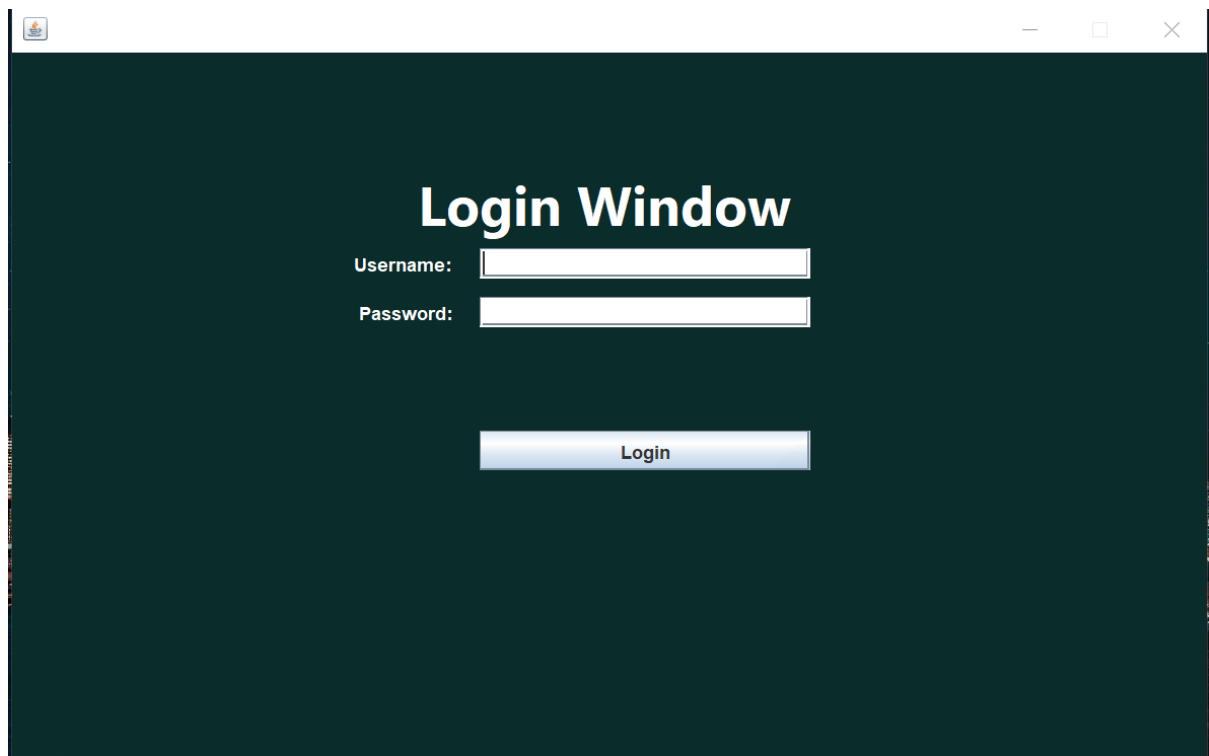
Το παράθυρο που υλοποιεί την αναζήτηση ταξιδιών με βάση την ημερομηνία αναχώρησης τους. Ο χρήστης μπορεί δυναμικά να βλέπει τα ταξίδια που είναι διαθέσιμα σε όλα τα υποκαταστήματα αλλάζοντας τις ζητούμενες ημερομηνίες από τις αντίστοιχες λίστες επιλογής. Τα περιεχόμενα της αναζήτησης εμφανίζονται στον από κάτω πίνακα.

BranchInfo :

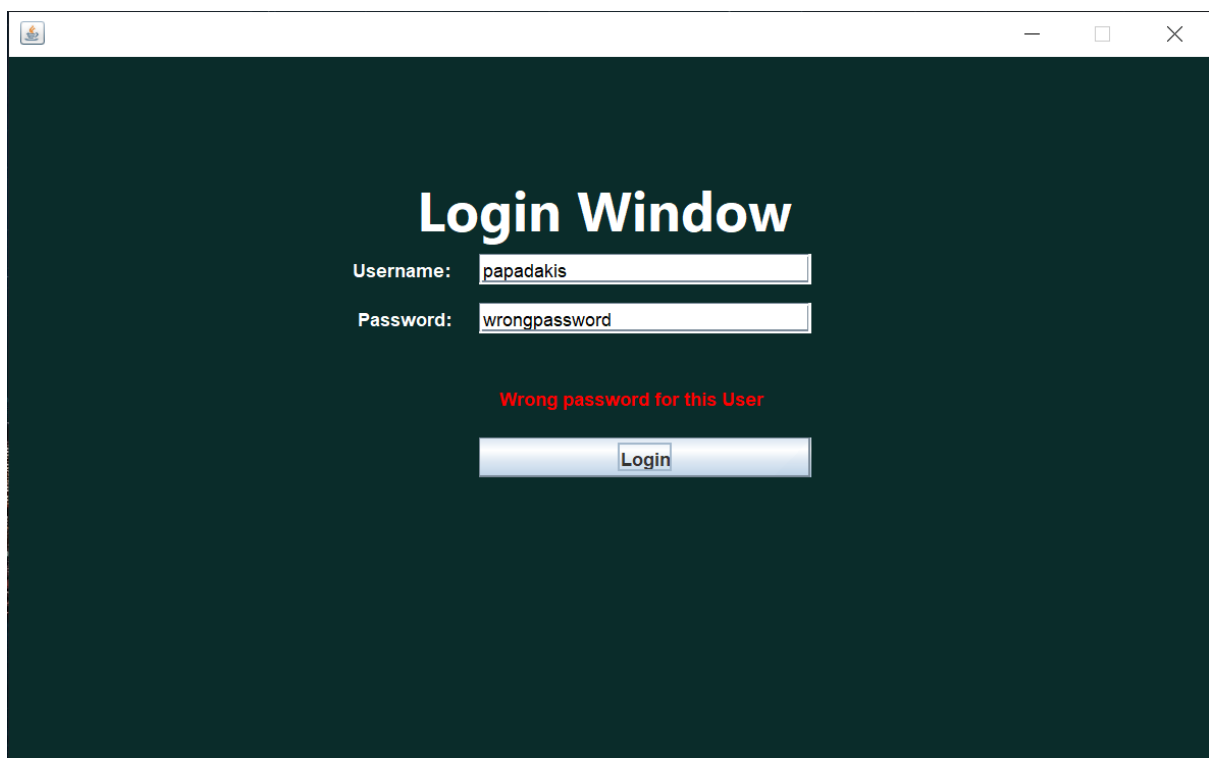
Το παράθυρο που υλοποιεί την αναζήτηση των πληροφοριών ενός υποκαταστήματος. Ο χρήστης επιλέγει το υποκατάστημα που θέλει από τη λίστα επιλογής και τα δεδομένα του υποκαταστήματος παρουσιάζονται στους τριγύρω πίνακες. Ο επάνω πίνακας παρουσιάζει τα δεδομένα του υποκαταστήματος, ο δεξιό πίνακας δείχνει τους υπαλλήλους και ο κάτω πίνακας δείχνει το συνολικό ποσό μισθών του υποκαταστήματος.

Διεπαφή με Screenshots :

Μενού για login και παραδειγμα εισαγωγής λάθος στοιχείων :

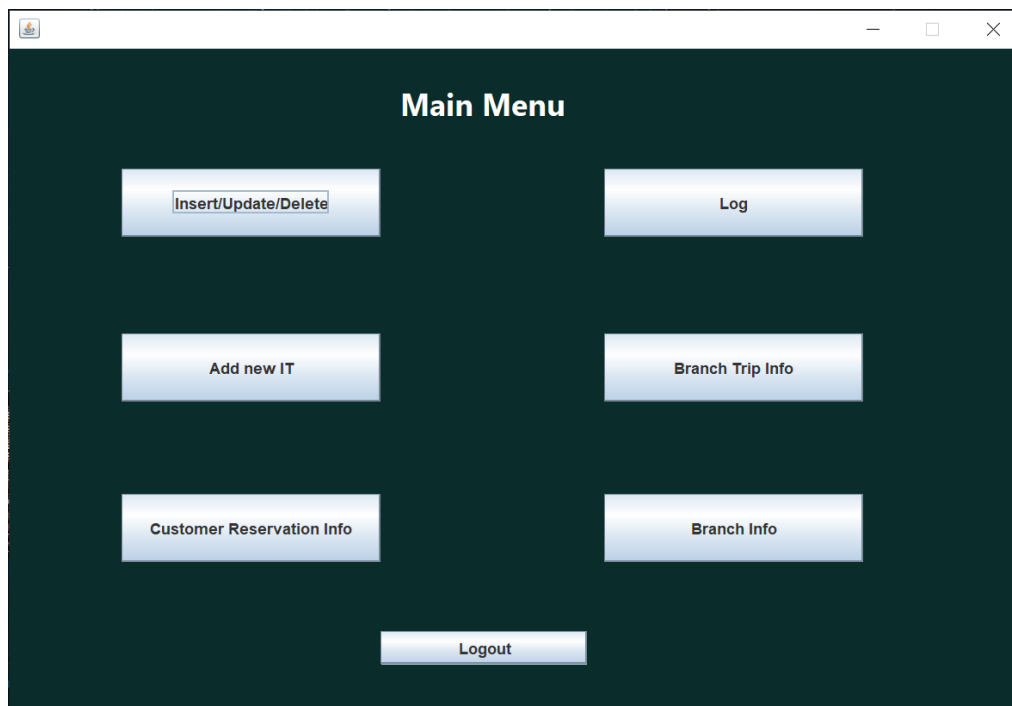


A screenshot of a web application window titled "Login Window". The window has a dark green background. In the center, there is a white "Login Window" title. Below the title, there are two input fields: "Username:" and "Password:". Both fields are empty. Below the input fields, there is a blue "Login" button.



A screenshot of the same "Login Window" web application. The "Username:" field now contains the text "papadakis" and the "Password:" field contains the text "wrongpassword". Below the input fields, there is a red error message that reads "Wrong password for this User". The blue "Login" button remains at the bottom.

Βασικό μενού με όλες τις επιλογές του χρήστη :

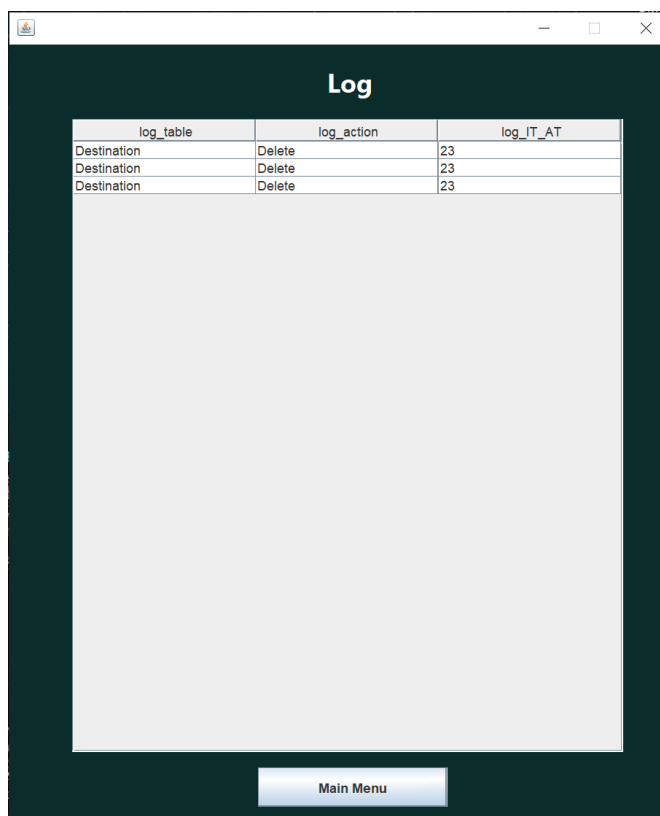
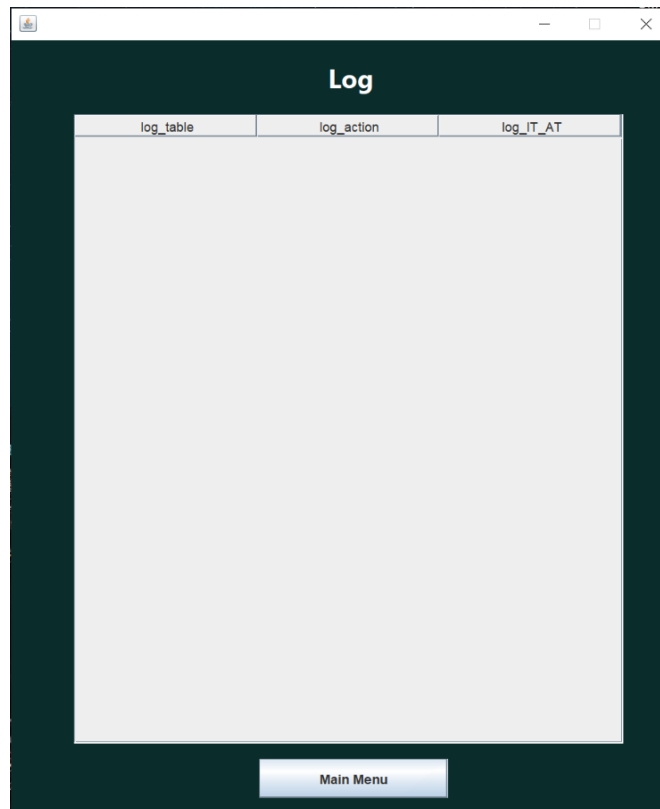


Το παράθυρο εισαγωγής νέου IT με όλα τα πεδία εισαγωγής :

The screenshot shows a window titled "Add IT" with a dark green background. It contains the following input fields and buttons:

- Input fields for "ID number:", "Password:", "First name:", "Last name:", and "Salary:" (with a numeric spinner).
- A "Branch:" label followed by a dropdown menu currently showing "Branch".
- A large "Add" button at the bottom center.
- A "Main Menu" button at the very bottom center.

Το παράθυρο παρατήρησης του log άδειο και με εγγραφές :



Το παράθυρο παρατήρησης των ταξιδίων ενός υποκαταστήματος σε συγκεκριμένο χρονικό διάστημα :

Branch Trip Info

From: 1 1 2022 To: 1 1 2022

Main Menu

Branch Trip Info

From: 1 1 2022 To: 1 1 2024

br_code	tr_cost	tr_maxseats	reservations	empty seats	Driver lname	Driver name	Guide lname	Guide name	tr_departure	tr_return
5	6578.0	119	3	116	NULL	NULL	Tsoukalas	Fwtios	2023-08-27T0...	2023-08-31T1...
6	3400.0	96	2	94	Prodromou	Elpida	Karahaliou	Dimitris	2023-02-17T1...	2023-04-28T1...
6	2345.0	94	2	92	Liapis	Dionusios	NULL	NULL	2023-04-21T1...	2023-04-28T1...
8	9678.0	78	4	74	Mauridi	Katerina	Iampropoulos	Marios	2023-04-05T1...	2023-04-18T1...
8	500.0	120	3	117	Giannaris	Kuriakos	Karahaliou	Dimitris	2023-04-06T1...	2023-05-01T1...
9	3835.0	87	2	85	Giannaris	Kuriakos	Lazarou	Iwanna	2023-07-13T1...	2023-07-19T1...
11	9678.0	35	3	32	Galana	Basiliki	Mpenos	Zaxarias	2023-05-16T1...	2023-05-26T1...
12	8745.0	123	2	121	NULL	NULL	Metaxas	Apostolos	2023-03-15T1...	2023-03-21T1...

Main Menu

Το παράθυρο παρατήρησης των δεδομενων ενός υποκαταστήματος με παράδειγμα για δυο διαφορετικά υποκαταστήματα :

Branch Info

br_code	br_street	br_num	br_city	wrk_name	wrk_lname	Reservations	SUM(Profit)
1	Papadopoulou	22	Volos	NULL	NULL	6	NULL

Select branc...

1

Total Salary

311000.0

br_code	wrk_name	wrk_lname	wrk_salary
1	Nikolaos	Dimitriadis	65000.0
1	Periklis	Dellakos	87000.0
1	Dimitris	Karamanos	99000.0
1	Despolina	Eleutheriadl	60000.0

Main Menu

Branch Info

br_code	br_street	br_num	br_city	wrk_name	wrk_lname	Reservations	SUM(Profit)
5	Paulou Mela	114	Iwannina	NULL	NULL	3	19734.0

Select branc...

5

Total Salary

123000.0

br_code	wrk_name	wrk_lname	wrk_salary
5	Theofilos	Georgiou	49000.0
5	Elpida	Prodromou	34000.0
5	Ioanna	Iwannou	40000.0

Main Menu

Το παράθυρο αναζήτησης κρατήσεων με βάση επώνυμο πελάτη :

Customer Reservations

Search by customer last name:

Customer Reservations

Search by customer last name:

res_ofr_lname	res_ofr_name	res_ofr_ofr_id	count(*)
Tate	Milagros	1	2
Tate	Nola	1	2
Tate	Karlee	1	2
Tate	Romeo	1	2
Tate	Eddie	1	2
Tate	Myah	1	2
Tate	Marques	1	2
Tate	Trinity	1	2
Tate	Cannon	1	2
Tate	Jorge	1	2
Tate	Beatrice	1	2
Tate	Asher	1	2
Tate	Kieran	1	2
Tate	Caitlyn	1	2
Tate	Milagros	2	2
Tate	Nola	2	2
Tate	Karlee	2	2
Tate	Romeo	2	2
Tate	Eddie	2	2
Tate	Myah	2	2
Tate	Marques	2	2
Tate	Trinity	2	2
Tate	Cannon	2	2
Tate	Jorge	2	2

Το μενού εισαγωγής/τροποποίησης/διαγραφής πριν και μετά την επιλογή πίνακα :

A screenshot of a software window with a dark green background. In the lower-left area, there are two dropdown menus. The first is labeled 'Select action:' and has 'Action' selected. The second is labeled 'Select table:' and has 'Table' selected. Below these menus is a 'Confirm' button. At the bottom center of the window is a 'Main Menu' button.

A screenshot of the same software window, but now displaying a table of data. The table has five columns: 'Index', 'br_code', 'br_street', 'br_num', and 'br_city'. It contains 12 rows of data. The 'Select action:' dropdown now shows 'Delete' and the 'Select table:' dropdown shows 'branch'. The 'Confirm' and 'Main Menu' buttons remain in their original positions.

Index	br_code	br_street	br_num	br_city
1	1	Papadopoulos	22	Volos
2	2	Xarilaou Trikoupi	43	Athina
3	3	Malizinos	78	Patra
4	4	Don Kiriati	81	Thessaloniki
5	5	Paulou Mela	114	Iwannina
6	6	Ragkavi	56	Rodos
7	7	Dimitriadi	9	Trikala
8	8	Triwn Nauarxwn	16	Patra
9	9	Amerikis	47	Lamia
10	10	Orfeos	25	Mutlilini
11	11	Xaralampous	124	Sparti
12	12	Korai	235	Athina

Τα υπό-παράθυρα για εισαγωγή, τροποποίηση και διαγραφή όταν ανοίξουν μετά από επιλογή του χρήστη στο μενού διαχείρισης τους :

The 'Insert into branch' dialog box features a dark green background with a white title bar. The title 'Insert into branch' is centered at the top. Below the title is a table with four columns: 'br_code', 'br_street', 'br_num', and 'br_city'. The table is currently empty. Below the table is a large 'Insert' button, and at the bottom is a smaller 'Close' button.

br_code	br_street	br_num	br_city
---------	-----------	--------	---------

Insert

Close

The 'Update branch' dialog box has a dark green background and a white title bar. The title 'Update branch' is centered at the top. On the left, there is a 'Select to update' label above a dropdown menu showing the value '1'. To the right of the dropdown is a table with four columns: 'br_code', 'br_street', 'br_num', and 'br_city'. The table contains one row with the values '1', 'Papadopoulou', '22', and 'Volos'. Below the table is a large 'Update' button, and at the bottom is a smaller 'Close' button.

Select to update

1

br_code	br_street	br_num	br_city
1	Papadopoulou	22	Volos

Update

Close

The 'Delete from branch' dialog box has a dark green background and a white title bar. The title 'Delete from branch' is centered at the top. On the left, there is a 'Select to delete' label above a dropdown menu showing the value '1'. To the right of the dropdown is a large 'Delete' button. At the bottom is a smaller 'Close' button.

Select to delete

1

Delete

Close

Παράρτημα Κώδικα ΒΔ (mySQL)

createStatements.sql :

```
-- CREATE STATEMENTS
```

```
-- CONSTRAINT CONSTRAINT: [referenced table (abbr.)]_[current field (abbr.)]_[referenced field (abbr.)].
```

```
-- CONSTRAINT EXAMPLE: TABLE guide REFERENCES TABLE worker ON FIELD wrk_AT -> wrk_gui_at.
```

```
-- ALL INTEGER PRIMARY KEYS ARE SET TO AUTO_INCREMENT
```

```
-- FOR EASE OF USE WHEN INSERTING RECORDS.
```

```
DROP DATABASE IF EXISTS travel_agency;
```

```
CREATE DATABASE travel_agency;
```

```
USE travel_agency;
```

```
-- A branch CANNOT HAVE A NULL VALUE IN ANY OF ITS FIELDS.
```

```
CREATE TABLE branch(
```

```
br_code INT(11) NOT NULL AUTO_INCREMENT,
```

```
br_street VARCHAR(30) NOT NULL,
```

```
br_num INT(4) NOT NULL,
```

```
br_city VARCHAR(30) NOT NULL,
```

```
PRIMARY KEY(br_code)
```

```
);
```

```
CREATE TABLE phones(
```

```
ph_br_code INT(11) NOT NULL,  
ph_number CHAR(10) NOT NULL,
```

```
PRIMARY KEY(ph_br_code, ph_number),
```

```
CONSTRAINT br_ph_codes
```

```
FOREIGN KEY(ph_br_code)
```

```
REFERENCES branch(br_code)
```

```
ON DELETE CASCADE ON UPDATE CASCADE
```

```
);
```

```
-- IF A branch IS DELETED SET ALL ITS workers'wrk_br_code
```

```
-- FIELDS TO NULL FOR EASE OF TRANSFERING workers.
```

```
CREATE TABLE worker(
```

```
wrk_AT CHAR(10) NOT NULL,
```

```
wrk_name VARCHAR(20) DEFAULT 'unknown',
```

```
wrk_lname VARCHAR(20) DEFAULT 'unknown',
```

```
wrk_salary FLOAT(7, 2),
```

```
wrk_br_code INT(11),
```

```
PRIMARY KEY(wrk_AT),
```

```
CONSTRAINT br_wrk_code
```

```
FOREIGN KEY(wrk_br_code)
```

```
REFERENCES branch(br_code)
```

```
ON DELETE SET NULL ON UPDATE CASCADE
```

```
);
```

-- A driver CANNOT HAVE A NULL VALUE IN ANY OF ITS FIELDS.

CREATE TABLE driver(

drv_AT CHAR(10) NOT NULL,

drv_license ENUM('A', 'B', 'C', 'D') NOT NULL,

drv_route ENUM('LOCAL', 'ABROAD') NOT NULL,

drv_experience TINYINT(4) DEFAULT 0,

PRIMARY KEY(drv_AT),

CONSTRAINT wrk_drv_at

FOREIGN KEY(drv_AT)

REFERENCES worker(wrk_AT)

ON DELETE CASCADE ON UPDATE CASCADE

);

-- AN admin MIGHT NOT HAVE A adm_diploma SO IT'S

-- THE ONLY FIELD ALLOWED TO HAVE A NULL VALUE.

CREATE TABLE admin(

adm_AT CHAR(10) NOT NULL,

adm_type ENUM('LOGISTICS', 'ADMINISTRATIVE', 'ACCOUNTING') NOT NULL,

adm_diploma VARCHAR(200),

PRIMARY KEY(adm_AT),

CONSTRAINT wrk_adm_at

FOREIGN KEY(adm_AT)

```
REFERENCES worker(wrk_AT)
ON DELETE CASCADE ON UPDATE CASCADE
```

```
);
```

```
CREATE TABLE manages(
```

```
mng_adm_AT CHAR(10) NOT NULL,
mng_br_code INT(11) NOT NULL,
```

```
PRIMARY KEY(mng_adm_AT, mng_br_code),
```

```
CONSTRAINT adm_mng_at
FOREIGN KEY(mng_adm_AT)
REFERENCES admin(adm_AT)
ON DELETE CASCADE ON UPDATE CASCADE,
```

```
CONSTRAINT br_mng_code
FOREIGN KEY(mng_br_code)
REFERENCES branch(br_code)
ON DELETE CASCADE ON UPDATE CASCADE
```

```
);
```

```
-- A guide MUST HAVE A gui_cv VALUE.
```

```
CREATE TABLE guide(
```

```
gui_AT CHAR(10) NOT NULL,
gui_cv TEXT NOT NULL,
```

PRIMARY KEY(gui_AT),

CONSTRAINT wrk_gui_at

FOREIGN KEY(gui_AT)

REFERENCES worker(wrk_AT)

ON DELETE CASCADE ON UPDATE CASCADE

);

CREATE TABLE languages(

lng_gui_AT CHAR(10) NOT NULL,

lng_language VARCHAR(30) NOT NULL,

PRIMARY KEY(lng_gui_AT, lng_language),

CONSTRAINT gui_lng_at

FOREIGN KEY(lng_gui_AT)

REFERENCES guide(gui_AT)

ON DELETE CASCADE ON UPDATE CASCADE

);

--

CREATE TABLE IT(

IT_AT CHAR(10) NOT NULL,

IT_password CHAR(10) NOT NULL DEFAULT 'password',


```
start_date DATE NOT NULL,  
end_date DATE DEFAULT NULL,
```

```
PRIMARY KEY(IT_AT),
```

```
CONSTRAINT wrk_it_at  
FOREIGN KEY(IT_AT)  
REFERENCES worker(wrk_AT)  
ON DELETE CASCADE ON UPDATE CASCADE
```

```
);
```

```
-- A trip MIGHT NOT HAVE A tr_return, A tr_cost, A driver OR A guide SPECIFIED,
```

```
CREATE TABLE trip(
```

```
tr_id INT(11) NOT NULL AUTO_INCREMENT,  
tr_departure DATETIME NOT NULL,  
tr_return DATETIME,  
tr_maxseats TINYINT(4) NOT NULL,  
tr_cost FLOAT(7, 2),  
tr_br_code INT(11) NOT NULL,  
tr_gui_AT CHAR(10),  
tr_drv_AT CHAR(10),
```

```
PRIMARY KEY(tr_id),
```

```
CONSTRAINT br_tr_code  
FOREIGN KEY(tr_br_code)  
REFERENCES branch(br_code)  
ON DELETE CASCADE ON UPDATE CASCADE,
```

```
CONSTRAINT gui_tr_at
FOREIGN KEY(tr_gui_AT)
REFERENCES guide(gui_AT)
ON DELETE SET NULL ON UPDATE CASCADE,
```

```
CONSTRAINT drv_tr_at
FOREIGN KEY(tr_drv_AT)
REFERENCES driver(drv_AT)
ON DELETE SET NULL ON UPDATE CASCADE
```

```
);
```

```
-- AN event CANNOT HAVE A NULL VALUE IN ANY OF ITS FIELDS.
```

```
CREATE TABLE event(
```

```
ev_tr_id INT(11) NOT NULL,
ev_start DATETIME NOT NULL,
ev_end DATETIME NOT NULL,
ev_descr TEXT NOT NULL,
```

```
PRIMARY KEY(ev_tr_id, ev_start),
```

```
CONSTRAINT tr_ev_id
FOREIGN KEY(ev_tr_id)
REFERENCES trip(tr_id)
ON DELETE CASCADE ON UPDATE CASCADE
```

```
);
```

```
-- A destination MUST HAVE dst_rtype & dst_language VALUES.
```

```
CREATE TABLE destination(  
  
    dst_id INT(11) NOT NULL AUTO_INCREMENT,  
    dst_name VARCHAR(50),  
    dst_descr TEXT,  
    dst_rtype ENUM('LOCAL', 'ABROAD') NOT NULL,  
    dst_language VARCHAR(30) NOT NULL,  
    dst_location INT(11),  
  
    PRIMARY KEY(dst_id),  
  
    CONSTRAINT dst_dst_id  
    FOREIGN KEY(dst_location)  
    REFERENCES destination(dst_id)  
    ON DELETE CASCADE ON UPDATE CASCADE  
  
);
```

--

```
CREATE TABLE travel_to(  
  
    to_tr_id INT(11) NOT NULL,  
    to_dst_id INT(11) NOT NULL,  
    to_arrival DATETIME,  
    to_departure DATETIME,  
  
    PRIMARY KEY(to_tr_id, to_dst_id),  
  
    CONSTRAINT tr_to_id  
    FOREIGN KEY(to_tr_id)  
    REFERENCES trip(tr_id)
```

ON DELETE CASCADE ON UPDATE CASCADE,

CONSTRAINT dst_to_id

FOREIGN KEY(to_dst_id)

REFERENCES destination(dst_id)

ON DELETE CASCADE ON UPDATE CASCADE

);

-- A reservation CANNOT HAVE A NULL VALUE IN ANY OF ITS FIELDS.

CREATE TABLE reservation(

res_tr_id INT(11) NOT NULL,

res_seatnum TINYINT(4) NOT NULL,

res_name VARCHAR(20) NOT NULL,

res_lname VARCHAR(20) NOT NULL,

res_isadult ENUM('ADULT', 'MINOR') NOT NULL,

PRIMARY KEY(res_tr_id, res_seatnum),

CONSTRAINT tr_res_id

FOREIGN KEY(res_tr_id)

REFERENCES trip(tr_id)

ON DELETE CASCADE ON UPDATE CASCADE

);

--

CREATE TABLE offer(

```
ofr_id INT(11) NOT NULL AUTO_INCREMENT,  
ofr_start_date DATE NOT NULL,  
ofr_end_date DATE NOT NULL,  
ofr_cost FLOAT(7, 2) NOT NULL,  
ofr_dst_id INT(11) NOT NULL,
```

```
PRIMARY KEY(ofr_id),
```

```
CONSTRAINT dst_ofr_id  
FOREIGN KEY(ofr_dst_id)  
REFERENCES destination(dst_id)  
ON DELETE CASCADE ON UPDATE CASCADE
```

```
);
```

```
--
```

```
CREATE TABLE reservation_offer(  
  

```

```
res_ofr_id INT(11) NOT NULL AUTO_INCREMENT,  
res_ofr_lname VARCHAR(20) DEFAULT 'unknown',  
res_ofr_name VARCHAR(20) DEFAULT 'unknown',  
res_ofr_ofr_id INT(11) NOT NULL,  
res_ofr_fee FLOAT(7, 2) NOT NULL,
```

```
PRIMARY KEY(res_ofr_id),
```

```
CONSTRAINT ofr_res_ofr_id  
FOREIGN KEY(res_ofr_ofr_id)  
REFERENCES offer(ofr_id)  
ON DELETE CASCADE ON UPDATE CASCADE
```

```
);
```

```
--
```

```
CREATE TABLE table_log(
```

```
log_table ENUM('Trip', 'Reservation', 'Event', 'Travel_to', 'Destination'),
```

```
log_action ENUM('Insert', 'Update', 'Delete'),
```

```
log_IT_AT CHAR(10)
```

```
);
```

```
--
```

```
CREATE TABLE current_active_user(
```

```
user_IT_AT CHAR(10)
```

```
);
```

insertStatements.sql :

-- INSERT STATEMENTS

-- ALL INSERTED VALUES ARE USED TO DEPICT

-- A NORMAL DATASET, INCLUDING EDGE CASES.

INSERT INTO branch VALUES

(NULL, 'Papadopoulou', 22, 'Volos'),
(NULL, 'Xarilaou Trikoupi', 43, 'Athina'),
(NULL, 'Maizwnos', 78, 'Patra'),
(NULL, 'Don Kixwti', 81, 'Thessaloniki'),
(NULL, 'Paulou Mela', 114, 'Iwannina'),
(NULL, 'Ragkavi', 56, 'Rodos'),
(NULL, 'Dimitriadi', 09, 'Trikala'),
(NULL, 'Triwn Nauarxwn', 16, 'Patra'),
(NULL, 'Amerikis', 47, 'Lamia'),
(NULL, 'Orfeos', 25, 'Mutilini'),
(NULL, 'Xaralampous', 124, 'Sparti'),
(NULL, 'Korai', 235, 'Athina'

);

INSERT INTO phones VALUES

(000000000001, '2421083671'),
(000000000002, '2107308002'),
(000000000003, '2610968475'),
(000000000004, '2310723938'),
(000000000004, '2310092334'),
(000000000005, '2651060987'),
(000000000006, '2241023450'),

(00000000007, '2431009236'),
(00000000009, '2231023405'),
(00000000010, '2251033859'),
(00000000011, '2731049746'),
(00000000012, '2251042768'),
(00000000003, '2610084882'),
(00000000004, '2310234597')

;

INSERT INTO worker VALUES

('0000000001', 'Nikolaos', 'Dimitriadis', 65000, 1),
(('0000000002', 'Lazaros', 'Oikonomou', NULL, NULL),
(('0000000003', 'Periklis', 'Dellakos', 87000, 1),
(('0000000004', NULL, 'Dimosthenous', 56000, 3),
(('0000000005', 'Athanasios', 'Makris', 48000, 7),
(('0000000006', 'Theofilos', NULL, 49000, 5),
(('0000000007', 'Maria', 'Papanikolaou', 98000, 4),
(('0000000008', 'Elpida', 'Prodromou', 34000, 5),
(('0000000009', 'Aggeliki', 'Maniatidi', 83000, NULL),
(('0000000010', 'Basiliki', 'Galana', NULL, 7),
(('0000000011', 'Paraskevi', 'Kwstopoulou', 69000, 7),
(('0000000012', 'Andromaxi', 'Kwsti', 94000, 7),
(('0000000013', 'Lydia', 'Anastopoulou', 72000, 6),
(('0000000014', NULL, 'Karamanos', 99000, 1),
(('0000000015', 'Apostolos', 'Metaxas', 32000, 9),
(('0000000016', 'Dionusios', 'Liapis', 49000, 10),
(('0000000017', 'Lewnidas', 'Karakasis', 50000, 10),
(('0000000018', 'Despoina', 'Eleutheriadi', 60000, NULL),
(('0000000019', NULL, 'Mauridi', 75000, 11),
(('0000000020', 'Iwanna', 'Lazarou', 85000, 11),

('0000000021', 'Niki', 'Kexagia', NULL, 12),
('0000000022', 'Marios', 'Iampropoulos', 77000, 10),
('0000000023', 'Kosmas', 'Papadakis', 52000, 7),
('0000000024', 'Dimitris', NULL, 64000, 6),
('0000000025', 'Alexandros', 'Germanos', 67000, 8),
('0000000026', 'Andreas', 'Gennimatas', NULL, 3),
('0000000027', 'Zaxarias', NULL, 81000, 7),
('0000000028', 'Kuriakos', 'Giannaris', 86000, 11),
('0000000029', NULL, 'Iwannou', 40000, 5),
('0000000030', 'Fwtios', 'Tsoukalas', 49000, 9),
('0000000031', 'Kosmas', 'Papadakis', 52000, 7),
('0000000032', 'Kosmas', 'Papadakis', 52000, 7),
('0000000033', 'Kosmas', 'Papadakis', 52000, 7),
('0000000034', 'Kosmas', 'Papadakis', 52000, 7),
('0000000035', 'Kosmas', 'Papadakis', 52000, 7),
('0000000036', 'Kosmas', 'Papadakis', 52000, 7),
('0000000037', 'Kosmas', 'Papadakis', 52000, 7),
('0000000038', 'Kosmas', 'Papadakis', 52000, 7),
('0000000039', 'Kosmas', 'a', 52000, 7)

;

INSERT INTO driver VALUES

('0000000008', 'A', 'Local' , 34),
('0000000010', 'A', 'Abroad', 117),
('0000000005', 'C', 'Local', 127),
('0000000019', 'D', 'Abroad', NULL),
('0000000028', 'D', 'Abroad', 123),
('0000000016', 'C', 'Local', NULL),
('0000000014', 'A', 'Local', 78),
('0000000025', 'B', 'Abroad', 45)

;

INSERT INTO admin VALUES

('0000000001', 'Logistics', 'Data Management'),
('0000000002', 'Accounting', 'Economics'),
('0000000007', 'Accounting', NULL),
('0000000009', 'Administrative', 'Marketing'),
('0000000003', 'Logistics', 'Data Mining'),
('0000000012', 'Administrative', 'System Security'),
('0000000017', 'Logistics', 'Business Logistics'),
('0000000021', 'Accounting', NULL),
('0000000029', 'Administrative', 'Public Relations'),
('0000000018', 'Accounting', NULL),
('0000000013', 'Administrative', 'Marketing'),
('0000000006', 'Logistics', 'Business Management')

;

INSERT INTO manages VALUES

('0000000012', 00000000010),
('0000000029', 00000000006),
('0000000009', 00000000009),
('0000000013', 00000000012)

;

INSERT INTO guide VALUES

('0000000024', 'Modern English History'),

('0000000011', 'South-Eastern Chinese History'),
('0000000015', 'Northern European Studies'),
('0000000020', 'Middle Eastern History'),
('0000000022', 'Japanese & Korean Mythology-History'),
('0000000027', '18th Century French History & Political Studies'),
('0000000004', 'Balkan Studies'),
('0000000026', 'Egyptian Myths & Legends'),
('0000000030', 'Eastern European Studies - The U.S.S.R')

;

INSERT INTO languages VALUES

('0000000024', 'English'),
('0000000011', 'Cantonese'),
('0000000015', 'German'),
('0000000015', 'Dutch'),
('0000000020', 'Arabic'),
('0000000022', 'Japanese'),
('0000000022', 'Korean'),
('0000000022', 'Mandarin'),
('0000000027', 'French'),
('0000000004', 'Turkish'),
('0000000004', 'Boulgarian'),
('0000000026', 'Egyptian'),
('0000000027', 'Spanish'),
('0000000030', 'Russian'),
('0000000030', 'Ukrainian'),
('0000000030', 'Romanian'),
('0000000030', 'Polish')

;

INSERT INTO IT VALUES

('0000000023', DEFAULT, '2023-03-21', NULL),
('0000000031', DEFAULT, '2023-03-21', NULL),
('0000000032', DEFAULT, '2023-03-21', NULL),
('0000000033', DEFAULT, '2023-03-21', NULL),
('0000000034', DEFAULT, '2023-03-21', NULL),
('0000000035', DEFAULT, '2023-03-21', NULL),
('0000000036', DEFAULT, '2023-03-21', NULL),
('0000000037', DEFAULT, '2023-03-21', NULL),
('0000000038', DEFAULT, '2023-03-21', NULL),
('0000000039', 'a', '2023-03-21', NULL)

;

INSERT INTO trip VALUES

(NULL, '2023-02-17 11:30:00', NULL, 96, 3400, 00000000006, '0000000022', '0000000008'),
(NULL, '2023-03-15 12:00:00', '2023-03-21 10:00:00', 123, 8745, 00000000012, '0000000015', NULL),
(NULL, '2023-04-05 10:00:00', '2023-04-18 11:45:00', 78, NULL, 00000000008, '0000000022', '0000000019'),
(NULL, '2023-04-21 12:15:00', '2023-04-28 10:00:00', 94, 2345, 00000000003, NULL, '0000000016'),
(NULL, '2023-05-16 18:00:00', '2023-05-26 10:15:00', 35, 9678, 00000000011, '0000000027', '0000000010'),
(NULL, '2023-07-13 14:45:00', '2023-07-19 10:00:00', 87, 3835, 00000000009, '0000000020', '0000000028'),
(NULL, '2023-08-27 08:15:00', '2023-08-31 10:45:00', 119, 6578, 00000000005, '0000000030', NULL),
(NULL, '2023-03-06 12:30:00', '2023-03-12 11:55:00', 120, 500, 00000000008, '0000000024', '0000000028'),
(NULL, '2024-02-03 11:30:00', NULL, 96, 3400, 00000000006, '0000000024', '0000000012'),
(NULL, '2024-02-21 12:00:00', '2024-02-21 10:00:00', 123, 8745, 00000000012, '0000000015', NULL),
(NULL, '2023-04-05 10:00:00', '2023-04-18 11:45:00', 78, NULL, 00000000008, '0000000022', '0000000019'),

```
(NULL, '2023-09-21 12:15:00', '2023-09-28 10:00:00', 94, 2345, 000000000003, NULL, '0000000016'),
(NULL, '2024-08-26 18:00:00', '2024-08-26 10:15:00', 35, 9678, 000000000011, '0000000027',
'0000000010'),
(NULL, '2023-10-13 14:45:00', '2023-10-19 10:00:00', 87, 3835, 000000000009, '0000000020',
'0000000028'),
(NULL, '2024-08-31 08:15:00', '2024-08-31 10:45:00', 119, 6578, 000000000005, '0000000030', NULL),
(NULL, '2023-07-06 12:30:00', NULL, 120, 500, 000000000008, '0000000024', '0000000028'),
(NULL, '2024-06-03 11:30:00', '2024-06-05 10:00:00', 96, 3400, 000000000006, '0000000011',
'0000000021'),
(NULL, '2024-02-21 12:00:00', '2024-02-21 10:00:00', 123, 8745, 000000000012, '0000000015', NULL),
(NULL, '2023-09-05 10:00:00', '2023-09-18 11:45:00', 78, NULL, 000000000008, '0000000022',
'0000000019'),
(NULL, '2024-01-28 12:15:00', '2024-01-28 10:00:00', 94, 2345, 000000000003, NULL, '0000000016'),
(NULL, '2023-11-16 18:00:00', NULL, 35, 9678, 000000000011, '0000000027', '0000000010'),
(NULL, '2024-12-19 14:45:00', '2024-12-21 10:00:00', 87, 3835, 000000000009, '0000000020',
'0000000028'),
(NULL, '2024-06-30 08:15:00', NULL, 119, 6578, 000000000005, '0000000030', NULL),
(NULL, '2023-02-06 12:30:00', '2023-02-14 11:55:00', 120, 500, 000000000008, '0000000024',
'0000000028')

;
```

INSERT INTO event VALUES

```
(000000000002, '2023-03-17 13:45:00', '2023-03-20 13:45:00', 'CES 2023'),
(000000000005, '2023-05-20 09:00:00', '2023-05-24 18:30:00', 'E3 2023'),
(000000000007, '2023-08-28 012:30:00', '2023-08-30 23:30:00', 'Super Smash Bros Ultimate
Tournament'),
(000000000004, '2023-04-23 10:30:00', '2023-04-27 14:45:00', 'The Game Awards')

;
```

INSERT INTO destination VALUES

```
(NULL, NULL, NULL, 'Local', 'French', NULL),
```

```
(NULL, NULL, NULL, 'Abroad', 'Egyptian', NULL),  
(NULL, NULL, NULL, 'Abroad', 'English', NULL),  
(NULL, NULL, NULL, 'Local', 'Boulgarian', NULL),  
(NULL, NULL, NULL, 'Abroad', 'German', NULL),  
(NULL, NULL, NULL, 'Local', 'Spanish', NULL),  
(NULL, NULL, NULL, 'Local', 'Japanese', NULL)
```

```
;
```

```
INSERT INTO travel_to VALUES
```

```
(000000000001, 000000000003, NULL, NULL),  
(000000000006, 000000000004, NULL, NULL),  
(000000000003, 000000000006, NULL, NULL),  
(000000000002, 000000000005, NULL, NULL),  
(000000000005, 000000000007, NULL, NULL),  
(000000000004, 000000000001, NULL, NULL),  
(000000000007, 000000000002, NULL, NULL)
```

```
;
```

```
INSERT INTO reservation VALUES
```

```
(000000000005, 78, 'Nikolaos', 'Anastasiadis', 'Adult'),  
(000000000005, 79, 'Kwnstantinos', 'Anastasiadis', 'Minor'),  
(000000000005, 80, 'Georgia', 'Manta', 'Adult'),  
(000000000002, 94, 'Iwannis', 'Papadimitriou', 'Adult'),  
(000000000003, 34, 'Elpida', 'Papadopoulou', 'Adult'),  
(000000000003, 35, 'Antonios', 'Stamatopoulos', 'Minor'),  
(000000000003, 36, 'Eleni', 'Stamatopoulou', 'Minor'),  
(000000000003, 37, 'Petros', 'Stamatopoulos', 'Adult'),  
(000000000006, 57, 'Iakwvos', 'Papagiwtou', 'Minor'),
```

(000000000006, 58, 'Apostolia', 'Dervisi', 'Adult'),
(000000000002, 112, 'Vasiliki', 'Petraki', 'Adult'),
(000000000007, 12, 'Andreas', 'Kwstopoulos', 'Adult'),
(000000000007, 13, 'Kalliopi', 'Dimopoulou', 'Adult'),
(000000000007, 14, 'Markos', 'Kwstopoulos', 'Minor'),
(000000000001, 88, 'Danai', 'Mauridi', 'Minor'),
(000000000001, 89, 'Alexandros', 'Mauridis', 'Adult'),
(000000000004, 126, 'Athanasios', 'Karpas', 'Adult'),
(000000000004, 98, 'Grigoria', 'Karpa', 'Minor'),
(000000000008, 1, 'Petros', 'Filipidis', 'Adult'),
(000000000008, 2, 'Maria', 'Giannaki', 'Adult'),
(000000000008, 3, 'Argiris', 'Filipidis', 'Minor')

;

INSERT INTO offer VALUES

(NULL, '2022-10-23', '2022-12-15', 350, 2),
(NULL, '2023-01-30', '2023-03-24', 560, 4),
(NULL, '2022-11-05', '2023-01-06', 420, 5)

;

procedureStatements.sql :

DELIMITER \$

--

```
CREATE PROCEDURE assign_driver(
IN new_wrk_AT CHAR(10),
IN new_wrk_name VARCHAR(20),
IN new_wrk_lname VARCHAR(20),
IN new_wrk_salary FLOAT(7, 2),
IN new_drv_license ENUM('A', 'B', 'C', 'D'),
IN new_drv_route ENUM('LOCAL', 'ABROAD'),
IN new_drv_experience TINYINT(4)
)
BEGIN
    DECLARE min_value INT;
    DECLARE selected_branch INT(11);

    SELECT MIN(number_of_drivers)
    INTO min_value
    FROM(
        SELECT COUNT(*) AS 'number_of_drivers'
        FROM branch
        INNER JOIN worker ON br_code = wrk_br_code
        INNER JOIN driver ON wrk_AT = drv_AT
        GROUP BY br_code
    ) AS temp;

    SELECT br_code
    INTO selected_branch
    FROM(
```



```
        SELECT br_code, COUNT(*) AS 'number_of_drivers'
        FROM branch
        INNER JOIN worker ON br_code = wrk_br_code
        INNER JOIN driver ON wrk_AT = drv_AT
        GROUP BY br_code
        HAVING number_of_drivers = min_value
        LIMIT 1
    ) AS temp;
```

```
INSERT INTO worker VALUES(
    new_wrk_AT,
    new_wrk_name,
    new_wrk_lname,
    new_wrk_salary,
    selected_branch
);
```

```
INSERT INTO driver VALUES(
    new_wrk_AT,
    new_drv_license,
    new_drv_route,
    new_drv_experience
);
```

```
END $
```

```
--
```

```
CREATE PROCEDURE check_branch_trips(
    IN selected_br_code INT(11),
    IN first_date DATE,
    IN second_date DATE
)
```

```

BEGIN

    select
        tr_cost, tr_maxseats,
        count(*) as 'reservations', tr_maxseats - count(*) as 'empty seats',
        d.wrk_lname AS 'Driver lname', d.wrk_name AS 'Driver name',
        g.wrk_lname AS 'Guide lname', g.wrk_name AS 'Guide name',
        tr_departure, tr_return
    from branch
    inner join trip on br_code = tr_br_code
    inner join reservation on tr_id = res_tr_id
    left join guide on gui_AT = tr_gui_AT
    left join driver on drv_AT = tr_drv_AT
    left join worker AS g on gui_AT = g.wrk_AT
    left join worker AS d on drv_AT = d.wrk_AT
    where br_code = selected_br_code
        AND tr_departure >= first_date
        AND tr_departure <= second_date
    group by tr_id;

END $

```

```
--
```

```

CREATE PROCEDURE delete_admin(
    IN selected_name VARCHAR(20),
    IN selected_lname VARCHAR(20)
)
BEGIN
    DECLARE check_AT CHAR(10);
    DECLARE check_branch INT(11);

    SET check_AT = NULL;
    SET check_branch = NULL;

```

```

SELECT adm_AT
INTO check_AT
FROM worker
LEFT JOIN admin ON wrk_AT = adm_AT
WHERE wrk_name = selected_name
      AND wrk_lname = selected_lname;

IF(check_AT IS NULL) THEN
    SELECT 'This worker is not an admin.' AS 'Message';
ELSE
    SELECT mng_br_code
    INTO check_branch
    FROM manages
    WHERE mng_adm_AT = check_AT;

    IF(check_branch IS NULL) THEN
        DELETE FROM worker
        WHERE wrk_AT = check_AT;

        SELECT 'Deleted successfully!' AS 'Message';
    ELSE
        SELECT 'This admin manages a branch, so he cannot be deleted.' AS
'Message';
    END IF;
END IF;
END $

--

CREATE PROCEDURE find_reservation_offer_by_fee(
IN bottom_limit FLOAT(7, 2),
IN upper_limit FLOAT(7, 2)

```

)

BEGIN

SELECT res_ofr_lname, res_ofr_name

FROM reservation_offer

WHERE res_ofr_fee >= bottom_limit

AND res_ofr_fee <= upper_limit;

END \$

--

CREATE PROCEDURE find_reservation_by_last_name(IN selected_lname VARCHAR(20))

BEGIN

SELECT res_ofr_lname, res_ofr_name, res_ofr_ofr_id, count(*)

FROM reservation_offer

WHERE res_ofr_lname = selected_lname

GROUP BY res_ofr_name, res_ofr_ofr_id;

END \$

--

DELIMITER ;

triggerStatements.sql :

DELIMITER \$

--

```
CREATE TRIGGER trip_insert_log_trigger
AFTER INSERT
ON trip
FOR EACH ROW
BEGIN
    DECLARE user_AT CHAR(10);

    SET user_AT = (
        SELECT user_IT_AT
        FROM current_active_user
    );

    INSERT INTO table_log VALUES
    ('Trip', 'Insert', user_AT);
END $
```

--

```
CREATE TRIGGER trip_update_log_trigger
AFTER UPDATE
ON trip
FOR EACH ROW
BEGIN
    DECLARE user_AT CHAR(10);

    SET user_AT = (
```

```

        SELECT user_IT_AT
        FROM current_active_user
    );

    INSERT INTO table_log VALUES
    ('Trip', 'UPDATE', user_AT);
END $

--

CREATE TRIGGER trip_delete_log_trigger
AFTER DELETE
ON trip
FOR EACH ROW
BEGIN
    DECLARE user_AT CHAR(10);

    SET user_AT = (
        SELECT user_IT_AT
        FROM current_active_user
    );

    INSERT INTO table_log VALUES
    ('Trip', 'Delete', user_AT);
END $

--

CREATE TRIGGER reservation_insert_log_trigger
AFTER INSERT
ON reservation
FOR EACH ROW

```

```

BEGIN
    DECLARE user_AT CHAR(10);

    SET user_AT = (
        SELECT user_IT_AT
        FROM current_active_user
    );

    INSERT INTO table_log VALUES
        ('Reservation', 'Insert', user_AT);
END $

--

CREATE TRIGGER reservation_update_log_trigger
AFTER UPDATE
ON reservation
FOR EACH ROW
BEGIN
    DECLARE user_AT CHAR(10);

    SET user_AT = (
        SELECT user_IT_AT
        FROM current_active_user
    );

    INSERT INTO table_log VALUES
        ('Reservation', 'Update', user_AT);
END $

--

```

```
CREATE TRIGGER reservation_delete_log_trigger
AFTER DELETE
ON reservation
FOR EACH ROW
BEGIN
    DECLARE user_AT CHAR(10);

    SET user_AT = (
        SELECT user_IT_AT
        FROM current_active_user
    );

    INSERT INTO table_log VALUES
    ('Reservation', 'Delete', user_AT);
END $
```

--

```
CREATE TRIGGER event_insert_log_trigger
AFTER INSERT
ON event
FOR EACH ROW
BEGIN
    DECLARE user_AT CHAR(10);

    SET user_AT = (
        SELECT user_IT_AT
        FROM current_active_user
    );

    INSERT INTO table_log VALUES
    ('Event', 'Insert', user_AT);
```


END \$

--

CREATE TRIGGER event_update_log_trigger

AFTER UPDATE

ON event

FOR EACH ROW

BEGIN

 DECLARE user_AT CHAR(10);

 SET user_AT = (

 SELECT user_IT_AT

 FROM current_active_user

);

 INSERT INTO table_log VALUES

 ('Event', 'Update', user_AT);

END \$

--

CREATE TRIGGER event_delete_log_trigger

AFTER DELETE

ON event

FOR EACH ROW

BEGIN

 DECLARE user_AT CHAR(10);

 SET user_AT = (

 SELECT user_IT_AT

 FROM current_active_user

```

);

INSERT INTO table_log VALUES
('Event', 'Delete', user_AT);
END $

--

CREATE TRIGGER travel_to_insert_log_trigger
AFTER INSERT
ON travel_to
FOR EACH ROW
BEGIN
    DECLARE user_AT CHAR(10);

    SET user_AT = (
        SELECT user_IT_AT
        FROM current_active_user
    );

    INSERT INTO table_log VALUES
    ('Travel_to', 'Insert', user_AT);
END $

--

CREATE TRIGGER travel_to_update_log_trigger
AFTER UPDATE
ON travel_to
FOR EACH ROW
BEGIN
    DECLARE user_AT CHAR(10);

```

```

        SET user_AT = (
            SELECT user_IT_AT
            FROM current_active_user
        );

        INSERT INTO table_log VALUES
        ('Travel_to', 'Update', user_AT);
END $

--

CREATE TRIGGER travel_to_delete_log_trigger
AFTER DELETE
ON travel_to
FOR EACH ROW
BEGIN
    DECLARE user_AT CHAR(10);

    SET user_AT = (
        SELECT user_IT_AT
        FROM current_active_user
    );

    INSERT INTO table_log VALUES
    ('Travel_to', 'Delete', user_AT);
END $

--

CREATE TRIGGER destination_insert_log_trigger
AFTER INSERT

```

```
ON destination
FOR EACH ROW
BEGIN
    DECLARE user_AT CHAR(10);

    SET user_AT = (
        SELECT user_IT_AT
        FROM current_active_user
    );

    INSERT INTO table_log VALUES
    ('Destination', 'Insert', user_AT);
END $
```

--

```
CREATE TRIGGER destination_update_log_trigger
AFTER UPDATE
ON destination
FOR EACH ROW
BEGIN
    DECLARE user_AT CHAR(10);

    SET user_AT = (
        SELECT user_IT_AT
        FROM current_active_user
    );

    INSERT INTO table_log VALUES
    ('Destination', 'Update', user_AT);
END $
```

--

```
CREATE TRIGGER destination_delete_log_trigger
AFTER DELETE
ON destination
FOR EACH ROW
BEGIN
    DECLARE user_AT CHAR(10);

    SET user_AT = (
        SELECT user_IT_AT
        FROM current_active_user
    );

    INSERT INTO table_log VALUES
    ('Destination', 'Delete', user_AT);
END $
```

--

```
CREATE TRIGGER chech_reservation_of_trip
BEFORE UPDATE
ON trip
FOR EACH ROW
BEGIN
    DECLARE num_of_reservations INT;

    SELECT COUNT(*)
    INTO num_of_reservations
    FROM trip
    INNER JOIN reservation ON tr_id = res_tr_id
    WHERE tr_id = OLD.tr_id;
```

```

        IF(num_of_reservations != 0) THEN
            SIGNAL SQLSTATE VALUE '45000'
            SET MESSAGE_TEXT = 'ERROR';
        END IF;
    END $

--

CREATE TRIGGER check_worker_salary_change
BEFORE UPDATE
ON worker
FOR EACH ROW
BEGIN
    IF(OLD.wrk_salary > NEW.wrk_salary) THEN
        SIGNAL SQLSTATE VALUE '45000'
        SET MESSAGE_TEXT = 'ERROR';
    END IF;
END $

--

DELIMITER ;

```

bulkInserts.sql :

```
CREATE TABLE IF NOT EXISTS temp_names(
```

```
name VARCHAR(20) NOT NULL,
```

```
lname VARCHAR(20) NOT NULL
```

```
);
```

```
--
```

```
LOAD DATA
```

```
INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/names.txt'
```

```
INTO TABLE travel_agency.temp_names
```

```
FIELDS TERMINATED BY '\t'
```

```
ENCLOSED BY ''
```

```
LINES TERMINATED BY '\r\n';
```

```
--
```

```
INSERT INTO reservation_offer(res_ofr_id, res_ofr_lname, res_ofr_name, res_ofr_ofr_id,  
res_ofr_fee)
```

```
SELECT NULL, lname, name, 1, ROUND(RAND()*(200 - 50 + 1) + 50, 0)
```

```
FROM temp_names;
```

```
ALTER TABLE reservation_offer AUTO_INCREMENT = 10001;
```

```
INSERT INTO reservation_offer(res_ofr_id, res_ofr_lname, res_ofr_name, res_ofr_ofr_id,  
res_ofr_fee)
```

```
SELECT NULL, lname, name, 1, ROUND(RAND()*(200 - 50 + 1) + 50, 0)
```

```
FROM temp_names;
```

```
ALTER TABLE reservation_offer AUTO_INCREMENT = 20001;
```

```
INSERT INTO reservation_offer(res_ofr_id, res_ofr_lname, res_ofr_name, res_ofr_ofr_id,
res_ofr_fee)
```

```
SELECT NULL, lname, name, 2, ROUND(RAND()*(200 - 50 + 1) + 50, 0)
```

```
FROM temp_names;
```

```
ALTER TABLE reservation_offer AUTO_INCREMENT = 30001;
```

```
INSERT INTO reservation_offer(res_ofr_id, res_ofr_lname, res_ofr_name, res_ofr_ofr_id,
res_ofr_fee)
```

```
SELECT NULL, lname, name, 2, ROUND(RAND()*(200 - 50 + 1) + 50, 0)
```

```
FROM temp_names;
```

```
ALTER TABLE reservation_offer AUTO_INCREMENT = 40001;
```

```
INSERT INTO reservation_offer(res_ofr_id, res_ofr_lname, res_ofr_name, res_ofr_ofr_id,
res_ofr_fee)
```

```
SELECT NULL, lname, name, 3, ROUND(RAND()*(200 - 50 + 1) + 50, 0)
```

```
FROM temp_names;
```

```
ALTER TABLE reservation_offer AUTO_INCREMENT = 50001;
```

```
INSERT INTO reservation_offer(res_ofr_id, res_ofr_lname, res_ofr_name, res_ofr_ofr_id,
res_ofr_fee)
```

```
SELECT NULL, lname, name, 3, ROUND(RAND()*(200 - 50 + 1) + 50, 0)
```

```
FROM temp_names;
```

```
ALTER TABLE reservation_offer AUTO_INCREMENT = 60001;
```

```
--
```

```
DROP TABLE IF EXISTS temp_names;
```


sourceStatements.sql :

-- SOURCE STATEMENTS

source C:\Users\Mako\Desktop\Project DB 22-23\Project_Files\createStatements.sql

source C:\Users\Mako\Desktop\Project DB 22-23\Project_Files\insertStatements.sql

source C:\Users\Mako\Desktop\Project DB 22-23\Project_Files\procedureStatements.sql

source C:\Users\Mako\Desktop\Project DB 22-23\Project_Files\triggerStatements.sql

source C:\Users\Mako\Desktop\Project DB 22-23\Project_Files\bulkInserts.sql

Παράρτημα Κώδικα GUI (JAVA)

DBProject.java :

```
import java.sql.*;

public class DBProject {

    private static final String DB_URL = "jdbc:mysql://localhost:3306/travel_agency";

    private static final String USERNAME = "root";

    private static final String PASSWORD = "2912002";

    private static Connection myConnection;

    public static void main(String[] args) {

        try{

            myConnection = DriverManager.getConnection(DB_URL, USERNAME, PASSWORD);

            LoginWindow loginWin = new LoginWindow();

        }

        catch(SQLException ex){

            ex.printStackTrace();

        }

        finally{

        }

    }

    public static Connection passConnection(){

        return myConnection;

    }

}
```

LoginWindow.java :

```
import java.awt.*;
import java.sql.*;
import java.awt.event.KeyEvent;

public class LoginWindow extends javax.swing.JFrame {

    private static final String DB_URL = "jdbc:mysql://localhost:3306/travel_agency";
    private static final String USERNAME = "root";
    private static final String PASSWORD = "2912002";
    private static Connection myConnection;
    private static PreparedStatement myPreparedStatement;
    private static ResultSet myResultSet;
    private static ResultSetMetaData myMetaData;

    public LoginWindow() {
        initComponents();
        setVisible(true);

        setSize(800, 500);
        setLocationRelativeTo(null);
        setResizable(false);

        mainPanel.setBackground(new Color(666666));

        usernameErrorMessage.setVisible(false);
        usernameErrorMessage.setForeground(Color.red);

        passwordErrorMessage.setVisible(false);
        passwordErrorMessage.setForeground(Color.red);

        loginResult.setForeground(Color.red);
    }
}
```

```

private void loginProcedure(){
    usernameErrorMessage.setVisible(false);
    passwordErrorMessage.setVisible(false);

    if(usernameVariable.getText().equals("") || passwordVariable.getText().equals("")){
        if(usernameVariable.getText().equals(""))
            usernameErrorMessage.setVisible(true);
        if(passwordVariable.getText().equals(""))
            passwordErrorMessage.setVisible(true);
    }
    else{
        try{
            myConnection = DriverManager.getConnection(DB_URL, USERNAME, PASSWORD);
            String SEARCH_IT_QUERY =
                "SELECT wrk_lname, IT_password, IT_AT FROM worker INNER JOIN IT on IT_AT = wrk_AT WHERE
wrk_lname = ?";
            myPreparedStatement = myConnection.prepareStatement(SEARCH_IT_QUERY);
            myPreparedStatement.setString(1, usernameVariable.getText());
            myResultSet = myPreparedStatement.executeQuery();
            myMetaData = myResultSet.getMetaData();

            if(myResultSet.next()){
                boolean userFoundFlag = false;

                do{
                    if(myResultSet.getObject(2).toString().equals(passwordVariable.getText())){
                        userFoundFlag = true;
                        break;
                    }
                }while(myResultSet.next());

                if(userFoundFlag){
                    String CLEAR_USER = "DELETE FROM current_active_user";

                    String LOGIN_QUERY = "INSERT INTO current_active_user VALUES (" +
myResultSet.getObject(3).toString() + ")";

```

```

        myPreparedStatement = myConnection.prepareStatement(CLEAR_USER);
        myPreparedStatement.executeUpdate();
        myPreparedStatement = myConnection.prepareStatement(LOGIN_QUERY);
        myPreparedStatement.executeUpdate();

        MainMenu mainMen = new MainMenu();
        dispose();
    }
    else loginResult.setText("Wrong password for this User");
}
else loginResult.setText("There is no user with this Username. Please try again");
}
catch(SQLException ex){
    ex.printStackTrace();
}
finally{

}
}
}
}

```

```

@SuppressWarnings("unchecked")

```

```

// <editor-fold defaultstate="collapsed" desc="Generated Code">

```

```

private void initComponents() {

```

```

    mainPanel = new javax.swing.JPanel();
    layeredPanel = new javax.swing.JLayeredPane();
    usernameVariable = new javax.swing.JTextField();
    jLabel1 = new javax.swing.JLabel();
    loginResult = new javax.swing.JLabel();
    loginButton = new javax.swing.JButton();
    passwordErrorMessage = new javax.swing.JLabel();
    usernameErrorMessage = new javax.swing.JLabel();
    usernameLabel = new javax.swing.JLabel();

```

```
passwordVariable = new javax.swing.JTextField();
passwordLabel = new javax.swing.JLabel();

setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);

usernameVariable.setForeground(new java.awt.Color(0, 0, 0));
usernameVariable.addKeyListener(new java.awt.event.KeyAdapter() {
    public void keyPressed(java.awt.event.KeyEvent evt) {
        usernameVariableKeyPressed(evt);
    }
});

jLabel1.setFont(new java.awt.Font("Segoe UI", 1, 36)); // NOI18N
jLabel1.setForeground(new java.awt.Color(255, 255, 255));
jLabel1.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
jLabel1.setText("Login Window");
jLabel1.setAutoscrolls(true);

loginResult.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
loginResult.setText(" ");
loginResult.setAutoscrolls(true);

loginButton.setText("Login");
loginButton.setAutoscrolls(true);
loginButton.addMouseListener(new java.awt.event.MouseAdapter() {
    public void mouseClicked(java.awt.event.MouseEvent evt) {
        loginButtonMouseClicked(evt);
    }
});

passwordErrorMessage.setText("You must enter a Password!");
passwordErrorMessage.setAutoscrolls(true);

usernameErrorMessage.setText("You must enter a Username!");
```

```
usernameErrorMessage.setAutoscrolls(true);
```

```
usernameLabel.setForeground(new java.awt.Color(255, 255, 255));
```

```
usernameLabel.setText("Username:");
```

```
usernameLabel.setAutoscrolls(true);
```

```
passwordVariable.setForeground(new java.awt.Color(0, 0, 0));
```

```
passwordVariable.addKeyListener(new java.awt.event.KeyAdapter() {
```

```
    public void keyPressed(java.awt.event.KeyEvent evt) {
```

```
        passwordVariableKeyPressed(evt);
```

```
    }
```

```
});
```

```
passwordLabel.setForeground(new java.awt.Color(255, 255, 255));
```

```
passwordLabel.setText("Password:");
```

```
passwordLabel.setAutoscrolls(true);
```

```
layeredPanel.setLayer(usernameVariable, javax.swing.JLayeredPane.DEFAULT_LAYER);
```

```
layeredPanel.setLayer(jLabel1, javax.swing.JLayeredPane.DEFAULT_LAYER);
```

```
layeredPanel.setLayer(loginResult, javax.swing.JLayeredPane.DEFAULT_LAYER);
```

```
layeredPanel.setLayer(loginButton, javax.swing.JLayeredPane.DEFAULT_LAYER);
```

```
layeredPanel.setLayer(passwordErrorMessage, javax.swing.JLayeredPane.DEFAULT_LAYER);
```

```
layeredPanel.setLayer(usernameErrorMessage, javax.swing.JLayeredPane.DEFAULT_LAYER);
```

```
layeredPanel.setLayer(usernameLabel, javax.swing.JLayeredPane.DEFAULT_LAYER);
```

```
layeredPanel.setLayer(passwordVariable, javax.swing.JLayeredPane.DEFAULT_LAYER);
```

```
layeredPanel.setLayer(passwordLabel, javax.swing.JLayeredPane.DEFAULT_LAYER);
```

```
javax.swing.GroupLayout layeredPanelLayout = new javax.swing.GroupLayout(layeredPanel);
```

```
layeredPanel.setLayout(layeredPanelLayout);
```

```
layeredPanelLayout.setHorizontalGroup(
```

```
    layeredPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
```

```
        .addGroup(layeredPanelLayout.createSequentialGroup()
```

```
            .addContainerGap()
```

```
            .addGroup(layeredPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
```

```

        .addGroup(layeredPanelLayout.createSequentialGroup())
        .addGap(40, 40, 40)

    .addGroup(layeredPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING)
        .addComponent(jLabel1, javax.swing.GroupLayout.PREFERRED_SIZE, 272,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addGroup(javax.swing.GroupLayout.Alignment.LEADING,
layeredPanelLayout.createSequentialGroup())

    .addGroup(layeredPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING)
        .addComponent(passwordLabel)
        .addComponent(usernameLabel))
        .addGap(18, 18, 18)

    .addGroup(layeredPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING, false)
        .addComponent(passwordVariable)
        .addComponent(usernameVariable)
        .addComponent(loginButton, javax.swing.GroupLayout.PREFERRED_SIZE, 217,
javax.swing.GroupLayout.PREFERRED_SIZE)))
        .addGap(18, 18, 18)

    .addGroup(layeredPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addComponent(usernameErrorMessage)
        .addComponent(passwordErrorMessage)))
        .addComponent(loginResult, javax.swing.GroupLayout.PREFERRED_SIZE, 445,
javax.swing.GroupLayout.PREFERRED_SIZE))
        .addContainerGap())
    );

    layeredPanelLayout.setVerticalGroup(
        layeredPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(layeredPanelLayout.createSequentialGroup())
        .addContainerGap()
        .addComponent(jLabel1, javax.swing.GroupLayout.PREFERRED_SIZE, 47,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
        .addGroup(layeredPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
            .addComponent(usernameLabel)

```



```

        .addComponent(usernameVariable, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)

        .addComponent(usernameErrorMessage))

    .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)

    .addGroup(layeredPanellLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)

        .addComponent(passwordLabel)

        .addComponent(passwordVariable, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)

        .addComponent(passwordErrorMessage))

    .addGap(34, 34, 34)

    .addComponent(loginResult)

    .addGap(18, 18, 18)

    .addComponent(loginButton)

    .addContainerGap())
);

```

```

javax.swing.GroupLayout mainPanellLayout = new javax.swing.GroupLayout(mainPanel);
mainPanel.setLayout(mainPanellLayout);
mainPanellLayout.setHorizontalGroup(

    mainPanellLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

    .addGroup(javax.swing.GroupLayout.Alignment.TRAILING, mainPanellLayout.createSequentialGroup())

    .addContainerGap(147, Short.MAX_VALUE)

    .addComponent(layeredPanel, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)

    .addGap(144, 144, 144))
);
mainPanellLayout.setVerticalGroup(

    mainPanellLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

    .addGroup(mainPanellLayout.createSequentialGroup())

    .addGap(64, 64, 64)

    .addComponent(layeredPanel, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)

    .addContainerGap(224, Short.MAX_VALUE))
);

```

```

javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());

```

```

    getContentPane().setLayout(layout);

    layout.setHorizontalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addComponent(mainPanel, javax.swing.GroupLayout.DEFAULT_SIZE,
                javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
    );

    layout.setVerticalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addComponent(mainPanel, javax.swing.GroupLayout.DEFAULT_SIZE,
                javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
    );

    pack();
} // </editor-fold>

private void loginButtonMouseClicked(java.awt.event.MouseEvent evt) {
    loginProcedure();
}

private void passwordVariableKeyPressed(java.awt.event.KeyEvent evt) {
    if(evt.getKeyCode() == KeyEvent.VK_ENTER) {
        loginProcedure();
    }
}

private void usernameVariableKeyPressed(java.awt.event.KeyEvent evt) {
    if(evt.getKeyCode() == KeyEvent.VK_ENTER) {
        loginProcedure();
    }
}

public static void main(String args[]) {
    /* Set the Nimbus look and feel */
    //<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
    /* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.

```

```

* For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
*/
try {
    for (javax.swing.UIManager.LookAndFeelInfo info : javax.swing.UIManager.getInstalledLookAndFeels()) {
        if ("Nimbus".equals(info.getName())) {
            javax.swing.UIManager.setLookAndFeel(info.getClassName());
            break;
        }
    }
} catch (ClassNotFoundException ex) {
    java.util.logging.Logger.getLogger(LoginWindow.class.getName()).log(java.util.logging.Level.SEVERE,
null, ex);
} catch (InstantiationException ex) {
    java.util.logging.Logger.getLogger(LoginWindow.class.getName()).log(java.util.logging.Level.SEVERE,
null, ex);
} catch (IllegalAccessException ex) {
    java.util.logging.Logger.getLogger(LoginWindow.class.getName()).log(java.util.logging.Level.SEVERE,
null, ex);
} catch (javax.swing.UnsupportedLookAndFeelException ex) {
    java.util.logging.Logger.getLogger(LoginWindow.class.getName()).log(java.util.logging.Level.SEVERE,
null, ex);
}
//</editor-fold>

```

```

/* Create and display the form */
java.awt.EventQueue.invokeLater(new Runnable() {
    public void run() {
        new LoginWindow().setVisible(true);
    }
});
}

```

```

// Variables declaration - do not modify
private javax.swing.JLabel jLabel1;
private javax.swing.JLayeredPane layeredPanel;
private javax.swing.JButton loginButton;

```

```
private javax.swing.JLabel loginResult;
private javax.swing.JPanel mainPanel;
private javax.swing.JLabel passwordErrorMessage;
private javax.swing.JLabel passwordLabel;
private javax.swing.JTextField passwordVariable;
private javax.swing.JLabel usernameErrorMessage;
private javax.swing.JLabel usernameLabel;
private javax.swing.JTextField usernameVariable;
// End of variables declaration
}
```

MainMenu.java :

```
import java.awt.Color;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.logging.Level;
import java.util.logging.Logger;
import javax.swing.SwingConstants;

public class MainMenu extends javax.swing.JFrame {

    private static final String DB_URL = "jdbc:mysql://localhost:3306/travel_agency";
    private static final String USERNAME = "root";
    private static final String PASSWORD = "2912002";
    private static Connection myConnection;
    private static PreparedStatement myPreparedStatement;

    public MainMenu() {
        initComponents();
        setVisible(true);

        setSize(800, 550);
        setLocationRelativeTo(null);
        setResizable(false);

        mainPanel.setBackground(new Color(666666));

        tableNameLabel.setForeground(Color.WHITE);
        tableNameLabel.setHorizontalAlignment(SwingConstants.CENTER);
    }

    @SuppressWarnings("unchecked")
```

```
// <editor-fold defaultstate="collapsed" desc="Generated Code">
```

```
private void initComponents() {
```

```
    mainPanel = new javax.swing.JPanel();
```

```
    tableNameLabel = new javax.swing.JLabel();
```

```
    logWindowButton = new javax.swing.JButton();
```

```
    editTableMenuButton = new javax.swing.JButton();
```

```
    newITButton = new javax.swing.JButton();
```

```
    reservationinfoButton = new javax.swing.JButton();
```

```
    branchInfoButton = new javax.swing.JButton();
```

```
    branchTripInfoButton = new javax.swing.JButton();
```

```
    logoutButton = new javax.swing.JButton();
```

```
    setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
```

```
    tableNameLabel.setFont(new java.awt.Font("Segoe UI", 1, 24)); // NOI18N
```

```
    tableNameLabel.setText("Main Menu");
```

```
    logWindowButton.setText("Log");
```

```
    logWindowButton.addMouseListener(new java.awt.event.MouseAdapter() {
```

```
        public void mouseClicked(java.awt.event.MouseEvent evt) {
```

```
            logWindowButtonMouseClicked(evt);
```

```
        }
```

```
    });
```

```
    editTableMenuButton.setText("Insert/Update/Delete");
```

```
    editTableMenuButton.addMouseListener(new java.awt.event.MouseAdapter() {
```

```
        public void mouseClicked(java.awt.event.MouseEvent evt) {
```

```
            editTableMenuButtonMouseClicked(evt);
```

```
        }
```

```
    });
```

```
    newITButton.setText("Add new IT");
```

```
    newITButton.addMouseListener(new java.awt.event.MouseAdapter() {
```

```
    public void mouseClicked(java.awt.event.MouseEvent evt) {  
        newITButtonMouseClicked(evt);  
    }  
});
```

```
reservationInfoButton.setText("Customer Reservation Info");  
reservationInfoButton.addMouseListener(new java.awt.event.MouseAdapter() {  
    public void mouseClicked(java.awt.event.MouseEvent evt) {  
        reservationInfoButtonMouseClicked(evt);  
    }  
});
```

```
branchInfoButton.setText("Branch Info");  
branchInfoButton.addMouseListener(new java.awt.event.MouseAdapter() {  
    public void mouseClicked(java.awt.event.MouseEvent evt) {  
        branchInfoButtonMouseClicked(evt);  
    }  
});
```

```
branchTripInfoButton.setText("Branch Trip Info");  
branchTripInfoButton.addMouseListener(new java.awt.event.MouseAdapter() {  
    public void mouseClicked(java.awt.event.MouseEvent evt) {  
        branchTripInfoButtonMouseClicked(evt);  
    }  
});
```

```
logoutButton.setText("Logout");  
logoutButton.addMouseListener(new java.awt.event.MouseAdapter() {  
    public void mouseClicked(java.awt.event.MouseEvent evt) {  
        logoutButtonMouseClicked(evt);  
    }  
});
```

```
javax.swing.GroupLayout mainPanelLayout = new javax.swing.GroupLayout(mainPanel);
```

```

mainPanel.setLayout(mainPanelLayout);

mainPanelLayout.setHorizontalGroup(
    mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(mainPanelLayout.createSequentialGroup()
            .addGap(87, 87, 87)
            .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
false)
                .addComponent(reservationinfoButton, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
                .addComponent(newITButton, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
                .addComponent(editTableMenuButton, javax.swing.GroupLayout.DEFAULT_SIZE, 201,
Short.MAX_VALUE))
            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, 155, Short.MAX_VALUE)
            .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
false)
                .addComponent(logWindowButton, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
                .addComponent(branchInfoButton, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
                .addComponent(branchTripInfoButton, javax.swing.GroupLayout.DEFAULT_SIZE, 201,
Short.MAX_VALUE))
            .addGap(124, 124, 124))
        .addGroup(mainPanelLayout.createSequentialGroup()
            .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                .addGroup(mainPanelLayout.createSequentialGroup()
                    .addGap(225, 225, 225)
                    .addComponent(tableNameLabel, javax.swing.GroupLayout.PREFERRED_SIZE, 285,
javax.swing.GroupLayout.PREFERRED_SIZE))
                .addGroup(mainPanelLayout.createSequentialGroup()
                    .addGap(288, 288, 288)
                    .addComponent(logoutButton, javax.swing.GroupLayout.PREFERRED_SIZE, 160,
javax.swing.GroupLayout.PREFERRED_SIZE)))
            .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))
    );

mainPanelLayout.setVerticalGroup(
    mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(mainPanelLayout.createSequentialGroup()

```



```

        .addGap(19, 19, 19)

        .addComponent(tableNameLabel, javax.swing.GroupLayout.PREFERRED_SIZE, 47,
javax.swing.GroupLayout.PREFERRED_SIZE)

        .addGap(27, 27, 27)

        .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)

            .addComponent(logWindowButton, javax.swing.GroupLayout.PREFERRED_SIZE, 53,
javax.swing.GroupLayout.PREFERRED_SIZE)

            .addComponent(editTableMenuButton, javax.swing.GroupLayout.PREFERRED_SIZE, 53,
javax.swing.GroupLayout.PREFERRED_SIZE))

        .addGap(74, 74, 74)

        .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)

            .addComponent(newITButton, javax.swing.GroupLayout.PREFERRED_SIZE, 53,
javax.swing.GroupLayout.PREFERRED_SIZE)

            .addComponent(branchTripInfoButton, javax.swing.GroupLayout.PREFERRED_SIZE, 53,
javax.swing.GroupLayout.PREFERRED_SIZE))

        .addGap(71, 71, 71)

        .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
false)

            .addComponent(reservationinfoButton, javax.swing.GroupLayout.DEFAULT_SIZE, 53,
Short.MAX_VALUE)

            .addComponent(branchInfoButton, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))

        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, 40, Short.MAX_VALUE)

        .addComponent(logoutButton)

        .addGap(37, 37, 37))
    );

    javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
    getContentPane().setLayout(layout);
    layout.setHorizontalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addComponent(mainPanel, javax.swing.GroupLayout.Alignment.TRAILING,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
    );
    layout.setVerticalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addComponent(mainPanel, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
    );

```

```
);
```

```
pack();
```

```
}// </editor-fold>
```

```
private void editTableMenuButtonMouseClicked(java.awt.event.MouseEvent evt) {  
    EditTableMenu editTabMen = new EditTableMenu();  
    dispose();  
}
```

```
private void logWindowButtonMouseClicked(java.awt.event.MouseEvent evt) {  
    LogPage logPage = new LogPage();  
    dispose();  
}
```

```
private void newITButtonMouseClicked(java.awt.event.MouseEvent evt) {  
    AddITPage addNewITPage = new AddITPage();  
    dispose();  
}
```

```
private void branchTripInfoButtonMouseClicked(java.awt.event.MouseEvent evt) {  
    BranchTripInfoPage brInfPage = new BranchTripInfoPage();  
    dispose();  
}
```

```
private void reservationinfoButtonMouseClicked(java.awt.event.MouseEvent evt) {  
    CustomerReservationPage brInfPage = new CustomerReservationPage();  
    dispose();  
}
```

```
private void branchInfoButtonMouseClicked(java.awt.event.MouseEvent evt) {  
    BranchInfo brInfPage = new BranchInfo();  
    dispose();  
}
```

```

private void logoutButtonMouseClicked(java.awt.event.MouseEvent evt) {
    try {
        myConnection = DriverManager.getConnection(DB_URL, USERNAME, PASSWORD);

        String LOGOUT_QUERY = "DELETE FROM current_active_user";

        myPreparedStatement = myConnection.prepareStatement(LOGOUT_QUERY);
        myPreparedStatement.executeUpdate();

        LoginWindow loginWin = new LoginWindow();
        dispose();
    }
    catch (SQLException ex) {
        ex.printStackTrace();
    }
}

public static void main(String args[]) {
    /* Set the Nimbus look and feel */
    //<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
    /* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
    * For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
    */
    try {
        for (javax.swing.UIManager.LookAndFeelInfo info : javax.swing.UIManager.getInstalledLookAndFeels()) {
            if ("Nimbus".equals(info.getName())) {
                javax.swing.UIManager.setLookAndFeel(info.getClassName());
                break;
            }
        }
    } catch (ClassNotFoundException ex) {
        java.util.logging.Logger.getLogger(MainMenu.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
    } catch (InstantiationException ex) {

```

```

        java.util.logging.Logger.getLogger(MainMenu.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
    } catch (IllegalAccessException ex) {
        java.util.logging.Logger.getLogger(MainMenu.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
    } catch (javax.swing.UnsupportedLookAndFeelException ex) {
        java.util.logging.Logger.getLogger(MainMenu.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
    }
}
//</editor-fold>

```

```

/* Create and display the form */
java.awt.EventQueue.invokeLater(new Runnable() {
    public void run() {
        new MainMenu().setVisible(true);
    }
});
}

```

```

// Variables declaration - do not modify
private javax.swing.JButton branchInfoButton;
private javax.swing.JButton branchTripInfoButton;
private javax.swing.JButton editTableMenuButton;
private javax.swing.JButton logWindowButton;
private javax.swing.JButton logoutButton;
private javax.swing.JPanel mainPanel;
private javax.swing.JButton newITButton;
private javax.swing.JButton reservationinfoButton;
private javax.swing.JLabel tableNameLabel;
// End of variables declaration
}

```

EditTableMenu.java :

```
import java.awt.*;
import java.sql.*;
import java.util.logging.Level;
import java.util.logging.Logger;
import javax.swing.JFrame;
import javax.swing.JTable;
import javax.swing.table.DefaultTableModel;

public class EditTableMenu extends javax.swing.JFrame {

    private static final String DB_URL = "jdbc:mysql://localhost:3306/travel_agency";
    private static final String USERNAME = "root";
    private static final String PASSWORD = "2912002";
    private static Connection myConnection;
    private static PreparedStatement myPreparedStatement;
    private static ResultSet myResultSet;
    private static ResultSetMetaData myMetaData;

    String GET_TABLE_NAMES_QUERY = "SHOW TABLES";
    String GET_TABLE_QUERY = "SELECT * FROM ";

    private final String[] actions = {"Action", "Insert", "Update", "Delete"};

    public static JTable currentTableReference;
    public static String[] currentColumnTypes;
    public static String currentSelectedTable;
    public static String[] currentTableColumns;
    public static DefaultTableModel currentTableModel;
    public static boolean activeSubWindow = false;
```

```

public EditTableMenu() {
    initComponents();
    setVisible(true);

    setSize(1100, 800);
    setLocationRelativeTo(null);
    setResizable(false);

    mainPanel.setBackground(new Color(666666));

    tableScrollPanel.setVisible(false);

    tableData.setEnabled(false);

    actionSelectionText.setForeground(Color.WHITE);
    tableSelectionText.setForeground(Color.WHITE);
    normalText1.setForeground(Color.WHITE);

    actionSelectionErrorMessage.setForeground(Color.RED);
    tableSelectionErrorMessage.setForeground(Color.RED);
    actionSelectionErrorMessage.setVisible(false);
    tableSelectionErrorMessage.setVisible(false);

    initializeThisMenu();
}

private void initializeThisMenu(){
    try {
        myConnection = DriverManager.getConnection(DB_URL, USERNAME, PASSWORD);
        myPreparedStatement = myConnection.prepareStatement(GET_TABLE_NAMES_QUERY);
        myResultSet = myPreparedStatement.executeQuery();

        for(String action : actions)
            actionSelection.addItem(action);
    }
}

```

```

        tableSelection.addItem("Table");

        while(myResultSet.next())

            tableSelection.addItem(myResultSet.getObject(1).toString());

    }

    catch (SQLException ex) {

        ex.printStackTrace();

    }

    finally{

        if(myConnection != null)

            try{

                myConnection.close();

            } catch(SQLException ex){}

        if(myPreparedStatement != null)

            try{

                myPreparedStatement.close();

            } catch(SQLException ex){}

        if(myResultSet != null)

            try{

                myResultSet.close();

            } catch(SQLException ex){}

    }

}

private void fillTable(){

    try {

        myConnection = DriverManager.getConnection(DB_URL, USERNAME, PASSWORD);

        myPreparedStatement =

myConnection.prepareStatement(GET_TABLE_QUERY.concat(tableSelection.getSelectedItem().toString()));

        myResultSet = myPreparedStatement.executeQuery();

        myMetaData = myResultSet.getMetaData();

```

```

DefaultTableModel tModel = (DefaultTableModel) tableData.getModel();
int newColumnCount = myMetaData.getColumnCount();
currentTableColumns = new String[newColumnCount];
currentColumnTypes = new String[newColumnCount];
currentTableModel = tModel;
int counter = 1;

tModel.setRowCount(0);
tModel.setColumnCount(0);

tModel.addColumn("index");
for(int i = 1; i <= newColumnCount; i++){
    tModel.addColumn(myMetaData.getColumnName(i));
    currentTableColumns[i - 1] = myMetaData.getColumnName(i);
    currentColumnTypes[i - 1] = myMetaData.getColumnTypeName(i);
}

while(myResultSet.next()){
    String[] newRowData = new String[newColumnCount + 1];

    newRowData[0] = String.valueOf(counter);
    counter++;

    for(int i = 1; i <= newColumnCount; i++)
        if(myResultSet.getObject(i) == null)
            newRowData[i] = "NULL";
        else
            newRowData[i] = myResultSet.getObject(i).toString();

    tModel.addRow(newRowData);
}

currentTableReference = tableData;
}

```



```

catch (SQLException ex) {
    ex.printStackTrace();
}
finally{
    if(myConnection != null)
        try{
            myConnection.close();
        } catch(SQLException ex){}

    if(myPreparedStatement != null)
        try{
            myPreparedStatement.close();
        } catch(SQLException ex){}

    if(myResultSet != null)
        try{
            myResultSet.close();
        } catch(SQLException ex){}
    }
}

```

```

@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">
private void initComponents() {

```

```

    mainPanel = new javax.swing.JPanel();
    jLayeredPane2 = new javax.swing.JLayeredPane();
    actionSelectionText = new javax.swing.JLabel();
    tableSelection = new javax.swing.JComboBox<>();
    actionSelection = new javax.swing.JComboBox<>();
    confirmButton = new javax.swing.JButton();
    normalText1 = new javax.swing.JLabel();
    tableSelectionText = new javax.swing.JLabel();
    actionSelectionErrorMessage = new javax.swing.JLabel();

```

```

tableSelectionErrorMessage = new javax.swing.JLabel();
tableScrollPanel = new javax.swing.JScrollPane();
tableData = new javax.swing.JTable();
returnButton = new javax.swing.JButton();

setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);

actionSelectionText.setText("Select action:");

tableSelection.addPopupMenuListener(new javax.swing.event.PopupMenuListener() {
    public void popupMenuCanceled(javax.swing.event.PopupMenuEvent evt) {
    }
    public void popupMenuWillBecomeInvisible(javax.swing.event.PopupMenuEvent evt) {
        tableSelectionPopupMenuWillBecomeInvisible(evt);
    }
    public void popupMenuWillBecomeVisible(javax.swing.event.PopupMenuEvent evt) {
    }
});

actionSelection.addPopupMenuListener(new javax.swing.event.PopupMenuListener() {
    public void popupMenuCanceled(javax.swing.event.PopupMenuEvent evt) {
    }
    public void popupMenuWillBecomeInvisible(javax.swing.event.PopupMenuEvent evt) {
        actionSelectionPopupMenuWillBecomeInvisible(evt);
    }
    public void popupMenuWillBecomeVisible(javax.swing.event.PopupMenuEvent evt) {
    }
});

confirmButton.setText("Confirm");
confirmButton.addMouseListener(new java.awt.event.MouseAdapter() {
    public void mouseClicked(java.awt.event.MouseEvent evt) {
        confirmButtonMouseClicked(evt);
    }
}

```

```

});

normalText1.setText("Intro");

tableSelectionText.setText("Select table:");

actionSelectionErrorMessage.setText("You must select an action!");

tableSelectionErrorMessage.setText("You must select a table!");

jLayeredPane2.setLayer(actionSelectionText, javax.swing.JLayeredPane.DEFAULT_LAYER);
jLayeredPane2.setLayer(tableSelection, javax.swing.JLayeredPane.DEFAULT_LAYER);
jLayeredPane2.setLayer(actionSelection, javax.swing.JLayeredPane.DEFAULT_LAYER);
jLayeredPane2.setLayer(confirmButton, javax.swing.JLayeredPane.DEFAULT_LAYER);
jLayeredPane2.setLayer(normalText1, javax.swing.JLayeredPane.DEFAULT_LAYER);
jLayeredPane2.setLayer(tableSelectionText, javax.swing.JLayeredPane.DEFAULT_LAYER);
jLayeredPane2.setLayer(actionSelectionErrorMessage, javax.swing.JLayeredPane.DEFAULT_LAYER);
jLayeredPane2.setLayer(tableSelectionErrorMessage, javax.swing.JLayeredPane.DEFAULT_LAYER);

javax.swing.GroupLayout jLayeredPane2Layout = new javax.swing.GroupLayout(jLayeredPane2);
jLayeredPane2.setLayout(jLayeredPane2Layout);
jLayeredPane2Layout.setHorizontalGroup(
    jLayeredPane2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(jLayeredPane2Layout.createSequentialGroup()
            .addGap(10, 10, 10)
            .addGroup(jLayeredPane2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
false)
                .addComponent(actionSelection, 0, 110, Short.MAX_VALUE)
                .addComponent(actionSelectionText, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))
            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
            .addComponent(normalText1)
            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
            .addGroup(jLayeredPane2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
false)

```

```

        .addComponent(tableSelectionText, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)

        .addComponent(tableSelection, 0, 109, Short.MAX_VALUE))

        .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))

        .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
jLayeredPane2Layout.createSequentialGroup())

        .addGap(0, 0, Short.MAX_VALUE)

        .addComponent(confirmButton, javax.swing.GroupLayout.PREFERRED_SIZE, 167,
javax.swing.GroupLayout.PREFERRED_SIZE)

        .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))

        .addGroup(jLayeredPane2Layout.createSequentialGroup())

        .addGap(66, 66, 66)

        .addGroup(jLayeredPane2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
false)

        .addComponent(actionSelectionErrorMessage, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)

        .addComponent(tableSelectionErrorMessage, javax.swing.GroupLayout.PREFERRED_SIZE, 151,
javax.swing.GroupLayout.PREFERRED_SIZE))

        .addContainerGap(71, Short.MAX_VALUE))

);

jLayeredPane2Layout.setVerticalGroup(

jLayeredPane2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

.addGroup(jLayeredPane2Layout.createSequentialGroup())

.addContainerGap()

.addGroup(jLayeredPane2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)

.addComponent(actionSelectionText)

.addComponent(tableSelectionText))

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)

.addGroup(jLayeredPane2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)

.addComponent(actionSelection, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)

.addComponent(normalText1)

.addComponent(tableSelection, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE))

.addGap(18, 18, 18)

.addComponent(actionSelectionErrorMessage)

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)

.addComponent(tableSelectionErrorMessage)

```

```

        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, 29, Short.MAX_VALUE)

        .addComponent(confirmButton, javax.swing.GroupLayout.PREFERRED_SIZE, 41,
javax.swing.GroupLayout.PREFERRED_SIZE)

        .addContainerGap())
    );

```

```

tableData.setModel(new javax.swing.table.DefaultTableModel(
    new Object [][] {

    },
    new String [] {

    }
));

```

```

tableScrollPane.setViewportView(tableData);

```

```

returnButton.setText("Main Menu");
returnButton.addMouseListener(new java.awt.event.MouseAdapter() {
    public void mouseClicked(java.awt.event.MouseEvent evt) {
        returnButtonMouseClicked(evt);
    }
});

```

```

javax.swing.GroupLayout mainPanelLayout = new javax.swing.GroupLayout(mainPanel);
mainPanel.setLayout(mainPanelLayout);
mainPanelLayout.setHorizontalGroup(
    mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(mainPanelLayout.createSequentialGroup()
            .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                .addGap(64, 64, 64)
                .addComponent(jLayeredPane2, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, 49, Short.MAX_VALUE)
                .addComponent(tableScrollPane, javax.swing.GroupLayout.PREFERRED_SIZE, 654,
javax.swing.GroupLayout.PREFERRED_SIZE)
                .addGap(59, 59, 59))
            .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

```

```

        .addGap(501, 501, 501)

        .addComponent(returnButton, javax.swing.GroupLayout.PREFERRED_SIZE, 100,
javax.swing.GroupLayout.PREFERRED_SIZE)

        .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))
    );

    mainPanelLayout.setVerticalGroup(
        mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(javax.swing.GroupLayout.Alignment.TRAILING, mainPanelLayout.createSequentialGroup())
        .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(mainPanelLayout.createSequentialGroup())
                .addContainerGap(120, Short.MAX_VALUE)
                .addComponent(tableScrollPanel, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, 81,
Short.MAX_VALUE))
            .addGroup(mainPanelLayout.createSequentialGroup())
                .addGap(207, 207, 207)
                .addComponent(jLayeredPane2, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)))
        .addComponent(returnButton)
        .addGap(32, 32, 32))
    );

    javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
    getContentPane().setLayout(layout);

    layout.setHorizontalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addComponent(mainPanel, javax.swing.GroupLayout.Alignment.TRAILING,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
    );

    layout.setVerticalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addComponent(mainPanel, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
    );

```

```
pack();  
} // </editor-fold>
```

```
private void actionSelectionPopupMenuWillBecomeInvisible(javax.swing.event.PopupMenuEvent evt) {  
    if(!actionSelection.getSelectedItemAt().equals("Action"))  
        actionSelection.removeItem("Action");  
}
```

```
private void tableSelectionPopupMenuWillBecomeInvisible(javax.swing.event.PopupMenuEvent evt) {  
    if(!tableSelection.getSelectedItemAt().equals("Table")){  
        tableSelection.removeItem("Table");  
  
        tableScrollPanel.setVisible(true);  
  
        currentSelectedTable = tableSelection.getSelectedItemAt().toString();  
        fillTable();  
    }  
}
```

```
private void returnButtonMouseClicked(java.awt.event.MouseEvent evt) {  
    MainMenu mainMen = new MainMenu();  
    dispose();  
}
```

```
private void confirmButtonMouseClicked(java.awt.event.MouseEvent evt) {  
    actionSelectionErrorMessage.setVisible(false);  
    tableSelectionErrorMessage.setVisible(false);  
  
    if(actionSelection.getSelectedItemAt().equals("Action") ||  
tableSelection.getSelectedItemAt().equals("Table")){  
        if(actionSelection.getSelectedItemAt().equals("Action"))  
            actionSelectionErrorMessage.setVisible(true);  
  
        if(tableSelection.getSelectedItemAt().equals("Table"))
```

```

        tableSelectionErrorMessage.setVisible(true);
    }
    else{
        if(actionSelection.getSelectedItem().equals("Insert") && activeSubWindow == false){
            InsertPage inPage = new InsertPage();
            activeSubWindow = true;
        }
        if(actionSelection.getSelectedItem().equals("Update") && activeSubWindow == false){
            UpdatePage upPage = new UpdatePage();
            activeSubWindow = true;
        }
        if(actionSelection.getSelectedItem().equals("Delete") && activeSubWindow == false){
            DeletePage delPage = new DeletePage();
            activeSubWindow = true;
        }
    }
}

```

```

public static void main(String args[]) {
    /* Set the Nimbus look and feel */
    //<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
    /* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
    * For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
    */
    try {
        for (javax.swing.UIManager.LookAndFeelInfo info : javax.swing.UIManager.getInstalledLookAndFeels()) {
            if ("Nimbus".equals(info.getName())) {
                javax.swing.UIManager.setLookAndFeel(info.getClassName());
                break;
            }
        }
    } catch (ClassNotFoundException ex) {
        java.util.logging.Logger.getLogger(EditTableMenu.class.getName()).log(java.util.logging.Level.SEVERE,
        null, ex);
    } catch (InstantiationException ex) {

```



```

        java.util.logging.Logger.getLogger(EditTableMenu.class.getName()).log(java.util.logging.Level.SEVERE,
null, ex);
    } catch (IllegalAccessException ex) {
        java.util.logging.Logger.getLogger(EditTableMenu.class.getName()).log(java.util.logging.Level.SEVERE,
null, ex);
    } catch (javax.swing.UnsupportedLookAndFeelException ex) {
        java.util.logging.Logger.getLogger(EditTableMenu.class.getName()).log(java.util.logging.Level.SEVERE,
null, ex);
    }
}
//</editor-fold>

```

```

/* Create and display the form */
java.awt.EventQueue.invokeLater(new Runnable() {
    public void run() {
        new EditTableMenu().setVisible(true);
    }
});
}

```

```

// Variables declaration - do not modify
private javax.swing.JComboBox<String> actionSelection;
private javax.swing.JLabel actionSelectionErrorMessage;
private javax.swing.JLabel actionSelectionText;
private javax.swing.JButton confirmButton;
private javax.swing.JLayeredPane jLayeredPane2;
private javax.swing.JPanel mainPanel;
private javax.swing.JLabel normalText1;
private javax.swing.JButton returnButton;
private javax.swing.JTable tableData;
private javax.swing.JScrollPane tableScrollPanel;
private javax.swing.JComboBox<String> tableSelection;
private javax.swing.JLabel tableSelectionErrorMessage;
private javax.swing.JLabel tableSelectionText;
// End of variables declaration
}

```

InsertPage.java :

```
import java.awt.Color;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.ResultSetMetaData;
import java.sql.SQLException;
import java.util.logging.Level;
import java.util.logging.Logger;
import javax.swing.SwingConstants;
import javax.swing.table.DefaultTableModel;

public class InsertPage extends javax.swing.JFrame {

    private static final String DB_URL = "jdbc:mysql://localhost:3306/travel_agency";
    private static final String USERNAME = "root";
    private static final String PASSWORD = "2912002";
    private static Connection myConnection;
    private static PreparedStatement myPreparedStatement;
    private static ResultSet myResultSet;
    private static ResultSetMetaData myMetaData;

    public InsertPage() {
        initComponents();
        setVisible(true);

        setLocationRelativeTo(null);
        setResizable(false);

        mainPanel.setBackground(new Color(666666));

        menuNameLabel.setForeground(Color.WHITE);
        menuNameLabel.setHorizontalAlignment(SwingConstants.CENTER);
```

```

        errorMessage.setForeground(Color.RED);
        errorMessage.setHorizontalAlignment(SwingConstants.CENTER);
        errorMessage.setVisible(false);

        initializeThisPage();
    }

    private void initializeThisPage(){
        menuNameLabel.setText(menuNameLabel.getText() + EditTableMenu.currentSelectedTable);

        DefaultTableModel tModel = (DefaultTableModel) newDataTable.getModel();

        for(String columnName : EditTableMenu.currentTableColumns)
            tModel.addColumn(columnName);

        tModel.addRow(new String[tModel.getColumnCount()]);
    }

    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {

        mainPanel = new javax.swing.JPanel();
        menuNameLabel = new javax.swing.JLabel();
        closeButton = new javax.swing.JButton();
        jScrollPane1 = new javax.swing.JScrollPane();
        newDataTable = new javax.swing.JTable();
        insertButton = new javax.swing.JButton();
        errorMessage = new javax.swing.JLabel();

        setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);

        menuNameLabel.setFont(new java.awt.Font("Segoe UI", 1, 24)); // NOI18N

```

```
menuNameLabel.setText("Insert into ");
```

```
closeButton.setText("Close");
```

```
closeButton.addMouseListener(new java.awt.event.MouseAdapter() {  
    public void mouseClicked(java.awt.event.MouseEvent evt) {  
        closeButtonMouseClicked(evt);  
    }  
});
```

```
newDataTable.setRowHeight(30);
```

```
jScrollPane1.setViewportView(newDataTable);
```

```
insertButton.setText("Insert");
```

```
insertButton.addMouseListener(new java.awt.event.MouseAdapter() {  
    public void mouseClicked(java.awt.event.MouseEvent evt) {  
        insertButtonMouseClicked(evt);  
    }  
});
```

```
errorMessage.setText("Something went wrong! Try again.");
```

```
javax.swing.GroupLayout mainPanelLayout = new javax.swing.GroupLayout(mainPanel);
```

```
mainPanel.setLayout(mainPanelLayout);
```

```
mainPanelLayout.setHorizontalGroup(  
    mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)  
        .addGroup(mainPanelLayout.createSequentialGroup()  
            .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)  
                .addGroup(mainPanelLayout.createSequentialGroup()  
                    .addGroup(mainPanelLayout.createSequentialGroup()  
                        .addGap(275, 275, 275)  
                        .addComponent(insertButton, javax.swing.GroupLayout.PREFERRED_SIZE, 230,  
javax.swing.GroupLayout.PREFERRED_SIZE))  
                    .addGroup(mainPanelLayout.createSequentialGroup()  
                        .addGap(17, 17, 17)  
                        .addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE, 751,  
javax.swing.GroupLayout.PREFERRED_SIZE))
```

```
                .addGroup(mainPanelLayout.createSequentialGroup()  
                    .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)  
                        .addGroup(mainPanelLayout.createSequentialGroup()  
                            .addGroup(mainPanelLayout.createSequentialGroup()  
                                .addGap(275, 275, 275)  
                                .addComponent(insertButton, javax.swing.GroupLayout.PREFERRED_SIZE, 230,  
javax.swing.GroupLayout.PREFERRED_SIZE))  
                            .addGroup(mainPanelLayout.createSequentialGroup()  
                                .addGap(17, 17, 17)  
                                .addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE, 751,  
javax.swing.GroupLayout.PREFERRED_SIZE))
```

```

        .addGroup(mainPanelLayout.createSequentialGroup())
        .addGap(350, 350, 350)
        .addComponent(closeButton)))
    .addContainerGap(17, Short.MAX_VALUE))
    .addGroup(javax.swing.GroupLayout.Alignment.TRAILING, mainPanelLayout.createSequentialGroup())
    .addGap(0, 0, Short.MAX_VALUE)
    .addComponent(menuNameLabel, javax.swing.GroupLayout.PREFERRED_SIZE, 288,
javax.swing.GroupLayout.PREFERRED_SIZE)
    .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))
    .addGroup(mainPanelLayout.createSequentialGroup())
    .addGap(265, 265, 265)
    .addComponent(errorMessage, javax.swing.GroupLayout.PREFERRED_SIZE, 249,
javax.swing.GroupLayout.PREFERRED_SIZE)
    .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))
);
mainPanelLayout.setVerticalGroup(
    mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(mainPanelLayout.createSequentialGroup())
    .addContainerGap()
    .addComponent(menuNameLabel, javax.swing.GroupLayout.PREFERRED_SIZE, 38,
javax.swing.GroupLayout.PREFERRED_SIZE)
    .addGap(26, 26, 26)
    .addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE, 44,
javax.swing.GroupLayout.PREFERRED_SIZE)
    .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, 9, Short.MAX_VALUE)
    .addComponent(errorMessage)
    .addGap(18, 18, 18)
    .addComponent(insertButton, javax.swing.GroupLayout.PREFERRED_SIZE, 70,
javax.swing.GroupLayout.PREFERRED_SIZE)
    .addGap(40, 40, 40)
    .addComponent(closeButton)
    .addGap(14, 14, 14))
);

javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
getContentPane().setLayout(layout);

```

```

        layout.setHorizontalGroup(
            layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                .addComponent(mainPanel, javax.swing.GroupLayout.Alignment.TRAILING,
                    javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
            );
        layout.setVerticalGroup(
            layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                .addComponent(mainPanel, javax.swing.GroupLayout.DEFAULT_SIZE,
                    javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
            );

        pack();
    } // </editor-fold>

```

```

private void closeButtonMouseClicked(java.awt.event.MouseEvent evt) {
    EditTableMenu.activeSubWindow = false;
    dispose();
}

```

```

private void insertButtonMouseClicked(java.awt.event.MouseEvent evt) {
    errorMessage.setVisible(false);

```

```

    DefaultTableModel tModel = (DefaultTableModel) newDataTable.getModel();

```

```

    String[] insertData = new String[tModel.getColumnCount()];

```

```

    for(int i = 0; i < tModel.getColumnCount(); i++)

```

```

        if(tModel.getValueAt(0, i) == null)

```

```

            insertData[i] = "";

```

```

        else

```

```

            insertData[i] = tModel.getValueAt(0, i).toString();

```

```

    try{

```

```

        String INSERT_QUERY = "INSERT INTO " + EditTableMenu.currentSelectedTable + " VALUES (";

```

```

for(int i = 0; i < tModel.getColumnCount(); i++){
    if(EditTableMenu.currentColumnTypes[i].equals("INT") ||
        EditTableMenu.currentColumnTypes[i].equals("TINYINT") ||
        EditTableMenu.currentColumnTypes[i].equals("FLOAT") ||
        insertData[i].equals("NULL"))
        INSERT_QUERY = INSERT_QUERY + insertData[i];
    else
        INSERT_QUERY = INSERT_QUERY + "\"" + insertData[i] + "\"";

    if(i == tModel.getColumnCount() - 1)
        INSERT_QUERY = INSERT_QUERY + ";";
    else
        INSERT_QUERY = INSERT_QUERY + ",";
}

myConnection = DriverManager.getConnection(DB_URL, USERNAME, PASSWORD);
myPreparedStatement = myConnection.prepareStatement(INSERT_QUERY);
myPreparedStatement.executeUpdate();

String insertDataForModel[] = new String[insertData.length + 1];

insertDataForModel[0] = Integer.toString(EditTableMenu.currentTableReference.getRowCount() + 1);

for(int i = 0; i < insertData.length; i++)
    insertDataForModel[i + 1] = insertData[i];

EditTableMenu.currentTableModel.addRow(insertDataForModel);
}
catch (SQLException ex) {
    errorMessage.setVisible(true);
}
finally{

}

```

```
}
```

```
public static void main(String args[]) {
```

```
    java.awt.EventQueue.invokeLater(new Runnable() {
```

```
        public void run() {
```

```
            new InsertPage().setVisible(true);
```

```
        }
```

```
    });
```

```
}
```

```
// Variables declaration - do not modify
```

```
private javax.swing.JButton closeButton;
```

```
private javax.swing.JLabel errorMessage;
```

```
private javax.swing.JButton insertButton;
```

```
private javax.swing.JScrollPane jScrollPane1;
```

```
private javax.swing.JPanel mainPanel;
```

```
private javax.swing.JLabel menuNameLabel;
```

```
private javax.swing.JTable newDataTable;
```

```
// End of variables declaration
```

```
}
```


UpdatePage.java :

```
import java.sql.*;
import java.awt.Color;
import java.util.logging.Level;
import java.util.logging.Logger;
import javax.swing.SwingConstants;
import javax.swing.table.DefaultTableModel;

public class UpdatePage extends javax.swing.JFrame {

    private static final String DB_URL = "jdbc:mysql://localhost:3306/travel_agency";
    private static final String USERNAME = "root";
    private static final String PASSWORD = "2912002";
    private static Connection myConnection;
    private static PreparedStatement myPreparedStatement;
    private static ResultSet myResultSet;
    private static ResultSetMetaData myMetaData;

    String[] selectedRowData;

    public UpdatePage() {
        initComponents();
        setVisible(true);

        setLocationRelativeTo(null);
        setResizable(false);

        mainPanel.setBackground(new Color(666666));

        tableNameLabel.setForeground(Color.WHITE);
        tableNameLabel.setHorizontalAlignment(SwingConstants.CENTER);
        targetSelectionText.setForeground(Color.WHITE);
        updateErrorMessage.setForeground(Color.RED);
        updateErrorMessage.setHorizontalAlignment(SwingConstants.CENTER);
```

```

updateErrorMessage.setVisible(false);

initializeThisMenu();
}

private void initializeThisMenu(){
    tableNameLabel.setText(tableNameLabel.getText() + EditTableMenu.currentSelectedTable);

    DefaultTableModel tModel = (DefaultTableModel) selectedRowTable.getModel();

    int rows = EditTableMenu.currentTableModel.getRowCount();

    for(int i = 0; i < rows; i++)
        updateSelection.addItem(Integer.toString(i + 1));

    for(int i = 0; i < EditTableMenu.currentTableColumns.length; i++)
        tModel.addColumn(EditTableMenu.currentTableColumns[i]);

    fillTable();
}

private void fillTable(){
    DefaultTableModel tModel = (DefaultTableModel) selectedRowTable.getModel();

    int selectedRow = Integer.valueOf(updateSelection.getSelectedItemAt().toString());
    int columnCount = EditTableMenu.currentTableModel.getColumnCount() - 1;
    selectedRowData = new String[columnCount];

    tModel.setRowCount(0);

    for(int i = 0; i < columnCount; i++){
        selectedRowData[i] = EditTableMenu.currentTableModel.getValueAt(selectedRow - 1, i + 1).toString();
    }
}

```

```

        tModel.addRow(selectedRowData);
    }

    private String prepareUpdateQuery(String[] newRowData){

        String tempQuery = "UPDATE " + EditTableMenu.currentSelectedTable + " SET " +
        EditTableMenu.currentTableColumns[0] + " = ";

        if(EditTableMenu.currentColumnTypes[0].equals("INT") ||
            EditTableMenu.currentColumnTypes[0].equals("TINYINT") ||
            EditTableMenu.currentColumnTypes[0].equals("FLOAT") ||
            newRowData[0].equals("NULL"))
        {
            tempQuery = tempQuery.concat(newRowData[0]);
        }
        else{
            tempQuery = tempQuery.concat("'" + newRowData[0] + "'");
        }

        for(int i = 1; i < newRowData.length; i++){
            tempQuery = tempQuery.concat(", " + EditTableMenu.currentTableColumns[i] + " = ");
            if(EditTableMenu.currentColumnTypes[i].equals("INT") ||
                EditTableMenu.currentColumnTypes[i].equals("TINYINT") ||
                EditTableMenu.currentColumnTypes[i].equals("FLOAT") ||
                newRowData[i].equals("NULL"))
            {
                tempQuery = tempQuery.concat(newRowData[i]);
            }
            else{
                tempQuery = tempQuery.concat("'" + newRowData[i] + "'");
            }
        }

        tempQuery = tempQuery.concat(" WHERE " + EditTableMenu.currentTableColumns[0]);

        if(selectedRowData[0].equals("NULL"))

```

```

        tempQuery = tempQuery + " IS ";
else
    tempQuery = tempQuery + " = ";

if(EditTableMenu.currentColumnTypes[0].equals("INT") ||
    EditTableMenu.currentColumnTypes[0].equals("TINYINT") ||
    EditTableMenu.currentColumnTypes[0].equals("FLOAT") ||
    selectedRowData[0].equals("NULL"))
{
    tempQuery = tempQuery.concat(selectedRowData[0]);
}
else{
    tempQuery = tempQuery.concat("'" + selectedRowData[0] + "'");
}

for(int i = 1; i < selectedRowData.length; i++){
    tempQuery = tempQuery.concat(" AND " + EditTableMenu.currentTableColumns[i]);

    if(selectedRowData[i].equals("NULL"))
        tempQuery = tempQuery + " IS ";
    else
        tempQuery = tempQuery + " = ";

    if(EditTableMenu.currentColumnTypes[i].equals("INT") ||
        EditTableMenu.currentColumnTypes[i].equals("TINYINT") ||
        EditTableMenu.currentColumnTypes[i].equals("FLOAT") ||
        selectedRowData[i].equals("NULL"))
    {
        tempQuery = tempQuery.concat(selectedRowData[i]);
    }
    else{
        tempQuery = tempQuery.concat("'" + selectedRowData[i] + "'");
    }
}

```

```
    return tempQuery;
}
```

```
@SuppressWarnings("unchecked")
```

```
// <editor-fold defaultstate="collapsed" desc="Generated Code">
```

```
private void initComponents() {
```

```
    mainPanel = new javax.swing.JPanel();
    tableNameLabel = new javax.swing.JLabel();
    closeButton = new javax.swing.JButton();
    updateSelection = new javax.swing.JComboBox<>();
    jScrollPane1 = new javax.swing.JScrollPane();
    selectedRowTable = new javax.swing.JTable();
    targetSelectionText = new javax.swing.JLabel();
    updateButton = new javax.swing.JButton();
    updateErrorMessage = new javax.swing.JLabel();
```

```
    setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
```

```
    tableNameLabel.setFont(new java.awt.Font("Segoe UI", 1, 24)); // NOI18N
```

```
    tableNameLabel.setText("Update ");
```

```
    closeButton.setText("Close");
```

```
    closeButton.addMouseListener(new java.awt.event.MouseAdapter() {
```

```
        public void mouseClicked(java.awt.event.MouseEvent evt) {
```

```
            closeButtonMouseClicked(evt);
```

```
        }
```

```
    });
```

```
    updateSelection.addPopupMenuListener(new javax.swing.event.PopupMenuListener() {
```

```
        public void popupMenuCanceled(javax.swing.event.PopupMenuEvent evt) {
```

```
        }
```

```
        public void popupMenuWillBecomeInvisible(javax.swing.event.PopupMenuEvent evt) {
```

```

        updateSelectionPopupMenuWillBecomeInvisible(evt);
    }

    public void popupMenuWillBecomeVisible(javax.swing.event.PopupMenuEvent evt) {
    }

    });

    selectedRowTable.setModel(new javax.swing.table.DefaultTableModel(
        new Object [][] {

        },
        new String [] {

        }
    ));
    jScrollPane1.setViewportView(selectedRowTable);

    targetSelectionText.setText("Select to update");

    updateButton.setText("Update");
    updateButton.addMouseListener(new java.awt.event.MouseAdapter() {
        public void mouseClicked(java.awt.event.MouseEvent evt) {
            updateButtonMouseClicked(evt);
        }
    });

    updateErrorMessage.setText("Something went wrong!");

    javax.swing.GroupLayout mainPanelLayout = new javax.swing.GroupLayout(mainPanel);
    mainPanel.setLayout(mainPanelLayout);
    mainPanelLayout.setHorizontalGroup(
        mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(mainPanelLayout.createSequentialGroup()
                .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                    .addGap(28, 28, 28)
                    .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
false)

```

```

        .addComponent(updateSelection, 0, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)

        .addComponent(targetSelectionText, javax.swing.GroupLayout.DEFAULT_SIZE, 94,
Short.MAX_VALUE))

        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, 49, Short.MAX_VALUE)

        .addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE, 552,
javax.swing.GroupLayout.PREFERRED_SIZE)

        .addGap(16, 16, 16))

    .addGroup(mainPanelLayout.createSequentialGroup())

    .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

        .addGroup(mainPanelLayout.createSequentialGroup())

            .addGap(61, 61, 61)

            .addComponent(tableNameLabel, javax.swing.GroupLayout.PREFERRED_SIZE, 489,
javax.swing.GroupLayout.PREFERRED_SIZE))

        .addGroup(mainPanelLayout.createSequentialGroup())

            .addGap(265, 265, 265)

            .addComponent(updateErrorMessage, javax.swing.GroupLayout.DEFAULT_SIZE, 445,
Short.MAX_VALUE)))

    .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))

    .addGroup(mainPanelLayout.createSequentialGroup())

    .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

        .addGroup(mainPanelLayout.createSequentialGroup())

            .addGap(255, 255, 255)

            .addComponent(closeButton))

        .addGroup(mainPanelLayout.createSequentialGroup())

            .addGap(239, 239, 239)

            .addComponent(updateButton, javax.swing.GroupLayout.PREFERRED_SIZE, 220,
javax.swing.GroupLayout.PREFERRED_SIZE)))

    .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))

);

mainPanelLayout.setVerticalGroup(

    mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

        .addGroup(mainPanelLayout.createSequentialGroup())

            .addGap(26, 26, 26)

            .addComponent(tableNameLabel, javax.swing.GroupLayout.PREFERRED_SIZE, 40,
javax.swing.GroupLayout.PREFERRED_SIZE)

            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, 45, Short.MAX_VALUE)

```

```

        .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
false)

        .addGroup(mainPanelLayout.createSequentialGroup()

            .addComponent(targetSelectionText)

            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)

            .addComponent(updateSelection, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE))

            .addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE, 0, Short.MAX_VALUE))

        .addGap(18, 18, 18)

        .addComponent(updateErrorMessage)

        .addGap(18, 18, 18)

        .addComponent(updateButton, javax.swing.GroupLayout.PREFERRED_SIZE, 64,
javax.swing.GroupLayout.PREFERRED_SIZE)

        .addGap(29, 29, 29)

        .addComponent(closeButton)

        .addGap(21, 21, 21))
    );

```

```

    javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
    getContentPane().setLayout(layout);

    layout.setHorizontalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addComponent(mainPanel, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
    );

    layout.setVerticalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addComponent(mainPanel, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
    );

```

```

    pack();
} // </editor-fold>

```

```

private void closeButtonMouseClicked(java.awt.event.MouseEvent evt) {
    EditTableMenu.activeSubWindow = false;
}

```



```

        dispose();
    }

    private void updateSelectionPopupMenuWillBecomeInvisible(javax.swing.event.PopupMenuEvent evt) {
        fillTable();
    }

    private void updateButtonMouseClicked(java.awt.event.MouseEvent evt) {
        updateErrorMessage.setVisible(false);

        DefaultTableModel tModel = (DefaultTableModel) selectedRowTable.getModel();
        String[] tempRow = new String[selectedRowData.length];

        for(int i = 0; i < selectedRowData.length; i++)
            tempRow[i] = tModel.getValueAt(0, i).toString();

        String UPDATE_TABLE_QUERY = prepareUpdateQuery(tempRow);

        try {
            myConnection = DriverManager.getConnection(DB_URL, USERNAME, PASSWORD);
            myPreparedStatement = myConnection.prepareStatement(UPDATE_TABLE_QUERY);

            myPreparedStatement.executeUpdate();

            for(int i = 0; i < tempRow.length; i++)
                EditTableMenu.currentTableReference.setValueAt(tempRow[i],
                    Integer.valueOf(updateSelection.getSelectedItemAt().toString()) - 1, i + 1);
        }
        catch (SQLException ex) {
            updateErrorMessage.setVisible(true);
        }
        finally{
            if(myConnection != null)
                try{
                    myConnection.close();
                }
        }
    }

```

```

        } catch(SQLException ex){}

        if(myPreparedStatement != null)
            try{
                myPreparedStatement.close();
            } catch(SQLException ex){}

        if(myResultSet != null)
            try{
                myResultSet.close();
            } catch(SQLException ex){}
    }
}

public static void main(String args[]) {
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new UpdatePage().setVisible(true);
        }
    });
}

// Variables declaration - do not modify
private javax.swing.JButton closeButton;
private javax.swing.JScrollPane jScrollPane1;
private javax.swing.JPanel mainPanel;
private javax.swing.JTable selectedRowTable;
private javax.swing.JLabel tableNameLabel;
private javax.swing.JLabel targetSelectionText;
private javax.swing.JButton updateButton;
private javax.swing.JLabel updateErrorMessage;
private javax.swing.JComboBox<String> updateSelection;

// End of variables declaration
}

```

DeletePage.java :

```
import java.awt.Color;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.ResultSetMetaData;
import java.sql.SQLException;
import java.util.logging.Level;
import java.util.logging.Logger;
import javax.swing.JTable;
import javax.swing.SwingConstants;

public class DeletePage extends javax.swing.JFrame {

    private static final String DB_URL = "jdbc:mysql://localhost:3306/travel_agency";
    private static final String USERNAME = "root";
    private static final String PASSWORD = "2912002";
    private static Connection myConnection;
    private static PreparedStatement myPreparedStatement;
    private static ResultSet myResultSet;
    private static ResultSetMetaData myMetaData;

    String GET_TABLE_QUERY = "SELECT * FROM ";

    public DeletePage() {
        initComponents();
        setVisible(true);

        setLocationRelativeTo(null);
        setResizable(false);

        mainPanel.setBackground(new Color(666666));
    }
}
```

```

        tableNameLabel.setForeground(Color.WHITE);
        tableNameLabel.setHorizontalAlignment(SwingConstants.CENTER);
        tableNameLabel2.setForeground(Color.WHITE);
        tableNameLabel2.setHorizontalAlignment(SwingConstants.CENTER);
        targetSelectionText.setForeground(Color.WHITE);

        initializeThisMenu();
    }

    private void initializeThisMenu(){
        String selectedTable = EditTableMenu.currentSelectedTable;

        try {
            myConnection = DriverManager.getConnection(DB_URL, USERNAME, PASSWORD);
            myPreparedStatement = myConnection.prepareStatement(GET_TABLE_QUERY.concat(selectedTable));
            myResultSet = myPreparedStatement.executeQuery();

            tableNameLabel2.setText(selectedTable);

            int counter = 1;

            while(myResultSet.next()){
                deleteSelection.addItem(String.valueOf(counter));
                counter++;
            }

        }
        catch (SQLException ex) {
            ex.printStackTrace();
        }
        finally{
            if(myConnection != null)
                try{

```

```

        myConnection.close();
    } catch(SQLException ex){}

    if(myPreparedStatement != null)
    try{
        myPreparedStatement.close();
    } catch(SQLException ex){}

    if(myResultSet != null)
    try{
        myResultSet.close();
    } catch(SQLException ex){}
}
}

private String prepareDeleteQuery(String[] selectedRow, int columns){
    String tempQuery = "DELETE FROM " + EditTableMenu.currentSelectedTable + " WHERE " +
EditTableMenu.currentTableColumns[0];

    if(selectedRow[0].equals("NULL"))
        tempQuery = tempQuery + " IS ";
    else
        tempQuery = tempQuery + " = ";

    if(EditTableMenu.currentColumnTypes[0].equals("INT") ||
        EditTableMenu.currentColumnTypes[0].equals("TINYINT") ||
        EditTableMenu.currentColumnTypes[0].equals("FLOAT") ||
        selectedRow[0].equals("NULL"))
    {
        tempQuery = tempQuery.concat(selectedRow[0]);
    }
    else{
        tempQuery = tempQuery.concat("'" + selectedRow[0] + "'");
    }
}

```

```

for(int i = 1; i < columns; i++){
    tempQuery = tempQuery.concat(" AND " + EditTableMenu.currentTableColumns[i]);

    if(selectedRow[i].equals("NULL"))
        tempQuery = tempQuery + " IS ";
    else
        tempQuery = tempQuery + " = ";

    if(EditTableMenu.currentColumnTypes[i].equals("INT") ||
        EditTableMenu.currentColumnTypes[i].equals("TINYINT") ||
        EditTableMenu.currentColumnTypes[i].equals("FLOAT") ||
        selectedRow[i].equals("NULL"))
    {
        tempQuery = tempQuery.concat(selectedRow[i]);
    }
    else{
        tempQuery = tempQuery.concat "\"" + selectedRow[i] + "\"";
    }
}

return tempQuery;
}

```

```

@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">
private void initComponents() {

    mainPanel = new javax.swing.JPanel();
    closeButton = new javax.swing.JButton();
    targetSelectionText = new javax.swing.JLabel();
    deleteSelection = new javax.swing.JComboBox<>();
    deleteButton = new javax.swing.JButton();
    tableNameLabel = new javax.swing.JLabel();
    tableNameLabel2 = new javax.swing.JLabel();

```

```
setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
```

```
closeButton.setText("Close");
```

```
closeButton.addMouseListener(new java.awt.event.MouseAdapter() {  
    public void mouseClicked(java.awt.event.MouseEvent evt) {  
        closeButtonMouseClicked(evt);  
    }  
});
```

```
targetSelectionText.setText("Select to delete");
```

```
deleteButton.setText("Delete");
```

```
deleteButton.addMouseListener(new java.awt.event.MouseAdapter() {  
    public void mouseClicked(java.awt.event.MouseEvent evt) {  
        deleteButtonMouseClicked(evt);  
    }  
});
```

```
tableNameLabel.setFont(new java.awt.Font("Segoe UI", 1, 24)); // NOI18N
```

```
tableNameLabel.setText("Delete from");
```

```
tableNameLabel2.setFont(new java.awt.Font("Segoe UI", 1, 24)); // NOI18N
```

```
tableNameLabel2.setText("Placeholder Text");
```

```
javax.swing.GroupLayout mainPanelLayout = new javax.swing.GroupLayout(mainPanel);
```

```
mainPanel.setLayout(mainPanelLayout);
```

```
mainPanelLayout.setHorizontalGroup(  
    mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
```

```
        .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
```

```
            .addGroup(mainPanelLayout.createSequentialGroup()  
                .addGap(134, 134, 134)
```

```
                .addComponent(closeButton)
```

```
            .addGroup(mainPanelLayout.createSequentialGroup()  
                .addGap(134, 134, 134)
```

```
                .addComponent(deleteButton)
```

```
            .addGroup(mainPanelLayout.createSequentialGroup()  
                .addGap(134, 134, 134)
```

```

        .addGroup(mainPanelLayout.createSequentialGroup())

        .addGap(26, 26, 26)

        .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

            .addComponent(tableNameLabel2, javax.swing.GroupLayout.PREFERRED_SIZE, 287,
javax.swing.GroupLayout.PREFERRED_SIZE)

            .addComponent(tableNameLabel, javax.swing.GroupLayout.PREFERRED_SIZE, 287,
javax.swing.GroupLayout.PREFERRED_SIZE))))

        .addContainerGap(19, Short.MAX_VALUE))

    .addGroup(javax.swing.GroupLayout.Alignment.TRAILING, mainPanelLayout.createSequentialGroup())

    .addGap(34, 34, 34)

    .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
false)

        .addComponent(targetSelectionText, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)

        .addComponent(deleteSelection, javax.swing.GroupLayout.PREFERRED_SIZE, 87,
javax.swing.GroupLayout.PREFERRED_SIZE))

    .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)

    .addComponent(deleteButton, javax.swing.GroupLayout.PREFERRED_SIZE, 95,
javax.swing.GroupLayout.PREFERRED_SIZE)

    .addGap(41, 41, 41))
);

mainPanelLayout.setVerticalGroup(

    mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

    .addGroup(mainPanelLayout.createSequentialGroup())

    .addGap(22, 22, 22)

    .addComponent(tableNameLabel, javax.swing.GroupLayout.PREFERRED_SIZE, 34,
javax.swing.GroupLayout.PREFERRED_SIZE)

    .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)

    .addComponent(tableNameLabel2, javax.swing.GroupLayout.PREFERRED_SIZE, 34,
javax.swing.GroupLayout.PREFERRED_SIZE)

    .addGap(18, 18, 18)

    .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

        .addComponent(deleteButton, javax.swing.GroupLayout.PREFERRED_SIZE, 61,
javax.swing.GroupLayout.PREFERRED_SIZE)

        .addGroup(mainPanelLayout.createSequentialGroup())

        .addComponent(targetSelectionText)

        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)

```



```

        .addComponent(deleteSelection, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)))

        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, 104, Short.MAX_VALUE)

        .addComponent(closeButton)

        .addContainerGap())

);

```

```

javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
getContentPane().setLayout(layout);

layout.setHorizontalGroup(

    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

        .addComponent(mainPanel, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)

);

layout.setVerticalGroup(

    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

        .addComponent(mainPanel, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)

);

```

```

pack();
} // </editor-fold>

```

```

private void closeButtonMouseClicked(java.awt.event.MouseEvent evt) {

    EditTableMenu.activeSubWindow = false;

    dispose();

}

```

```

private void deleteButtonMouseClicked(java.awt.event.MouseEvent evt) {

    JTable myTable = EditTableMenu.currentTableReference;

    int columnCount = myTable.getColumnCount() - 1;

    String[] rowData = new String[columnCount];

    for(int i = 0; i < columnCount; i++){

        rowData[i] = myTable.getValueAt(deleteSelection.getSelectedIndex(), i + 1).toString();

    }
}

```

```
String DELETE_ROW_QUERY = prepareDeleteQuery(rowData, columnCount);
```

```
try {
```

```
    myConnection = DriverManager.getConnection(DB_URL, USERNAME, PASSWORD);
```

```
    myPreparedStatement = myConnection.prepareStatement(DELETE_ROW_QUERY);
```

```
    myPreparedStatement.executeUpdate();
```

```
    EditTableMenu.currentTableModel.removeRow(deleteSelection.getSelectedIndex());
```

```
    deleteSelection.removeItem(deleteSelection.getSelectedItem().toString());
```

```
}
```

```
catch (SQLException ex) {
```

```
    ex.printStackTrace();
```

```
}
```

```
finally{
```

```
    if(myConnection != null)
```

```
    {
```

```
        try{
```

```
            myConnection.close();
```

```
        } catch(SQLException ex){}
```

```
    if(myPreparedStatement != null)
```

```
    {
```

```
        try{
```

```
            myPreparedStatement.close();
```

```
        } catch(SQLException ex){}
```

```
    if(myResultSet != null)
```

```
    {
```

```
        try{
```

```
            myResultSet.close();
```

```
        } catch(SQLException ex){}
```

```
}
```

```
}
```

```
public static void main(String args[]) {
```

```
    /* Set the Nimbus look and feel */
```

```

//<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
/* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
 * For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
 */
try {
    for (javax.swing.UIManager.LookAndFeelInfo info : javax.swing.UIManager.getInstalledLookAndFeels()) {
        if ("Nimbus".equals(info.getName())) {
            javax.swing.UIManager.setLookAndFeel(info.getClassName());
            break;
        }
    }
} catch (ClassNotFoundException ex) {
    java.util.logging.Logger.getLogger(DeletePage.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
} catch (InstantiationException ex) {
    java.util.logging.Logger.getLogger(DeletePage.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
} catch (IllegalAccessException ex) {
    java.util.logging.Logger.getLogger(DeletePage.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
} catch (javax.swing.UnsupportedLookAndFeelException ex) {
    java.util.logging.Logger.getLogger(DeletePage.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
}
//</editor-fold>

```

```

/* Create and display the form */
java.awt.EventQueue.invokeLater(new Runnable() {
    public void run() {
        new DeletePage().setVisible(true);
    }
});
}

```

```

// Variables declaration - do not modify
private javax.swing.JButton closeButton;

```

```
private javax.swing.JButton deleteButton;
private javax.swing.JComboBox<String> deleteSelection;
private javax.swing.JPanel mainPanel;
private javax.swing.JLabel tableNameLabel;
private javax.swing.JLabel tableNameLabel2;
private javax.swing.JLabel targetSelectionText;
// End of variables declaration
}
```

LogPage.java :

```
import java.awt.Color;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.ResultSetMetaData;
import javax.swing.SwingConstants;
import javax.swing.table.DefaultTableModel;
import java.sql.*;

public class LogPage extends javax.swing.JFrame {

    private static final String DB_URL = "jdbc:mysql://localhost:3306/travel_agency";
    private static final String USERNAME = "root";
    private static final String PASSWORD = "2912002";
    private static Connection myConnection;
    private static PreparedStatement myPreparedStatement;
    private static ResultSet myResultSet;
    private static ResultSetMetaData myMetaData;

    String GET_LOG_QUERY = "SELECT * FROM table_log";

    public LogPage() {
        initComponents();
        setVisible(true);

        setSize(620, 750);
        setLocationRelativeTo(null);
        setResizable(false);

        mainPanel.setBackground(new Color(666666));

        tableNameLabel.setForeground(Color.WHITE);
```

```
tableNameLabel.setHorizontalAlignment(SwingConstants.CENTER);
```

```
initializeThisPage();
```

```
}
```

```
private void initializeThisPage(){
```

```
    try {
```

```
        myConnection = DriverManager.getConnection(DB_URL, USERNAME, PASSWORD);
```

```
        myPreparedStatement = myConnection.prepareStatement(GET_LOG_QUERY);
```

```
        myResultSet = myPreparedStatement.executeQuery();
```

```
        myMetaData = myResultSet.getMetaData();
```

```
        DefaultTableModel tModel = (DefaultTableModel) logTable.getModel();
```

```
        int columnCount = myMetaData.getColumnCount();
```

```
        for(int i = 1; i <= columnCount; i++)
```

```
            tModel.addColumn(myMetaData.getColumnName(i));
```

```
        while(myResultSet.next()){
```

```
            String[] newRowData = new String[columnCount];
```

```
            for(int i = 0; i < columnCount; i++)
```

```
                newRowData[i] = myResultSet.getObject(i + 1).toString();
```

```
            tModel.addRow(newRowData);
```

```
        }
```

```
    }
```

```
    catch (SQLException ex) {
```

```
        ex.printStackTrace();
```

```
    }
```

```
    finally{
```

```
        if(myConnection != null)
```

```
            try{
```

```
                myConnection.close();
```

```

        } catch(SQLException ex){}

        if(myPreparedStatement != null)
            try{
                myPreparedStatement.close();
            } catch(SQLException ex){}

        if(myResultSet != null)
            try{
                myResultSet.close();
            } catch(SQLException ex){}
    }
}

@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">
private void initComponents() {

    mainPanel = new javax.swing.JPanel();
    jScrollPane1 = new javax.swing.JScrollPane();
    logTable = new javax.swing.JTable();
    tableNameLabel = new javax.swing.JLabel();
    returnButton = new javax.swing.JButton();

    setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);

    logTable.setModel(new javax.swing.table.DefaultTableModel(
        new Object [][] {

        },
        new String [] {

        }
    ));

```

```

jScrollPane1.setViewportView(logTable);

tableNameLabel.setFont(new java.awt.Font("Segoe UI", 1, 24)); // NOI18N
tableNameLabel.setText("Log");

returnButton.setText("Main Menu");
returnButton.addMouseListener(new java.awt.event.MouseAdapter() {
    public void mouseClicked(java.awt.event.MouseEvent evt) {
        returnButtonMouseClicked(evt);
    }
});

javax.swing.GroupLayout mainPanelLayout = new javax.swing.GroupLayout(mainPanel);
mainPanel.setLayout(mainPanelLayout);
mainPanelLayout.setHorizontalGroup(
    mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(mainPanelLayout.createSequentialGroup()
            .add(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                .add(mainPanelLayout.createSequentialGroup()
                    .addGap(58, 58, 58)
                    .addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE, 504,
javax.swing.GroupLayout.PREFERRED_SIZE)
                    .addContainerGap(44, Short.MAX_VALUE)
                    .addGroup(javax.swing.GroupLayout.Alignment.TRAILING, mainPanelLayout.createSequentialGroup()
                        .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
                        .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                            .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
mainPanelLayout.createSequentialGroup()
                                .addComponent(tableNameLabel, javax.swing.GroupLayout.PREFERRED_SIZE, 313,
javax.swing.GroupLayout.PREFERRED_SIZE)
                                .addGap(138, 138, 138)
                                .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
mainPanelLayout.createSequentialGroup()
                                    .addComponent(returnButton, javax.swing.GroupLayout.PREFERRED_SIZE, 173,
javax.swing.GroupLayout.PREFERRED_SIZE)
                                    .addGap(205, 205, 205))))
                            .addGroup(mainPanelLayout.createSequentialGroup()
                                .addComponent(returnButton, javax.swing.GroupLayout.PREFERRED_SIZE, 173,
javax.swing.GroupLayout.PREFERRED_SIZE)
                                .addGap(205, 205, 205))))
                        .addGap(205, 205, 205))))
            .addContainerGap(138, Short.MAX_VALUE))
        .addGroup(javax.swing.GroupLayout.Alignment.TRAILING, mainPanelLayout.createSequentialGroup()
            .addContainerGap()
            .addComponent(returnButton, javax.swing.GroupLayout.PREFERRED_SIZE, 173,
javax.swing.GroupLayout.PREFERRED_SIZE)
            .addGap(205, 205, 205)
            .addContainerGap())
    );
mainPanelLayout.setVerticalGroup(

```



```

        mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(mainPanelLayout.createSequentialGroup()
                .addContainerGap()
                .addComponent(tableNameLabel, javax.swing.GroupLayout.PREFERRED_SIZE, 44,
                    javax.swing.GroupLayout.PREFERRED_SIZE)
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED,
                    javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
                .addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE, 579,
                    javax.swing.GroupLayout.PREFERRED_SIZE)
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, 18, Short.MAX_VALUE)
                .addComponent(returnButton, javax.swing.GroupLayout.PREFERRED_SIZE, 36,
                    javax.swing.GroupLayout.PREFERRED_SIZE)
                .addContainerGap(20, Short.MAX_VALUE))
            );

```

```

        javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
        getContentPane().setLayout(layout);
        layout.setHorizontalGroup(
            layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                .addComponent(mainPanel, javax.swing.GroupLayout.Alignment.TRAILING,
                    javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        );
        layout.setVerticalGroup(
            layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                .addComponent(mainPanel, javax.swing.GroupLayout.Alignment.TRAILING,
                    javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        );

```

```

        pack();
    } // </editor-fold>

```

```

    private void returnButtonMouseClicked(java.awt.event.MouseEvent evt) {
        MainMenu mainMen = new MainMenu();
        dispose();
    }

```

```

    public static void main(String args[]) {

```

```

/* Set the Nimbus look and feel */
//<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
/* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
 * For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
 */
try {
    for (javax.swing.UIManager.LookAndFeelInfo info : javax.swing.UIManager.getInstalledLookAndFeels()) {
        if ("Nimbus".equals(info.getName())) {
            javax.swing.UIManager.setLookAndFeel(info.getClassName());
            break;
        }
    }
} catch (ClassNotFoundException ex) {
    java.util.logging.Logger.getLogger(LogPage.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);
} catch (InstantiationException ex) {
    java.util.logging.Logger.getLogger(LogPage.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);
} catch (IllegalAccessException ex) {
    java.util.logging.Logger.getLogger(LogPage.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);
} catch (javax.swing.UnsupportedLookAndFeelException ex) {
    java.util.logging.Logger.getLogger(LogPage.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);
}
//</editor-fold>

/* Create and display the form */
java.awt.EventQueue.invokeLater(new Runnable() {
    public void run() {
        new LogPage().setVisible(true);
    }
});
}

// Variables declaration - do not modify
private javax.swing.JScrollPane jScrollPane1;
private javax.swing.JTable logTable;

```

```
private javax.swing.JPanel mainPanel;  
private javax.swing.JButton returnButton;  
private javax.swing.JLabel tableNameLabel;  
// End of variables declaration  
}
```

AddITPage.java :

```
import java.awt.Color;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import javax.swing.SwingConstants;

public class AddITPage extends javax.swing.JFrame {

    private static final String DB_URL = "jdbc:mysql://localhost:3306/travel_agency";
    private static final String USERNAME = "root";
    private static final String PASSWORD = "2912002";
    private static Connection myConnection;
    private static PreparedStatement myPreparedStatement;
    private static ResultSet myResultSet;

    public AddITPage() {
        initComponents();
        setVisible(true);

        setLocationRelativeTo(null);
        setResizable(false);

        mainPanel.setBackground(new Color(666666));

        tableNameLabel.setForeground(Color.WHITE);
        tableNameLabel.setHorizontalAlignment(SwingConstants.CENTER);
        idLabel.setForeground(Color.WHITE);
        nameLabel.setForeground(Color.WHITE);
        lnameLabel.setForeground(Color.WHITE);
        passwordLabel.setForeground(Color.WHITE);
        salaryLabel.setForeground(Color.WHITE);
```

```

        branchLabel.setForeground(Color.WHITE);
        errorMessage.setForeground(Color.RED);
        errorMessage.setVisible(false);

        initializeBranchList();
    }

    private void initializeBranchList(){
        try{
            myConnection = DriverManager.getConnection(DB_URL, USERNAME, PASSWORD);
            myPreparedStatement = myConnection.prepareStatement("SELECT br_code FROM branch");
            myResultSet = myPreparedStatement.executeQuery();

            branch.removeAllItems();
            branch.addItem("Branch");
            while(myResultSet.next())
                branch.addItem(myResultSet.getObject(1).toString());
        }
        catch(SQLException ex){

        }
        finally{

        }
    }
}

```

```

@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">
private void initComponents() {

    mainPanel = new javax.swing.JPanel();
    tableNameLabel = new javax.swing.JLabel();
    idLabel = new javax.swing.JLabel();
    addButton = new javax.swing.JButton();
}

```

```

branch = new javax.swing.JComboBox<>();
salary = new javax.swing.JSpinner();
id = new javax.swing.JTextField();
name = new javax.swing.JTextField();
password = new javax.swing.JTextField();
lname = new javax.swing.JTextField();
nameLabel = new javax.swing.JLabel();
passwordLabel = new javax.swing.JLabel();
branchLabel = new javax.swing.JLabel();
lnameLabel = new javax.swing.JLabel();
salaryLabel = new javax.swing.JLabel();
errorMessage = new javax.swing.JLabel();
returnButton1 = new javax.swing.JButton();

setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);

tableNameLabel.setFont(new java.awt.Font("Segoe UI", 1, 24)); // NOI18N
tableNameLabel.setText("Add IT");

idLabel.setText("ID number:");

addButton.setText("Add");
addButton.addMouseListener(new java.awt.event.MouseAdapter() {
    public void mouseClicked(java.awt.event.MouseEvent evt) {
        addButtonMouseClicked(evt);
    }
});

branch.addPopupMenuListener(new javax.swing.event.PopupMenuListener() {
    public void popupMenuCanceled(javax.swing.event.PopupMenuEvent evt) {
    }
    public void popupMenuWillBecomeInvisible(javax.swing.event.PopupMenuEvent evt) {
        branchPopupMenuWillBecomeInvisible(evt);
    }
}

```

```

        public void popupMenuWillBecomeVisible(javax.swing.event.PopupMenuEvent evt) {
            }
    });

    nameLabel.setText("First name:");

    passwordLabel.setText("Password:");

    branchLabel.setText("Branch:");

    lnameLabel.setText("Last name:");

    salaryLabel.setText("Salary:");

    errorMessage.setText("You need to fill in all input fields!");

    returnButton1.setText("Main Menu");
    returnButton1.addMouseListener(new java.awt.event.MouseAdapter() {
        public void mouseClicked(java.awt.event.MouseEvent evt) {
            returnButton1MouseClicked(evt);
        }
    });

    javax.swing.GroupLayout mainPanelLayout = new javax.swing.GroupLayout(mainPanel);
    mainPanel.setLayout(mainPanelLayout);
    mainPanelLayout.setHorizontalGroup(
        mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(javax.swing.GroupLayout.Alignment.TRAILING, mainPanelLayout.createSequentialGroup()
                .addContainerGap()
                .addComponent(tableLabel, javax.swing.GroupLayout.PREFERRED_SIZE, 118,
                    javax.swing.GroupLayout.PREFERRED_SIZE)
                .addGap(118, 118, 118)
                .addComponent(salaryLabel, javax.swing.GroupLayout.PREFERRED_SIZE, 118,
                    javax.swing.GroupLayout.PREFERRED_SIZE)
                .addGap(118, 118, 118)
                .addComponent(branchLabel, javax.swing.GroupLayout.PREFERRED_SIZE, 118,
                    javax.swing.GroupLayout.PREFERRED_SIZE)
                .addGap(118, 118, 118)
                .addComponent(lnameLabel, javax.swing.GroupLayout.PREFERRED_SIZE, 118,
                    javax.swing.GroupLayout.PREFERRED_SIZE)
                .addGap(118, 118, 118)
                .addComponent(nameLabel, javax.swing.GroupLayout.PREFERRED_SIZE, 118,
                    javax.swing.GroupLayout.PREFERRED_SIZE)
                .addContainerGap(118, Short.MAX_VALUE))
    );

```

```

        .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
mainPanelLayout.createSequentialGroup())

        .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING)
            .addComponent(nameLabel)
            .addComponent(lnameLabel)
            .addComponent(idLabel))
        .addGap(18, 18, 18)
        .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING,
false)
            .addGroup(mainPanelLayout.createSequentialGroup()

                .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                    .addComponent(lname, javax.swing.GroupLayout.PREFERRED_SIZE, 71,
javax.swing.GroupLayout.PREFERRED_SIZE)
                    .addComponent(name, javax.swing.GroupLayout.PREFERRED_SIZE, 71,
javax.swing.GroupLayout.PREFERRED_SIZE))
                    .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)

                .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                    .addComponent(branchLabel, javax.swing.GroupLayout.Alignment.TRAILING)
                    .addComponent(salaryLabel, javax.swing.GroupLayout.Alignment.TRAILING)))
                .addGroup(mainPanelLayout.createSequentialGroup()
                    .addComponent(id, javax.swing.GroupLayout.PREFERRED_SIZE, 71,
javax.swing.GroupLayout.PREFERRED_SIZE)
                    .addGap(43, 43, 43)
                    .addComponent(passwordLabel)))
                .addGap(18, 18, 18)
                .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
false)
                    .addComponent(password, javax.swing.GroupLayout.PREFERRED_SIZE, 71,
javax.swing.GroupLayout.PREFERRED_SIZE)
                    .addComponent(salary)
                    .addComponent(branch, 0, 75, Short.MAX_VALUE))
                    .addGap(86, 86, 86))))
            .addGroup(javax.swing.GroupLayout.Alignment.TRAILING, mainPanelLayout.createSequentialGroup()
                .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
                .addComponent(errorMessage, javax.swing.GroupLayout.PREFERRED_SIZE, 199,
javax.swing.GroupLayout.PREFERRED_SIZE)

```



```

        .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))
    .addGroup(javax.swing.GroupLayout.Alignment.TRAILING, mainPanelLayout.createSequentialGroup())
        .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        .addComponent(addButton, javax.swing.GroupLayout.PREFERRED_SIZE, 173,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addGap(183, 183, 183))
    .addGroup(javax.swing.GroupLayout.Alignment.TRAILING, mainPanelLayout.createSequentialGroup())
        .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        .addComponent(returnButton1)
        .addGap(218, 218, 218))
);
mainPanelLayout.setVerticalGroup(
    mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(mainPanelLayout.createSequentialGroup())
        .addGap(17, 17, 17)
        .addComponent(tableNameLabel, javax.swing.GroupLayout.PREFERRED_SIZE, 44,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, 44, Short.MAX_VALUE)
        .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
            .addComponent(id, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
            .addComponent(idLabel)
            .addComponent(passwordLabel)
            .addComponent(password, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE))
        .addGap(18, 18, 18)
        .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
            .addComponent(name, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
            .addComponent(nameLabel)
            .addComponent(salaryLabel)
            .addComponent(salary, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE))
        .addGap(18, 18, 18)
        .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
            .addComponent(lname, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)

```

```

        .addComponent(lnameLabel)

        .addComponent(branchLabel)

        .addComponent(branch, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE))

        .addGap(29, 29, 29)

        .addComponent(errorMessage)

        .addGap(33, 33, 33)

        .addComponent(addButton, javax.swing.GroupLayout.PREFERRED_SIZE, 65,
javax.swing.GroupLayout.PREFERRED_SIZE)

        .addGap(26, 26, 26)

        .addComponent(returnButton1)

        .addGap(22, 22, 22))
    );

    javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
    getContentPane().setLayout(layout);
    layout.setHorizontalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addComponent(mainPanel, javax.swing.GroupLayout.Alignment.TRAILING,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
    );
    layout.setVerticalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addComponent(mainPanel, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
    );

    pack();
} // </editor-fold>

```

```

private void branchPopupMenuWillBecomeInvisible(javax.swing.event.PopupMenuEvent evt) {
    if(!branch.getSelectedItem().equals("Branch"))
        branch.removeItem("Branch");
}

```

```

private void addButtonMouseClicked(java.awt.event.MouseEvent evt) {

```

```

errorMessage.setVisible(false);

String newID = id.getText();
String newName = name.getText();
String newLname = lname.getText();
String newPassword = password.getText();
float newSalary = Integer.parseInt(salary.getValue().toString());
int newBranch = 0;

if(!branch.getSelectedItem().toString().equals("Branch"))
    newBranch = Integer.parseInt(branch.getSelectedItem().toString());

if(newID.equals("") || newName.equals("") ||
    newLname.equals("") || newPassword.equals("") ||
    newSalary == 0 || newBranch == 0){
    errorMessage.setVisible(true);
}
else{
    String WORKER_INSERT_QUERY = "INSERT INTO worker VALUES (\\"" + newID + "\",\"" + newName +
    "\",\"" + newLname + "\",\" + newSalary + \",\" + newBranch + \")";

    String IT_INSERT_QUERY = "INSERT INTO IT VALUES (\\"" + newID + "\",\"" + newPassword + "\",\"" +
    "2022-01-01" + "\",\" + \"NULL\");

    try{
        myConnection = DriverManager.getConnection(DB_URL, USERNAME, PASSWORD);
        myPreparedStatement = myConnection.prepareStatement(WORKER_INSERT_QUERY);
        int test = myPreparedStatement.executeUpdate();

        myPreparedStatement = myConnection.prepareStatement(IT_INSERT_QUERY);
        test = myPreparedStatement.executeUpdate();

    }
    catch(SQLException ex){

```

```
    }  
    finally{  
  
    }  
}  
}
```

```
private void returnButton1MouseClicked(java.awt.event.MouseEvent evt) {  
    MainMenu mainMen = new MainMenu();  
    dispose();  
}
```

```
public static void main(String args[]) {  
    /* Set the Nimbus look and feel */  
    //<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">  
    /* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.  
    * For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html  
    */  
    try {  
        for (javax.swing.UIManager.LookAndFeelInfo info : javax.swing.UIManager.getInstalledLookAndFeels()) {  
            if ("Nimbus".equals(info.getName())) {  
                javax.swing.UIManager.setLookAndFeel(info.getClassName());  
                break;  
            }  
        }  
    } catch (ClassNotFoundException ex) {  
        java.util.logging.Logger.getLogger(AddITPage.class.getName()).log(java.util.logging.Level.SEVERE, null,  
ex);  
    } catch (InstantiationException ex) {  
        java.util.logging.Logger.getLogger(AddITPage.class.getName()).log(java.util.logging.Level.SEVERE, null,  
ex);  
    } catch (IllegalAccessException ex) {  
        java.util.logging.Logger.getLogger(AddITPage.class.getName()).log(java.util.logging.Level.SEVERE, null,  
ex);  
    } catch (javax.swing.UnsupportedLookAndFeelException ex) {
```

```
        java.util.logging.Logger.getLogger(AddITPage.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
    }
    //</editor-fold>
```

```
    /* Create and display the form */
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new AddITPage().setVisible(true);
        }
    });
}
```

```
// Variables declaration - do not modify
private javax.swing.JButton addButton;
private javax.swing.JComboBox<String> branch;
private javax.swing.JLabel branchLabel;
private javax.swing.JLabel errorMessage;
private javax.swing.JTextField id;
private javax.swing.JLabel idLabel;
private javax.swing.JTextField lname;
private javax.swing.JLabel lnameLabel;
private javax.swing.JPanel mainPanel;
private javax.swing.JTextField name;
private javax.swing.JLabel nameLabel;
private javax.swing.JTextField password;
private javax.swing.JLabel passwordLabel;
private javax.swing.JButton returnButton;
private javax.swing.JButton returnButton1;
private javax.swing.JSpinner salary;
private javax.swing.JLabel salaryLabel;
private javax.swing.JLabel tableNameLabel;
// End of variables declaration
}
```

CustomerReservationPage.java :

```
import java.awt.Color;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.ResultSetMetaData;
import java.sql.SQLException;
import java.util.logging.Level;
import java.util.logging.Logger;
import javax.swing.SwingConstants;
import javax.swing.table.DefaultTableModel;

public class CustomerReservationPage extends javax.swing.JFrame {

    private static final String DB_URL = "jdbc:mysql://localhost:3306/travel_agency";
    private static final String USERNAME = "root";
    private static final String PASSWORD = "2912002";
    private static Connection myConnection;
    private static PreparedStatement myPreparedStatement;
    private static ResultSet myResultSet;
    private static ResultSetMetaData myMetaData;

    public CustomerReservationPage() {
        initComponents();
        setVisible(true);

        setLocationRelativeTo(null);
        setResizable(false);

        mainPanel.setBackground(new Color(666666));

        reservationTable.setEnabled(false);
    }
}
```

```

        tableNameLabel.setForeground(Color.WHITE);
        tableNameLabel.setHorizontalAlignment(SwingConstants.CENTER);
        lnameText.setForeground(Color.WHITE);

        errorMessage.setVisible(false);
        errorMessage.setForeground(Color.RED);
    }

    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {

        mainPanel = new javax.swing.JPanel();
        returnButton = new javax.swing.JButton();
        tableNameLabel = new javax.swing.JLabel();
        jScrollPane1 = new javax.swing.JScrollPane();
        reservationTable = new javax.swing.JTable();
        lnameText = new javax.swing.JLabel();
        searchButton = new javax.swing.JButton();
        lname = new javax.swing.JTextField();
        errorMessage = new javax.swing.JLabel();

        setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);

        returnButton.setText("Main Menu");
        returnButton.addMouseListener(new java.awt.event.MouseAdapter() {
            public void mouseClicked(java.awt.event.MouseEvent evt) {
                returnButtonMouseClicked(evt);
            }
        });

        tableNameLabel.setFont(new java.awt.Font("Segoe UI", 1, 24)); // NOI18N
        tableNameLabel.setText("Customer Reservations");

```

```
jScrollPane1.setViewportViewView(reservationTable);
```

```
lnameText.setText("Search by customer last name:");
```

```
searchButton.setText("Search");
```

```
searchButton.addMouseListener(new java.awt.event.MouseAdapter() {
```

```
    public void mouseClicked(java.awt.event.MouseEvent evt) {
```

```
        searchButtonMouseClicked(evt);
```

```
    }
```

```
});
```

```
errorMessage.setText("You have to enter a last name to search!");
```

```
javax.swing.GroupLayout mainPanelLayout = new javax.swing.GroupLayout(mainPanel);
```

```
mainPanel.setLayout(mainPanelLayout);
```

```
mainPanelLayout.setHorizontalGroup(
```

```
    mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
```

```
        .addGroup(mainPanelLayout.createSequentialGroup()
```

```
            .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
```

```
                .addGroup(mainPanelLayout.createSequentialGroup()
```

```
                    .addGap(339, 339, 339)
```

```
                    .addComponent(returnButton, javax.swing.GroupLayout.PREFERRED_SIZE, 173,  
javax.swing.GroupLayout.PREFERRED_SIZE))
```

```
                .addGroup(mainPanelLayout.createSequentialGroup()
```

```
                    .addGap(270, 270, 270)
```

```
                    .addComponent(tableNameLabel, javax.swing.GroupLayout.PREFERRED_SIZE, 313,  
javax.swing.GroupLayout.PREFERRED_SIZE)))
```

```
                .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))
```

```
        .addGroup(mainPanelLayout.createSequentialGroup()
```

```
            .addGap(290, 290, 290)
```

```
            .addComponent(errorMessage, javax.swing.GroupLayout.PREFERRED_SIZE, 241,  
javax.swing.GroupLayout.PREFERRED_SIZE))
```

```
            .addGap(0, 0, Short.MAX_VALUE))
```

```
        .addGroup(javax.swing.GroupLayout.Alignment.TRAILING, mainPanelLayout.createSequentialGroup()
```

```
            .addContainerGap(38, Short.MAX_VALUE)
```

```
            .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
```



```

        .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
mainPanelLayout.createSequentialGroup())

        .addComponent(lnameText)

        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)

        .addComponent(lname, javax.swing.GroupLayout.PREFERRED_SIZE, 71,
javax.swing.GroupLayout.PREFERRED_SIZE)

        .addGap(66, 66, 66)

        .addComponent(searchButton, javax.swing.GroupLayout.PREFERRED_SIZE, 115,
javax.swing.GroupLayout.PREFERRED_SIZE)

        .addGap(237, 237, 237))

        .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
mainPanelLayout.createSequentialGroup())

        .addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE, 791,
javax.swing.GroupLayout.PREFERRED_SIZE)

        .addGap(24, 24, 24))))
);
mainPanelLayout.setVerticalGroup(
    mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

    .addGroup(javax.swing.GroupLayout.Alignment.TRAILING, mainPanelLayout.createSequentialGroup())

    .addGap(20, 20, 20)

    .addComponent(tableNameLabel, javax.swing.GroupLayout.PREFERRED_SIZE, 44,
javax.swing.GroupLayout.PREFERRED_SIZE)

    .addGap(29, 29, 29)

    .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)

        .addComponent(lnameText)

        .addComponent(searchButton, javax.swing.GroupLayout.PREFERRED_SIZE, 37,
javax.swing.GroupLayout.PREFERRED_SIZE)

        .addComponent(lname, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE))

    .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, 30, Short.MAX_VALUE)

    .addComponent(errorMessage)

    .addGap(34, 34, 34)

    .addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)

    .addGap(50, 50, 50)

    .addComponent(returnButton, javax.swing.GroupLayout.PREFERRED_SIZE, 39,
javax.swing.GroupLayout.PREFERRED_SIZE)

    .addGap(16, 16, 16))

```

```

);

javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
getContentPane().setLayout(layout);

layout.setHorizontalGroup(
    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addComponent(mainPanel, javax.swing.GroupLayout.Alignment.TRAILING,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
);

layout.setVerticalGroup(
    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addComponent(mainPanel, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
);

pack();
} // </editor-fold>

```

```

private void returnButtonMouseClicked(java.awt.event.MouseEvent evt) {
    MainMenu mainMen = new MainMenu();
    dispose();
}

private void searchButtonMouseClicked(java.awt.event.MouseEvent evt) {
    errorMessage.setVisible(false);

    if(lname.getText().equals(""))
        errorMessage.setVisible(true);
    else{
        try {
            DefaultTableModel tModel = (DefaultTableModel) reservationTable.getModel();

            myConnection = DriverManager.getConnection(DB_URL, USERNAME, PASSWORD);

            String GET_CUSTOMER_RESERVATION_DATA = "CALL find_reservation_by_last_name(\"" +
lname.getText() + "\")";

```

```

myPreparedStatement = myConnection.prepareStatement(GET_CUSTOMER_RESERVATION_DATA);
myResultSet = myPreparedStatement.executeQuery();
myMetaData = myResultSet.getMetaData();

tModel.setRowCount(0);
tModel.setColumnCount(0);

for(int i = 1; i <= myMetaData.getColumnCount(); i++){
    tModel.addColumn(myMetaData.getColumnName(i));
}

while(myResultSet.next()){
    String[] newRow = new String[myMetaData.getColumnCount()];

    for(int i = 0; i < myMetaData.getColumnCount(); i++)
        newRow[i] = myResultSet.getObject(i + 1).toString();

    tModel.addRow(newRow);
}
}
catch (SQLException ex) {
    ex.printStackTrace();
}
finally{

}
}
}

public static void main(String args[]) {

    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {

```

```
        new CustomerReservationPage().setVisible(true);
    }
    });
}

// Variables declaration - do not modify
private javax.swing.JLabel errorMessage;
private javax.swing.JScrollPane jScrollPane1;
private javax.swing.JTextField lname;
private javax.swing.JLabel lnameText;
private javax.swing.JPanel mainPanel;
private javax.swing.JTable reservationTable;
private javax.swing.JButton returnButton;
private javax.swing.JButton searchButton;
private javax.swing.JLabel tableNameLabel;
// End of variables declaration
}
```

BranchTripInfoPage.java :

```
import java.awt.Color;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.ResultSetMetaData;
import java.sql.SQLException;
import java.util.logging.Level;
import java.util.logging.Logger;
import javax.swing.DefaultComboBoxModel;
import javax.swing.SwingConstants;
import javax.swing.table.DefaultTableModel;

public class BranchTripInfoPage extends javax.swing.JFrame {

    private static final String DB_URL = "jdbc:mysql://localhost:3306/travel_agency";
    private static final String USERNAME = "root";
    private static final String PASSWORD = "2912002";
    private static Connection myConnection;
    private static PreparedStatement myPreparedStatement;
    private static ResultSet myResultSet;
    private static ResultSet myResultSet2;
    private static ResultSetMetaData myMetaData;

    public BranchTripInfoPage() {
        initComponents();
        setVisible(true);

        setSize(1100, 750);
        setLocationRelativeTo(null);
        setResizable(false);
    }
}
```

```

mainPanel.setBackground(new Color(666666));

tripDataTable.setEnabled(false);

text1.setForeground(Color.WHITE);
text1.setHorizontalAlignment(SwingConstants.CENTER);
text2.setForeground(Color.WHITE);
text2.setHorizontalAlignment(SwingConstants.CENTER);
tableNameLabel.setForeground(Color.WHITE);
tableNameLabel.setHorizontalAlignment(SwingConstants.CENTER);
}

private void fillTable(){
    try {
        DefaultTableModel tModel = (DefaultTableModel) tripDataTable.getModel();
        tModel.setColumnCount(0);
        tModel.setRowCount(0);

        String[] columnNames = {"br_code", "tr_cost", "tr_maxseats", "reservations", "empty seats", "Driver
lname", "Driver name", "Guide lname", "Guide name", "tr_departure", "tr_return"};

        for(String name : columnNames)
            tModel.addColumn(name);

        myConnection = DriverManager.getConnection(DB_URL, USERNAME, PASSWORD);

        String GET_BRANCH_CODES = "SELECT br_code FROM branch";

        myPreparedStatement = myConnection.prepareStatement(GET_BRANCH_CODES);
        myResultSet = myPreparedStatement.executeQuery();

        while(myResultSet.next()){
            String CALL_PROCEDURE = "CALL check_branch_trips(" + myResultSet.getObject(1).toString() + ", \"\" +
formStartDate() + "\", \"\" + formEndDate() + \"\")";

```

```

myPreparedStatement = myConnection.prepareStatement(CALL_PROCEDURE);
myResultSet2 = myPreparedStatement.executeQuery();
myMetaData = myResultSet2.getMetaData();

while(myResultSet2.next()){
    String[] newRow = new String[myMetaData.getColumnCount() + 1];
    newRow[0] = myResultSet2.getObject(1).toString();

    for(int i = 1; i <= myMetaData.getColumnCount(); i++){
        if(myResultSet2.getObject(i) == null)
            newRow[i] = "NULL";
        else
            newRow[i] = myResultSet2.getObject(i).toString();
    }

    tModel.addRow(newRow);
}
} catch (SQLException ex) {
    ex.printStackTrace();
}

}

private String formStartDate(){
    return startYear.getSelectedItemAt().toString() + "-" + startMonth.getSelectedItemAt().toString() + "-" +
startDay.getSelectedItemAt().toString() + " 00:00:00";
}

private String formEndDate(){
    return endYear.getSelectedItemAt().toString() + "-" + endMonth.getSelectedItemAt().toString() + "-" +
endDay.getSelectedItemAt().toString() + " 00:00:00";
}

@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">

```

```

private void initComponents() {

    mainPanel = new javax.swing.JPanel();
    jScrollPane1 = new javax.swing.JScrollPane();
    tripDataTable = new javax.swing.JTable();
    returnButton = new javax.swing.JButton();
    tableNameLabel = new javax.swing.JLabel();
    endMonth = new javax.swing.JComboBox<>();
    startMonth = new javax.swing.JComboBox<>();
    endDay = new javax.swing.JComboBox<>();
    startDay = new javax.swing.JComboBox<>();
    startYear = new javax.swing.JComboBox<>();
    endYear = new javax.swing.JComboBox<>();
    text1 = new javax.swing.JLabel();
    text2 = new javax.swing.JLabel();

    setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);

    jScrollPane1.setViewportView(tripDataTable);

    returnButton.setText("Main Menu");
    returnButton.addMouseListener(new java.awt.event.MouseAdapter() {
        public void mouseClicked(java.awt.event.MouseEvent evt) {
            returnButtonMouseClicked(evt);
        }
    });

    tableNameLabel.setFont(new java.awt.Font("Segoe UI", 1, 24)); // NOI18N
    tableNameLabel.setText("Branch Trip Info");

    endMonth.setModel(new javax.swing.DefaultComboBoxModel<>(new String[] { "1", "2", "3", "4", "5", "6",
"7", "8", "9", "10", "11", "12" }));
    endMonth.addPopupMenuListener(new javax.swing.event.PopupMenuListener() {
        public void popupMenuCanceled(javax.swing.event.PopupMenuEvent evt) {
        }
    }

```



```

        public void popupMenuWillBecomeInvisible(javax.swing.event.PopupMenuEvent evt) {
            endMonthPopupMenuWillBecomeInvisible(evt);
        }

        public void popupMenuWillBecomeVisible(javax.swing.event.PopupMenuEvent evt) {
        }

    });

    startMonth.setModel(new javax.swing.DefaultComboBoxModel<>(new String[] { "1", "2", "3", "4", "5",
"6", "7", "8", "9", "10", "11", "12" }));

    startMonth.addPopupMenuListener(new javax.swing.event.PopupMenuListener() {
        public void popupMenuCanceled(javax.swing.event.PopupMenuEvent evt) {
        }

        public void popupMenuWillBecomeInvisible(javax.swing.event.PopupMenuEvent evt) {
            startMonthPopupMenuWillBecomeInvisible(evt);
        }

        public void popupMenuWillBecomeVisible(javax.swing.event.PopupMenuEvent evt) {
        }

    });

    startMonth.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            startMonthActionPerformed(evt);
        }

    });

    endDay.setModel(new javax.swing.DefaultComboBoxModel<>(new String[] { "1", "2", "3", "4", "5", "6",
"7", "8", "9", "10", "11", "12", "13", "14", "15", "16", "17", "18", "19", "20", "21", "22", "23", "24", "25", "26",
"27", "28", "29", "30", "31" }));

    endDay.addPopupMenuListener(new javax.swing.event.PopupMenuListener() {
        public void popupMenuCanceled(javax.swing.event.PopupMenuEvent evt) {
        }

        public void popupMenuWillBecomeInvisible(javax.swing.event.PopupMenuEvent evt) {
            endDayPopupMenuWillBecomeInvisible(evt);
        }

        public void popupMenuWillBecomeVisible(javax.swing.event.PopupMenuEvent evt) {
        }

    });

```

```
startDay.setModel(new javax.swing.DefaultComboBoxModel<>(new String[] { "1", "2", "3", "4", "5", "6",  
"7", "8", "9", "10", "11", "12", "13", "14", "15", "16", "17", "18", "19", "20", "21", "22", "23", "24", "25", "26",  
"27", "28", "29", "30", "31" }));
```

```
startDay.addPopupMenuListener(new javax.swing.event.PopupMenuListener() {  
    public void popupMenuCanceled(javax.swing.event.PopupMenuEvent evt) {  
    }  
    public void popupMenuWillBecomeInvisible(javax.swing.event.PopupMenuEvent evt) {  
        startDayPopupMenuWillBecomeInvisible(evt);  
    }  
    public void popupMenuWillBecomeVisible(javax.swing.event.PopupMenuEvent evt) {  
    }  
});
```

```
startYear.setModel(new javax.swing.DefaultComboBoxModel<>(new String[] { "2022", "2023", "2024",  
"2025", " " }));
```

```
startYear.addPopupMenuListener(new javax.swing.event.PopupMenuListener() {  
    public void popupMenuCanceled(javax.swing.event.PopupMenuEvent evt) {  
    }  
    public void popupMenuWillBecomeInvisible(javax.swing.event.PopupMenuEvent evt) {  
        startYearPopupMenuWillBecomeInvisible(evt);  
    }  
    public void popupMenuWillBecomeVisible(javax.swing.event.PopupMenuEvent evt) {  
    }  
});
```

```
endYear.setModel(new javax.swing.DefaultComboBoxModel<>(new String[] { "2022", "2023", "2024",  
"2025", " " }));
```

```
endYear.addPopupMenuListener(new javax.swing.event.PopupMenuListener() {  
    public void popupMenuCanceled(javax.swing.event.PopupMenuEvent evt) {  
    }  
    public void popupMenuWillBecomeInvisible(javax.swing.event.PopupMenuEvent evt) {  
        endYearPopupMenuWillBecomeInvisible(evt);  
    }  
    public void popupMenuWillBecomeVisible(javax.swing.event.PopupMenuEvent evt) {  
    }  
});
```

```

});

text1.setText("From:");

text2.setText("To:");

javax.swing.GroupLayout mainPanelLayout = new javax.swing.GroupLayout(mainPanel);
mainPanel.setLayout(mainPanelLayout);
mainPanelLayout.setHorizontalGroup(
    mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(mainPanelLayout.createSequentialGroup()
            .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                .addGroup(mainPanelLayout.createSequentialGroup()
                    .addGroup(mainPanelLayout.createSequentialGroup()
                        .addGroup(mainPanelLayout.createSequentialGroup()
                            .addGroup(mainPanelLayout.createSequentialGroup()
                                .addGap(317, 317, 317)
                                .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                                    .addGroup(mainPanelLayout.createSequentialGroup()
                                        .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
                                            mainPanelLayout.createSequentialGroup()
                                                .addComponent(tableNameLabel, javax.swing.GroupLayout.PREFERRED_SIZE, 313,
                                                    javax.swing.GroupLayout.PREFERRED_SIZE)
                                                .addGap(70, 70, 70))
                                            .addGroup(mainPanelLayout.createSequentialGroup()
                                                .addComponent(text1)
                                                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
                                                .addComponent(startMonth, javax.swing.GroupLayout.PREFERRED_SIZE, 45,
                                                    javax.swing.GroupLayout.PREFERRED_SIZE)
                                                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
                                                .addComponent(startDay, javax.swing.GroupLayout.PREFERRED_SIZE, 47,
                                                    javax.swing.GroupLayout.PREFERRED_SIZE)
                                                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
                                                .addComponent(startYear, javax.swing.GroupLayout.PREFERRED_SIZE, 61,
                                                    javax.swing.GroupLayout.PREFERRED_SIZE)
                                                .addGap(45, 45, 45)
                                                .addComponent(text2)
                                                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
                                                .addComponent(endMonth, javax.swing.GroupLayout.PREFERRED_SIZE, 45,
                                                    javax.swing.GroupLayout.PREFERRED_SIZE)
                                                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)

```

```

        .addComponent(endDay, javax.swing.GroupLayout.PREFERRED_SIZE, 47,
javax.swing.GroupLayout.PREFERRED_SIZE)

        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)

        .addComponent(endYear, javax.swing.GroupLayout.PREFERRED_SIZE, 61,
javax.swing.GroupLayout.PREFERRED_SIZE)))

    .addGroup(mainPanelLayout.createSequentialGroup())

    .addGap(453, 453, 453)

    .addComponent(returnButton, javax.swing.GroupLayout.PREFERRED_SIZE, 173,
javax.swing.GroupLayout.PREFERRED_SIZE))

    .addGroup(mainPanelLayout.createSequentialGroup())

    .addGap(29, 29, 29)

    .addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE, 1038,
javax.swing.GroupLayout.PREFERRED_SIZE))

    .addContainerGap(21, Short.MAX_VALUE)
);
mainPanelLayout.setVerticalGroup(
    mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(mainPanelLayout.createSequentialGroup())
    .addContainerGap(21, Short.MAX_VALUE)
    .addComponent(tableNameLabel, javax.swing.GroupLayout.PREFERRED_SIZE, 44,
javax.swing.GroupLayout.PREFERRED_SIZE)
    .addGap(36, 36, 36)
    .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
    .addComponent(endMonth, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
    .addComponent(startMonth, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
    .addComponent(endDay, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
    .addComponent(startDay, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
    .addComponent(startYear, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
    .addComponent(endYear, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
    .addComponent(text1)
    .addComponent(text2))
    .addGap(32, 32, 32)

```

```

        .addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE, 445,
javax.swing.GroupLayout.PREFERRED_SIZE)

        .addGap(18, 18, 18)

        .addComponent(returnButton, javax.swing.GroupLayout.PREFERRED_SIZE, 39,
javax.swing.GroupLayout.PREFERRED_SIZE)

        .addGap(21, 21, 21))
    );

    javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
    getContentPane().setLayout(layout);
    layout.setHorizontalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addComponent(mainPanel, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
    );
    layout.setVerticalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addComponent(mainPanel, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
    );

    pack();
} // </editor-fold>

```

```

private void returnButtonMouseClicked(java.awt.event.MouseEvent evt) {
    MainMenu mainMen = new MainMenu();
    dispose();
}

```

```

private void startMonthPopupMenuWillBecomeInvisible(javax.swing.event.PopupMenuEvent evt) {
    startDay.removeAllItems();

```

```

    if(startMonth.getSelectedItem().toString().equals("4") ||
        startMonth.getSelectedItem().toString().equals("6") ||
        startMonth.getSelectedItem().toString().equals("9") ||
        startMonth.getSelectedItem().toString().equals("11")){

```

```

        for(int i = 1; i <= 30; i++)
            startDay.addItem(Integer.toString(i));
    }
    else if(startMonth.getSelectedItem().toString().equals("2")){
        for(int i = 1; i <= 28; i++)
            startDay.addItem(Integer.toString(i));
    }
    else{
        for(int i = 1; i <= 31; i++)
            startDay.addItem(Integer.toString(i));
    }

    fillTable();
}

private void startDayPopupMenuWillBecomeInvisible(javax.swing.event.PopupMenuEvent evt) {
    fillTable();
}

private void startYearPopupMenuWillBecomeInvisible(javax.swing.event.PopupMenuEvent evt) {
    fillTable();
}

private void endMonthPopupMenuWillBecomeInvisible(javax.swing.event.PopupMenuEvent evt) {
    endDay.removeAllItems();

    if(endMonth.getSelectedItem().toString().equals("4") ||
        endMonth.getSelectedItem().toString().equals("6") ||
        endMonth.getSelectedItem().toString().equals("9") ||
        endMonth.getSelectedItem().toString().equals("11")){
        for(int i = 1; i <= 30; i++)
            endDay.addItem(Integer.toString(i));
    }
    else if(startMonth.getSelectedItem().toString().equals("2")){

```

```

        for(int i = 1; i <= 28; i++)
            endDay.addItem(Integer.toString(i));
    }
    else{
        for(int i = 1; i <= 31; i++)
            endDay.addItem(Integer.toString(i));
    }

    fillTable();
}

private void endDayPopupMenuWillBecomeInvisible(javax.swing.event.PopupMenuEvent evt) {
    fillTable();
}

private void endYearPopupMenuWillBecomeInvisible(javax.swing.event.PopupMenuEvent evt) {
    fillTable();
}

private void startMonthActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
}

public static void main(String args[]) {
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new BranchTripInfoPage().setVisible(true);
        }
    });
}

// Variables declaration - do not modify
private javax.swing.JComboBox<String> endDay;
private javax.swing.JComboBox<String> endMonth;

```

```
private javax.swing.JComboBox<String> endYear;
private javax.swing.JScrollPane jScrollPane1;
private javax.swing.JPanel mainPanel;
private javax.swing.JButton returnButton;
private javax.swing.JComboBox<String> startDay;
private javax.swing.JComboBox<String> startMonth;
private javax.swing.JComboBox<String> startYear;
private javax.swing.JLabel tableNameLabel;
private javax.swing.JLabel text1;
private javax.swing.JLabel text2;
private javax.swing.JTable tripDataTable;
// End of variables declaration
}
```


BranchInfo.java :

```
import java.awt.Color;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.ResultSetMetaData;
import java.sql.SQLException;
import java.util.logging.Level;
import java.util.logging.Logger;
import javax.swing.SwingConstants;
import javax.swing.table.DefaultTableModel;

public class BranchInfo extends javax.swing.JFrame {

    private static final String DB_URL = "jdbc:mysql://localhost:3306/travel_agency";
    private static final String USERNAME = "root";
    private static final String PASSWORD = "2912002";
    private static Connection myConnection;
    private static PreparedStatement myPreparedStatement;
    private static ResultSet myResultSet;
    private static ResultSetMetaData myMetaData;

    public BranchInfo() {
        initComponents();
        setVisible(true);

        setLocationRelativeTo(null);
        setResizable(false);

        mainPanel.setBackground(new Color(666666));

        tableNameLabel.setForeground(Color.WHITE);
```

```
tableNameLabel.setHorizontalAlignment(SwingConstants.CENTER);
```

```
text1.setForeground(Color.WHITE);
```

```
initializeBranchList();
```

```
fillBranchInfo();
```

```
fillWorkerInfo();
```

```
fillTotalSalaries();
```

```
}
```

```
private void initializeBranchList(){
```

```
    try {
```

```
        myConnection = DriverManager.getConnection(DB_URL, USERNAME, PASSWORD);
```

```
        myPreparedStatement = myConnection.prepareStatement("SELECT br_code FROM branch");
```

```
        myResultSet = myPreparedStatement.executeQuery();
```

```
        while(myResultSet.next())
```

```
            branchSelection.addItem(myResultSet.getObject(1).toString());
```

```
    }
```

```
    catch (SQLException ex) {
```

```
    }
```

```
    finally{
```

```
    }
```

```
}
```

```
private void fillBranchInfo(){
```

```
    String GET_BRANCH_INFO_QUERY =
```

```
        "SELECT br_code, br_street, br_num, br_city, wrk_name, wrk_lname, SUM(Reservations) AS  
'Reservations', SUM(Profit) " +
```

```
        "FROM( " +
```

```
        "        SELECT br_code, br_street, br_num, br_city, wrk_name, wrk_lname, COUNT(*) AS  
'Reservations', COUNT(*) * tr_cost AS 'Profit' " +
```

```
        "        FROM branch " +
```

```

"        LEFT JOIN manages ON br_code = mng_br_code " +
"        LEFT JOIN admin ON mng_adm_AT = adm_AT " +
"        LEFT JOIN worker ON adm_AT = wrk_AT " +
"        LEFT JOIN trip ON br_code = tr_br_code " +
"        LEFT JOIN reservation ON tr_id = res_tr_id " +
"        GROUP BY tr_id " +
") AS temp " +
"WHERE br_code = " + branchSelection.getSelectedItemAt().toString();

```

```

try {
    myConnection = DriverManager.getConnection(DB_URL, USERNAME, PASSWORD);

    myPreparedStatement = myConnection.prepareStatement(GET_BRANCH_INFO_QUERY);
    myResultSet = myPreparedStatement.executeQuery();
    myMetaData = myResultSet.getMetaData();

    DefaultTableModel tModel = (DefaultTableModel) branchInfo.getModel();
    tModel.setColumnCount(0);
    tModel.setRowCount(0);

    String[] branchArr = new String[myMetaData.getColumnCount()];

    for(int i = 1; i <= myMetaData.getColumnCount(); i++)
        tModel.addColumn(myMetaData.getColumnName(i));

    myResultSet.next();

    for(int i = 0; i < myMetaData.getColumnCount(); i++){
        if(myResultSet.getObject(i + 1) == null)
            branchArr[i] = "NULL";
        else
            branchArr[i] = myResultSet.getObject(i + 1).toString();
    }
}

```

```

        tModel.addRow(branchArr);
    }
    catch (SQLException ex) {
        ex.printStackTrace();
    }
}

private void fillWorkerInfo(){
    String GET_BRANCH_WORKER_INFO_QUERY =
        "SELECT br_code, wrk_name, wrk_lname, wrk_salary " +
        "FROM branch " +
        "INNER JOIN worker ON br_code = wrk_br_code " +
        "WHERE br_code = " + branchSelection.getSelectedItem().toString();

    try {
        myConnection = DriverManager.getConnection(DB_URL, USERNAME, PASSWORD);

        myPreparedStatement = myConnection.prepareStatement(GET_BRANCH_WORKER_INFO_QUERY);
        myResultSet = myPreparedStatement.executeQuery();
        myMetaData = myResultSet.getMetaData();

        DefaultTableModel tModel = (DefaultTableModel) workerInfo.getModel();
        tModel.setColumnCount(0);
        tModel.setRowCount(0);

        for(int i = 1; i <= myMetaData.getColumnCount(); i++)
            tModel.addColumn(myMetaData.getColumnName(i));

        while(myResultSet.next()){
            String[] workerArr = new String[myMetaData.getColumnCount()];

            for(int i = 0; i < myMetaData.getColumnCount(); i++){
                if(myResultSet.getObject(i + 1) == null)
                    workerArr[i] = "NULL";
            }
        }
    }
}

```

```

        else

            workerArr[i] = myResultSet.getObject(i + 1).toString();

        }

        tModel.addRow(workerArr);

    }

}

catch (SQLException ex) {

    ex.printStackTrace();

}

}

private void fillTotalSalaries(){

    String GET_TOTAL_SALARIES_QUERY =

        "SELECT SUM(wrk_salary) AS 'Total Salary' " +

        "FROM branch " +

        "INNER JOIN worker ON br_code = wrk_br_code " +

        "WHERE br_code = " + branchSelection.getSelectedItem().toString();

    try {

        myConnection = DriverManager.getConnection(DB_URL, USERNAME, PASSWORD);

        myPreparedStatement = myConnection.prepareStatement(GET_TOTAL_SALARIES_QUERY);

        myResultSet = myPreparedStatement.executeQuery();

        myMetaData = myResultSet.getMetaData();

        DefaultTableModel tModel = (DefaultTableModel) totalSalaries.getModel();

        tModel.setColumnCount(0);

        tModel.setRowCount(0);

        String[] salaryArr = new String[myMetaData.getColumnCount()];

        myResultSet.next();

        for(int i = 0; i < myMetaData.getColumnCount(); i++){

```

```

        if(myResultSet.getObject(i + 1) == null)
            salaryArr[i] = "NULL";
        else
            salaryArr[i] = myResultSet.getObject(i + 1).toString();
    }

    tModel.addColumn(myMetaData.getColumnName(1));
    tModel.addRow(salaryArr);
}
catch (SQLException ex) {
    ex.printStackTrace();
}
}

```

```
@SuppressWarnings("unchecked")
```

```
// <editor-fold defaultstate="collapsed" desc="Generated Code">
```

```
private void initComponents() {
```

```

    mainPanel = new javax.swing.JPanel();
    tableNameLabel = new javax.swing.JLabel();
    jScrollPane1 = new javax.swing.JScrollPane();
    branchInfo = new javax.swing.JTable();
    jScrollPane2 = new javax.swing.JScrollPane();
    workerInfo = new javax.swing.JTable();
    branchSelection = new javax.swing.JComboBox<>();
    text1 = new javax.swing.JLabel();
    jScrollPane3 = new javax.swing.JScrollPane();
    totalSalaries = new javax.swing.JTable();
    closeButton = new javax.swing.JButton();

```

```
setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
```

```
tableNameLabel.setFont(new java.awt.Font("Segoe UI", 1, 24)); // NOI18N
```

```
tableNameLabel.setText("Branch Info");
```

```
branchInfo.setRowHeight(25);
```

```
jScrollPane1.setViewportView(branchInfo);
```

```
jScrollPane2.setViewportView(workerInfo);
```

```
branchSelection.addPopupMenuListener(new javax.swing.event.PopupMenuListener() {  
    public void popupMenuCanceled(javax.swing.event.PopupMenuEvent evt) {  
    }  
    public void popupMenuWillBecomeInvisible(javax.swing.event.PopupMenuEvent evt) {  
        branchSelectionPopupMenuWillBecomeInvisible(evt);  
    }  
    public void popupMenuWillBecomeVisible(javax.swing.event.PopupMenuEvent evt) {  
    }  
});
```

```
text1.setText("Select branch:");
```

```
jScrollPane3.setViewportView(totalSalaries);
```

```
closeButton.setText("Main Menu");
```

```
closeButton.addMouseListener(new java.awt.event.MouseAdapter() {  
    public void mouseClicked(java.awt.event.MouseEvent evt) {  
        closeButtonMouseClicked(evt);  
    }  
});
```

```
javax.swing.GroupLayout mainPanelLayout = new javax.swing.GroupLayout(mainPanel);
```

```
mainPanel.setLayout(mainPanelLayout);
```

```
mainPanelLayout.setHorizontalGroup(  
    mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
```

```
        .addGroup(mainPanelLayout.createSequentialGroup()  
            .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
```

```
                .addGroup(mainPanelLayout.createSequentialGroup()  
                    .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING)
```

```
                        .addGroup(mainPanelLayout.createSequentialGroup()  
                            .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
```

```

        .addGroup(mainPanelLayout.createSequentialGroup())

        .addGap(75, 75, 75)

        .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

            .addComponent(text1, javax.swing.GroupLayout.PREFERRED_SIZE, 84,
                javax.swing.GroupLayout.PREFERRED_SIZE)

            .addComponent(branchSelection, javax.swing.GroupLayout.PREFERRED_SIZE, 84,
                javax.swing.GroupLayout.PREFERRED_SIZE)

            .addComponent(jScrollPane3, javax.swing.GroupLayout.PREFERRED_SIZE, 89,
                javax.swing.GroupLayout.PREFERRED_SIZE))

        .addGap(90, 90, 90)

        .addComponent(jScrollPane2, javax.swing.GroupLayout.PREFERRED_SIZE, 692,
            javax.swing.GroupLayout.PREFERRED_SIZE))

        .addGroup(mainPanelLayout.createSequentialGroup())

        .addGap(0, 0, Short.MAX_VALUE)

        .addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE, 881,
            javax.swing.GroupLayout.PREFERRED_SIZE)))

        .addGap(0, 65, Short.MAX_VALUE))

    .addGroup(mainPanelLayout.createSequentialGroup())

    .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

        .addGroup(mainPanelLayout.createSequentialGroup())

        .addGap(400, 400, 400)

        .addComponent(closeButton, javax.swing.GroupLayout.PREFERRED_SIZE, 160,
            javax.swing.GroupLayout.PREFERRED_SIZE))

    .addGroup(mainPanelLayout.createSequentialGroup())

    .addGap(341, 341, 341)

    .addComponent(tableNameLabel, javax.swing.GroupLayout.PREFERRED_SIZE, 285,
        javax.swing.GroupLayout.PREFERRED_SIZE)))

    .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))
);

mainPanelLayout.setVerticalGroup(

    mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

    .addGroup(mainPanelLayout.createSequentialGroup())

    .addGap(24, 24, 24)

    .addComponent(tableNameLabel, javax.swing.GroupLayout.PREFERRED_SIZE, 47,
        javax.swing.GroupLayout.PREFERRED_SIZE)

    .addGap(27, 27, 27)

    .addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE, 51,
        javax.swing.GroupLayout.PREFERRED_SIZE)

```



```

        .addGroup(mainPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
false)

        .addGroup(mainPanelLayout.createSequentialGroup())

        .addGap(46, 46, 46)

        .addComponent(jScrollPane2, javax.swing.GroupLayout.PREFERRED_SIZE, 199,
javax.swing.GroupLayout.PREFERRED_SIZE))

        .addGroup(mainPanelLayout.createSequentialGroup())

        .addGap(30, 30, 30)

        .addComponent(text1)

        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)

        .addComponent(branchSelection, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)

        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)

        .addComponent(jScrollPane3, javax.swing.GroupLayout.PREFERRED_SIZE, 51,
javax.swing.GroupLayout.PREFERRED_SIZE)))

        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)

        .addComponent(closeButton)

        .addContainerGap(13, Short.MAX_VALUE)

    );

    javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
    getContentPane().setLayout(layout);

    layout.setHorizontalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

        .addComponent(mainPanel, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)

    );

    layout.setVerticalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

        .addComponent(mainPanel, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)

    );

    pack();
}

```

```

private void branchSelectionPopupMenuWillBecomeInvisible(javax.swing.event.PopupMenuEvent evt) {
    fillBranchInfo();
    fillWorkerInfo();
    fillTotalSalaries();
}

private void closeButtonMouseClicked(java.awt.event.MouseEvent evt) {
    MainMenu mainMen = new MainMenu();
    dispose();
}

public static void main(String args[]) {
    /* Set the Nimbus look and feel */
    //<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
    /* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
    * For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
    */
    try {
        for (javax.swing.UIManager.LookAndFeelInfo info : javax.swing.UIManager.getInstalledLookAndFeels()) {
            if ("Nimbus".equals(info.getName())) {
                javax.swing.UIManager.setLookAndFeel(info.getClassName());
                break;
            }
        }
    } catch (ClassNotFoundException ex) {
        java.util.logging.Logger.getLogger(BranchInfo.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
    } catch (InstantiationException ex) {
        java.util.logging.Logger.getLogger(BranchInfo.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
    } catch (IllegalAccessException ex) {
        java.util.logging.Logger.getLogger(BranchInfo.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
    } catch (javax.swing.UnsupportedLookAndFeelException ex) {
        java.util.logging.Logger.getLogger(BranchInfo.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
    }
}

```

```
}
```

```
//</editor-fold>
```

```
java.awt.EventQueue.invokeLater(new Runnable() {
```

```
    public void run() {
```

```
        new BranchInfo().setVisible(true);
```

```
    }
```

```
});
```

```
}
```

```
// Variables declaration - do not modify
```

```
private javax.swing.JTable branchInfo;
```

```
private javax.swing.JComboBox<String> branchSelection;
```

```
private javax.swing.JButton closeButton;
```

```
private javax.swing.JScrollPane jScrollPane1;
```

```
private javax.swing.JScrollPane jScrollPane2;
```

```
private javax.swing.JScrollPane jScrollPane3;
```

```
private javax.swing.JPanel mainPanel;
```

```
private javax.swing.JLabel tableNameLabel;
```

```
private javax.swing.JLabel text1;
```

```
private javax.swing.JTable totalSalaries;
```

```
private javax.swing.JTable workerInfo;
```

```
// End of variables declaration
```

```
}
```